

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH

KHOA ĐIỆN – ĐIỆN TỬ

BỘ MÔN KỸ THUẬT MÁY TÍNH – VIỄN THÔNG



HCM-UTE

ĐỒ ÁN TỐT NGHIỆP

**THIẾT KẾ VÀ ĐÁNH GIÁ PHẦN CỨNG
TĂNG TỐC THUẬT TOÁN MẬT MÃ HẠU LƯỢNG TỬ
CRYSTALS-KYBER TÍCH HỢP TRÊN
VI ĐIỀU KHIỂN LỖI RISC-V**

GVHD: PGS.TS. ĐỖ DUY TÂN

SVTH : PHẠM QUANG HỢP

MSSV: 22119178

ĐẶNG VĨNH TIẾN

MSSV: 22119239

TP. Hồ Chí Minh, tháng 6 năm 2026

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH

KHOA ĐIỆN – ĐIỆN TỬ

BỘ MÔN KỸ THUẬT MÁY TÍNH – VIỄN THÔNG



HCM-UTE

ĐỒ ÁN TỐT NGHIỆP

**THIẾT KẾ VÀ ĐÁNH GIÁ PHẦN CỨNG
TĂNG TỐC THUẬT TOÁN MẬT MÃ HẠ LƯỢNG TỬ
CRYSTALS-KYBER TÍCH HỢP TRÊN
VI ĐIỀU KHIỂN LỖI RISC-V**

GVHD: PGS.TS. ĐỖ DUY TÂN

SVTH : PHẠM QUANG HỢP

MSSV: 22119178

ĐẶNG VĨNH TIẾN

MSSV: 22119239

TP. Hồ Chí Minh, tháng 6 năm 2026

TRANG THÔNG TIN KHOÁ LUẬN TỐT NGHIỆP

1. Thông tin sinh viên

Họ và tên sinh viên: Phạm Quang Hợp	MSSV: 22119178
Email: 22119178@student.hcmute.edu.vn	Điện thoại: 0359213278
Họ và tên sinh viên: Đặng Vĩnh Tiến	MSSV: 22119239
Email: 22119239@student.hcmute.edu.vn	Điện thoại: 0817837903

2. Thông tin đề tài

Tên đề tài: **“Thiết kế và đánh giá phần cứng tăng tốc thuật toán mật mã hậu lượng tử Crystal-Kyber tích hợp trên vi điều khiển lõi RISC-V”.**

Đơn vị quản lý: Bộ môn Kỹ Thuật Máy Tính – Viễn thông, Khoa Điện - Điện Tử, Trường Đại học Công nghệ Kỹ Thuật Thành phố Hồ Chí Minh.

Thời gian thực hiện: Từ ngày 01/03/2026 đến ngày 26/06/2026.

Thời gian bảo vệ trước hội đồng: Ngày 03/07/2026.

3. Lời cam đoan của sinh viên

Nhờ sự hướng dẫn của PGS.TS. Đỗ Duy Tân, nhóm thành viên Phạm Quang Hợp và Đặng Vĩnh Tiến đã hoàn thành thành công khóa luận tốt nghiệp này. Chúng em xin đảm bảo kết quả có được này hoàn toàn nhờ quá trình làm việc nghiêm túc của nhóm và chắc chắn không có bất kỳ gian lận nào sao chép các đồ án khác.

Tp. Hồ Chí Minh, ngày 26 tháng 06 năm 2026

Sinh viên thực hiện đồ án

(Ký và ghi rõ họ tên)

Phạm Quang Hợp

Đặng Vĩnh Tiến

BẢN NHẬN XÉT KHOÁ LUẬN TỐT NGHIỆP

(Dành cho giảng viên hướng dẫn)

Đề tài: **THIẾT KẾ VÀ ĐÁNH GIÁ PHẦN CỨNG TĂNG TỐC THUẬT TOÁN MẬT MÃ HẠU LƯỢNG TỬ CRYSTALS-KYER TÍCH HỢP TRÊN VI ĐIỀU KHIỂN RISC-V**

Sinh viên: + Phạm Quang Hợp

MSSV: 22119178

+ Đặng Vĩnh Tiến

MSSV: 22119239

Hướng dẫn: PGS.TS. Đỗ Duy Tân

Nhận xét bao gồm các nội dung sau đây:

1. Tính hợp lý trong cách đặt vấn đề và giải quyết vấn đề; ý nghĩa khoa học và thực tiễn:

Đặt vấn đề rõ ràng, mục tiêu cụ thể; đề tài có tính mới, cấp thiết; đề tài có khả năng ứng dụng, tính sáng tạo.

Đặt vấn đề rõ ràng và mục tiêu cụ thể, đề tài có khả năng ứng dụng.

2. Phương pháp thực hiện/ phân tích/ thiết kế:

Phương pháp hợp lý và tin cậy dựa trên cơ sở lý thuyết; có phân tích và đánh giá phù hợp; có tính mới và tính sáng tạo.

Phương pháp thực hiện hợp lý, thiết kế đáng tin cậy dựa trên cơ sở lý thuyết và có phân tích.

3. Kết quả thực hiện/ phân tích và đánh giá kết quả/ kiểm định thiết kế:

Phù hợp với mục tiêu đề tài; phân tích và đánh giá/kiểm thử thiết kế hợp lý; có tính sáng tạo/kiểm định chặt chẽ và đảm bảo độ tin cậy.

Kết quả thực hiện phù hợp với mục tiêu đề ra, thực hiện phân tích, đánh giá, kiểm thử các tính năng của thiết kế với độ tin cậy cao.

4. Kết luận và đề xuất:

Kết luận phù hợp với đặt vấn đề, đề xuất mang tính cải tiến và thực tiễn; kết luận có đóng góp mới mẻ, đề xuất sáng tạo và thuyết phục.

Kết luận phù hợp với cách đặt vấn đề.

5. Hình thức trình bày và bố cục báo cáo:

Văn phong nhất quán, bố cục hợp lý, cấu trúc rõ ràng, đúng định dạng mẫu; có tính hấp dẫn, thể hiện năng lực tốt, văn bản trau chuốt.

Văn phong nhất quán và bố cục trình bày hợp lý, cấu trúc rõ ràng, diễn đạt tốt.

6. Kỹ năng chuyên nghiệp và tính sáng tạo:

Thể hiện các kỹ năng giao tiếp, kỹ năng làm việc nhóm, và các kỹ năng chuyên nghiệp khác trong việc thực hiện đề tài.

Có năng lực giải quyết vấn đề và kỹ năng giao tiếp tốt.

7. Tài liệu trích dẫn

Tính trung thực trong việc trích dẫn tài liệu tham khảo; tính phù hợp của các tài liệu trích dẫn; trích dẫn theo đúng chỉ dẫn APA.

Tài liệu trích dẫn phù hợp.

8. Đánh giá về sự trùng lặp của đề tài

Cần khẳng định đề tài có trùng lặp hay không? Nếu có, đề nghị ghi rõ mức độ, tên đề tài, nơi và năm công bố của đề tài đã công bố.

Kết quả được kiểm tra với công cụ Turnitin: 14% trùng lặp.

9. Những nhược điểm và thiếu sót, những điểm cần được bổ sung và chỉnh sửa*

- Giải thích rõ hơn kết quả đánh giá qua mô phỏng

- Kiểm tra các lỗi đánh máy

10. Nhận xét tinh thần, thái độ học tập, nghiên cứu của sinh viên

Sinh viên có tinh thần trách nhiệm cao, có thái độ học tập và làm việc tốt.

Đề nghị của giảng viên hướng dẫn

Ghi rõ: "Báo cáo đạt/ không đạt yêu cầu của một khóa luận tốt nghiệp kỹ sư, được phép/ không được phép bảo vệ khóa luận tốt nghiệp"

Báo cáo **đạt yêu cầu** của một KLTN kỹ sư và **được phép bảo vệ** KLTN.

Tp. HCM, ngày 25 tháng 06 năm 2024

Người nhận xét


Đỗ Duy Tân

* Giảng viên hướng dẫn có thể ghi tiếp vào mặt sau

LỜI CẢM ƠN

Lời đầu tiên, chúng tôi xin chân thành cảm ơn quý thầy cô Trường Đại học Công nghệ Kỹ thuật Thành phố Hồ Chí Minh nói chung, cụ thể hơn là quý thầy cô trong Khoa Điện - Điện tử và Bộ môn Kỹ thuật Máy tính - Viễn thông nói riêng, đã đầu tư công sức giảng dạy, truyền đạt kiến thức và tạo nhiều điều kiện tốt cho chúng tôi trong toàn bộ quá trình học tập và làm đồ án tốt nghiệp.

Đặc biệt, chúng tôi bày tỏ sự biết ơn sâu sắc thầy PGS.TS. Đỗ Duy Tân, người thầy đã tận tình hướng dẫn, định hướng, hỗ trợ, góp ý và theo sát chúng tôi hoàn thiện nghiên cứu, thiết kế của đề tài. Nhờ những chỉ dẫn không chỉ về kiến thức chuyên môn, kinh nghiệm nghiên cứu mà còn nhờ sự dìu dắt, động viên của thầy, chúng tôi có thể nỗ lực từng bước giải quyết các vấn đề và hoàn thành đồ án này.

Cũng không quên gửi lời cảm ơn đến gia đình, bạn bè và những người thân yêu đã luôn hỗ trợ, ủng hộ và tạo điều kiện tốt nhất, chúng tôi mới có thể có điều kiện học tập, yên tâm nghiên cứu và hoàn thành đồ án. Chính sự quan tâm, khích lệ đó hình thành động lực lớn khiến chúng tôi vượt qua những khó khăn, rào cản trong thời gian hoàn thành đồ án.

Mặc dù với tinh thần nghiêm túc, trách nhiệm và nỗ lực không ngừng nghỉ, thì kinh nghiệm và kiến thức chúng tôi vẫn còn hạn chế, đồ án không tránh khỏi những thiếu sót. Chúng tôi rất mong và chân trọng nhận được sự góp ý của quý thầy cô để bù đắp thiếu sót cho đề tài được hoàn thiện hơn.

Lời kết, chúng tôi xin chân thành gửi lời cảm ơn đến tất cả!

TP. Hồ Chí Minh, ngày 26 tháng 06 năm 2026

Sinh viên thực hiện

Phạm Quang Hợp

Đặng Vĩnh Tiến

TÓM TẮT

Thời đại mà máy tính lượng tử ngày càng phát triển và có thể phá vỡ các hệ mật mã truyền thống đã và đang đến gần, NIST (Viện Tiêu chuẩn và Công nghệ Quốc gia Mỹ) đã tiến hành chuẩn hóa các thuật toán mật mã hậu lượng tử nhằm hướng tới bảo vệ thông tin trước những mối đe dọa mới. Vì vậy, mà việc hiện thực, kiểm định và tối ưu các thuật toán này trên nhiều nền tảng phần cứng có thể nói là một yêu cầu quan trọng cho tương lai. Qua nhiều vòng thử nghiệm, đánh giá, CRYSTALS-Kyber là một thuật toán trao đổi khóa an toàn đã nhận được nhiều sự quan tâm từ cộng đồng nghiên cứu và được NIST chuẩn hoá vào năm 2024. Với nhiều hướng tiếp cận nhằm cải thiện tốc độ xử lý, cân bằng với sử dụng tài nguyên và mức tiêu thụ năng lượng. Nên việc xây dựng một kiến trúc vừa đạt hiệu năng cao, vừa sử dụng tài nguyên hợp lý là một yêu cầu bắt buộc cho những thiết bị có diện tích và bộ nhớ hạn chế.

Đồ án này tập trung triển khai một kiến trúc duy nhất cho CRYSTALS-Kyber, để thiết kế có thể hỗ trợ đầy đủ chế độ hoạt động gồm sinh khóa, đóng gói khoá và giải gói khoá. Kiến trúc này chia sẻ và tái sử dụng các khối xử lý chính giữa ba chế độ, góp phần giảm diện tích mà vẫn bảo đảm hiệu quả. Các thành phần quan trọng được tái sử dụng như phép biến đổi NTT phục vụ nhân đa thức tốc độ cao và lõi SHA3 dùng cho các hàm băm được tập trung thực hiện và tối ưu.

Thiết kế lõi tăng tốc sau đó cũng được tích hợp vào một hệ thống vi điều khiển được thiết kế sử dụng CPU RISC-V mã nguồn mở với bộ lệnh RV32I để chứng minh lõi Kyber được thiết kế có thể hoạt động ổn định trong các hệ thống nhúng nhỏ gọn. Kết quả của đề tài được đánh giá thông qua các tiêu chí như số chu kỳ thực thi, tài nguyên phần cứng tiêu tốn, cũng như độ tương đương chức năng so với mô hình phần mềm tham chiếu. Các kết quả cũng được so sánh với các công trình khác.

Thành quả cho thấy việc triển khai CRYSTALS-Kyber trên phần cứng là khả thi, qua đó minh chứng tiềm năng áp dụng các thuật toán mật mã hậu lượng tử vào các hệ thống nhúng và vi điều khiển tích hợp bảo mật lượng tử trong tương lai.

MỤC LỤC

LỜI CẢM ƠN	i
TÓM TẮT	ii
MỤC LỤC	iii
DANH MỤC HÌNH ẢNH.....	vi
DANH MỤC BẢNG.....	ix
DANH MỤC TỪ VIẾT TẮT.....	x
CHƯƠNG 1 GIỚI THIỆU ĐỀ TÀI.....	1
1.1. LÝ DO CHỌN ĐỀ TÀI	1
1.2. MỤC TIÊU CỦA ĐỀ TÀI	2
1.3. PHẠM VI NGHIÊN CỨU	2
1.4. BỐ CỤC CỦA ĐỒ ÁN	3
CHƯƠNG 2 CƠ SỞ LÝ THUYẾT	4
2.1. TỔNG QUAN VỀ MÃ HOÁ.....	4
2.2. TỔNG QUAN VỀ MẬT MÃ HẠU LƯỢNG TỬ	5
2.3. GIẢI THUẬT CỦA CRYSTALS-KYBER.....	6
2.3.1. Các đặc điểm chính trong CRYSTALS-Kyber	6
2.3.2. Các thành phần lõi của CRYSTALS-Kyber.....	6
2.3.3. Các khái niệm nền tảng và ký hiệu trong CRYSTALS-Kyber	7
2.3.4. Các bước hoạt động của CRYSTALS-Kyber	12
2.4. VI ĐIỀU KHIỂN (MCU).....	21
2.4.1. Bộ xử lý trung tâm - lõi RISC-V	22
2.4.2. GPIO (General-Purpose Input/Output)	23
2.4.3. I2C (Inter-Integrated Circuit)	24
2.4.4. Interrupt	25

2.4.5. Timer	25
2.4.6. UART (Universal Asynchronous Receiver Transmitter)	26
2.4.7. Bus Wishbone	27
CHƯƠNG 3 THIẾT KẾ VÀ TRIỂN KHAI ĐỀ TÀI.....	29
3.1. VI ĐIỀU KHIỂN RISC-V	29
3.1.1. Kiến trúc vi xử lý	30
3.1.2. Kiến trúc bộ nhớ	32
3.1.3. Hệ thống Bus (Interconnect)	33
3.1.4. GPIO	34
3.1.5. TIMER	36
3.1.6. UART	39
3.1.7. Interrupt Controller	43
3.1.8. I2C	45
3.2. KYBER IP ACCELERATOR	47
3.2.1. Kiến trúc tổng quan của phần cứng CRYSTALS-Kyber	48
3.2.2. Thiết kế Hash Core dựa trên SHA3	49
3.2.3. Thiết kế phần cứng lấy mẫu đồng nhất	53
3.2.4. Thiết kế phần cứng lấy mẫu phân phối nhị thức trung tâm	54
3.2.5. Thiết kế phần cứng NTT/INTT	56
3.2.6. Thiết kế phần cứng Pointwise Multiply Accumulate	63
3.2.7. Thiết kế phần cứng Unified Add/Sub mod q	65
3.2.8. Thiết kế phần cứng khối Encode, Decode, Compress, Decompress	66
3.2.9. Giao diện Wishbone Bus	68
CHƯƠNG 4 KIỂM TRA VÀ ĐÁNH GIÁ KẾT QUẢ.....	71
4.1. BOOTLOADER, THƯ VIỆN C VÀ CHƯƠNG TRÌNH NẠP	71

4.1.1. Thiết kế Bootloader	71
4.1.2. Thư viện lập trình	72
4.1.3. Chương trình nạp firmware	72
4.2. KIỂM TRA TOÀN BỘ HỆ THỐNG	74
4.2.1. Kiểm tra lỗi RISC-V	74
4.2.2. Kiểm tra GPIO	77
4.2.3. Kiểm tra TIMER	78
4.2.4. Kiểm tra UART	81
4.2.5. Kiểm tra Kyber Accelerator	83
4.2.6. Kiểm tra toàn bộ vi điều khiển	86
4.3. HIỆN THỰC TOÀN BỘ HỆ THỐNG TRÊN FPGA, KIỂM TRA VÀ ĐÁNH GIÁ KẾT QUẢ	88
4.3.1. Kết quả tổng hợp	88
4.3.2. Hiện thực trên FPGA	90
4.3.3. Đánh giá phần cứng Kyber và so sánh với các nghiên cứu tiên tiến khác trên FPGA	97
CHƯƠNG 5 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	99
5.1. KẾT LUẬN	99
5.2. Hướng phát triển	99
TÀI LIỆU THAM KHẢO	101

DANH MỤC HÌNH ẢNH

Hình 2.1. Mô hình mã hoá đối xứng	4
Hình 2.2. Mô hình mã hoá bất đối xứng	4
Hình 2.3. Quá trình tạo shared key dùng chung giữa Sender và Receiver	12
Hình 2.4. Sơ đồ mô tả kiến trúc vi điều khiển 8051	22
Hình 2.5. Waveform giao tiếp của I2C	24
Hình 2.6. Waveform của giao tiếp UART	26
Hình 2.7. Minh họa kết nối giữa Master và Slave	27
Hình 2.8.a) Quá trình Read của Wishbone	28
Hình 2.8.b) Quá trình Write của Wishbone	28
Hình 3.1. Kiến trúc tổng quan hệ thống SoC	29
Hình 3.2. Kiến trúc lõi RISC-V RV32I	30
Hình 3.3. Sơ đồ khối tổng quan GPIO	34
Hình 3.4. Sơ đồ khối TIMER.....	36
Hình 3.5. Sơ đồ khối UART	39
Hình 3.6. Sơ đồ khối I2C	45
Hình 3.7. Kiến trúc tổng quan của phần cứng CRYSTALS-Kyber.....	48
Hình 3.8.a) Kiến trúc phần cứng Hash Unit.....	49
Hình 3.8.b) Phần cứng Keccak	49
Hình 3.9. Kiến trúc tổng quan phần cứng lấy mẫu đồng nhất	54
Hình 3.10. Kiến trúc tổng quan phần cứng lấy mẫu phân phối nhị thức trung tâm..	55
Hình 3.11.a) Butterfly Cooley-Tukey	58
Hình 3.11.b) Butterfly Gentleman-Sande	58
Hình 3.12. Mô hình NTT với $n = 8$	58

Hình 3.13. Kiến trúc tổng quan phần cứng NTT/INTT	59
Hình 3.14. Kiến trúc phần cứng Pipelined Butterfly Unit	61
Hình 3.15. Kiến trúc khối Montgomery Reduce.....	63
Hình 3.16. Kiến trúc khối Barrett Reduce	63
Hình 3.17. Kiến trúc tổng quan phần cứng Pointwise Multiply Accumulate	64
Hình 3.18. Kiến trúc khối Basemul.....	65
Hình 3.19. Kiến trúc phần cứng Unified Add/Sub mod q	65
Hình 3.20. Kiến trúc phần cứng <i>Compressdu</i>	67
Hình 3.21. Kiến trúc phần cứng a) <i>Decompressdu</i> b) Multiplier by Q.....	67
Hình 3.22. Kiến trúc phần cứng Encode ₁	68
Hình 3.23. Kiến trúc phần cứng Decode ₁	68
Hình 3.24. Sơ đồ Wishbone Bus và Kyber Accelerator	69
Hình 4.1. Sơ đồ hoạt động khi reset hoặc bật vi điều khiển	71
Hình 4.2. Giao diện chương trình nạp firmware	73
Hình 4.3. Mô phỏng các lệnh LUI, R-TYPE, I-TYPE, JUMP, BRANCH, LOAD và STORE	75
Hình 4.4. Kết quả mô phỏng GPIO	78
Hình 4.5. Kết quả mô phỏng TIMER.....	79
Hình 4.6. Mô phỏng truyền/nhận UART	82
Hình 4.7. Mô hình kiểm tra Kyber Accelerator với Golden Model.....	83
Hình 4.8. Kết quả bài test KAT (Known Answer Test) thứ 1	85
Hình 4.9. Kết quả bài test KAT (Known Answer Test) thứ 10,000	86
Hình 4.10. Kết quả mô phỏng toàn bộ hệ thống	87
Hình 4.11. Tổng công suất tiêu thụ của thiết kế	90
Hình 4.12. Mô hình triển khai hệ thống trên bo mạch FPGA.....	91

Hình 4.13. Lưu đồ hoạt động của firmware điều khiển FPGA.....	93
Hình 4.14. Kết quả thực nghiệm các chức năng ngoại vi trên FPGA.....	94
Hình 4.15. Kết quả thực nghiệm KeyGen trên FPGA	95
Hình 4.16. Kết quả thực nghiệm Encaps trên FPGA	96
Hình 4.17. Kết quả thực nghiệm Decaps trên FPGA.....	97

DANH MỤC BẢNG

Bảng 2.1: Các tham số của CRYSTALS-Kyber KEM	6
Bảng 3.1: Phân vùng bộ nhớ	32
Bảng 3.2: Sơ đồ ánh xạ địa chỉ (Address Map)	33
Bảng 3.3: Các tín hiệu chính của giao thức Wishbone	34
Bảng 3.4: Địa chỉ các thanh ghi của GPIO0 và GPIO1	35
Bảng 3.5: Địa chỉ các thanh ghi của TIMER	37
Bảng 3.6: Địa chỉ các thanh ghi của UART	40
Bảng 3.7: Địa chỉ các thanh ghi của Interrupt Controller	44
Bảng 3.8: Địa chỉ các thanh ghi của I2C.....	46
Bảng 3.9: Các chế độ hoạt động của phần cứng Hash Unit.....	50
Bảng 3.10: Địa chỉ các thanh ghi của Kyber.....	69
Bảng 4.1: Tóm tắt testcase kiểm tra lỗi RISC-V RV32I.....	75
Bảng 4.2: Tóm tắt testcase kiểm tra module GPIO.....	77
Bảng 4.3: Tóm tắt testcase kiểm tra module Timer	79
Bảng 4.4: Tóm tắt testcase kiểm tra module UART_WB.....	81
Bảng 4.5: Tốc độ thực thi trung bình của phần cứng Kyber từ mô phỏng	86
Bảng 4.6: Tóm tắt testcase kiểm tra vi điều khiển	87
Bảng 4.7: Tài nguyên của từng module trong hệ thống trên ZCU102 FPGA	89
Bảng 4.8: Chức năng các DIP switch trong chương trình thử nghiệm	92
Bảng 4.9: Số chu kỳ và hệ số tăng tốc của thuật toán Kyber.....	97
Bảng 4.10. So sánh với các nghiên cứu tiên tiến dựa trên kết quả sau khi triển khai thành công CRYSTALS-Kyber Accelerator trên FPGA	98

DANH MỤC TỪ VIẾT TẮT

STT	Ký hiệu chữ viết tắt	Chữ viết đầy đủ
1	AI	Artificial Intelligence
2	ASIC	Application Specific Integrated Circuit
3	ADP	Area-Delay Product
4	AES	Advanced Encryption Standard
5	ALU	Arithmetic Logic Unit
6	BRAM	Block Random Access Memory
7	CPU	Central Processing Unit
8	ECC	Elliptic Curve Cryptography
9	FPGA	Field-programmable gate array
10	FIFO	First In, First Out
11	FIPS	Federal Information Processing Standards
12	FF	Flip-Flop
13	GPIO	General-Purpose Input/Output
14	IoT	Internet of Things
15	INTT	Inverse Number Theoretic Transform
16	I2C	Inter-Integrated Circuit
17	ISR	Interrupt Service Routine
18	IP	Intellectual Property
19	KEM	Key Encapsulation Mechanism
20	LWE	Learning-With-Errors
21	LUT	Look-Up Table
22	ML	Module Lattice-based
23	NIST	National Institute of Standards and Technology
24	NTT	Number Theoretic Transform
25	PQC	Post-Quantum Cryptography
26	PWMA	Pointwise Multiply Accumulate
27	PDP	Power-Delay Product

28	RSA	Rivest-Shamir-Adleman
29	RISC	Reduced Instruction Set Computer
30	RV32I	RISC-V 32-bit Base Integer Instruction Set
31	RAM	Random Access Memory
32	ROM	Read-Only Memory
33	SoC	System on Chip
34	SHA	Secure Hash Algorithm
35	SRAM	Static Random Access Memory
36	TRNG	True Random Number Generator
37	UART	Universal Asynchronous Receiver Transmitter

CHƯƠNG 1

GIỚI THIỆU ĐỀ TÀI

1.1. LÝ DO CHỌN ĐỀ TÀI

Thời đại mà sự phát triển các lĩnh vực chăm sóc sức khỏe, viễn thông, trí tuệ nhân tạo (AI), và internet vạn vật (IoT),... ngày càng phổ biến thì các mối đe dọa an toàn thông tin cũng gia tăng. Song song là sự phát triển nhanh chóng của máy tính lượng tử, khả năng sẽ vượt qua siêu máy tính trở thành mối đe dọa lớn. Vào năm 1994, Peter Shor, nhà toán học đã công bố thuật toán sử dụng máy tính lượng tử có thể dễ dàng tấn công các thuật toán bảo mật như RSA hay ECC [1]. Qua đó, hành động phát triển thuật toán bảo vệ trước các cuộc tấn công sau này cần được triển khai.

Viện Tiêu chuẩn và Công nghệ Quốc gia Hoa Kỳ (NIST) đã khởi tranh công cuộc chuẩn hoá các thuật toán mật mã hậu lượng tử (PQC) vào năm 2016. NIST đã công nhận năm thuật toán gồm CRYSTALS-Kyber, CRYSTALS-Dilithium, FALCON, SPHINCS+ và HQC. Mà các thuật toán này đều xây dựng dựa vào bài toán phức tạp như lý thuyết lưới (lattice-based) và hàm băm (Hash-based) với hàm lượng toán và độ phức tạp cao. Trong số đó, chúng tôi chọn CRYSTALS-Kyber, thuật toán cho trao đổi khoá bí mật an toàn, đã và đang được cộng đồng học thuật hiện thực phần cứng và liên tục tối ưu giảm thời gian xử lý và tài nguyên chiếm hữu.

Nhờ sức hút lớn, mà hiện nay có nhiều nghiên cứu triển khai Kyber ra phần cứng với nhiều hướng tiếp cận khác nhau như ‘đồng thiết kế phần cứng – phần mềm’ hay ‘phần cứng thuần’ trên các nền tảng RISC-V, ASIC hay FPGA... Các nghiên cứu này đều thể hiện được việc hiện thực phần cứng có thể tăng tốc thuật toán mật mã như Kyber lên nhiều lần so với chỉ sử dụng phần mềm. Đồng thời giảm tiêu thụ năng lượng đáng kể. Theo chúng tôi thấy, vẫn còn nhiều nghiên cứu mặc dù đều chứng minh rằng phần cứng của họ có thể hoạt động tốt nhưng đa số chỉ thực hiện một phần (điểm nghẽn) như lời Hash [2] hay phép biến đổi NTT [3]. Những nghiên cứu thuần phần cứng lại chỉ hoạt động như Client hoặc Server [4] [5] chứ chưa đảm bảo được chỉ cần một phần cứng duy nhất có thể chuyển đổi linh hoạt giữa ba chế độ. Dù cũng

có nhiều triển khai [6] [7] đạt được vai trò, nhưng khả năng cân bằng tốt về diện tích và hiệu suất thực thi (ADP) vẫn cần được tối ưu. Song, việc tiếp tục mở rộng và nghiên cứu theo cách này vẫn đã và đang được tiếp diễn.

Với nhiều nguyên nhân trên, nên khoá luận này nhóm chúng tôi quyết định sẽ làm đề tài “Thiết kế và đánh giá phần cứng tăng tốc thuật toán mật mã hậu lượng tử CRYSTALS-Kyber tích hợp trên vi điều khiển lõi RISC-V” với mục tiêu chia sẻ tài nguyên dùng chung có thể linh hoạt chuyển đổi các chế độ sinh khoá, đóng và giải gói khoá. Đồng thời cân bằng giữa diện tích và hiệu suất để đạt được hiệu quả ADP tốt như mong đợi. Sau đó lõi tăng tốc cũng được tích hợp vào một thiết kế vi điều khiển cụ thể nhằm minh chứng Kyber mà chúng tôi đã thực hiện có thể hoạt động ổn định trong một hệ thống cụ thể, làm tiền đề ứng dụng thực tế trong các hệ thống nhúng, thiết bị IoT nhỏ gọn.

1.2. MỤC TIÊU CỦA ĐỀ TÀI

Mục tiêu của đề tài là thiết kế một vi điều khiển có CPU RISC-V với tập lệnh RV32I, sử dụng Wishbone Bus quản lý giao tiếp các thành phần có trong hệ thống, tích hợp phần cứng tăng tốc CRYSTALS-Kyber dùng ngôn ngữ mô tả phần cứng Verilog. Bên cạnh đó, việc sử dụng các ngôn ngữ lập trình như C và Shell Script cũng được thực hiện để triển khai phần mềm hỗ trợ truyền nhận dữ liệu cho vi điều khiển trong hệ thống kiểm thử, so sánh kết quả mã hoá với bộ tham chiếu là mã nguồn C của Kyber không chỉ qua mô phỏng mà còn đánh giá thực tế trên ZCU102 FPGA.

Phần cứng sau thiết kế có thể đảm bảo có:

- 1) IP mã hoá thực hiện được 3 chế độ: Tạo khoá (Key Generation – Key Gen), đóng gói khoá (Encapsulation – Enc.), giải gói khoá (Decapsulation – Dec.).
- 2) Tích hợp các giao diện ngoại vi tiêu chuẩn như UART, GPIO, TIMER, I2C và xác minh được chức năng thông qua mô phỏng và demo trên FPGA.
- 3) Phần cứng hoạt động với hiệu suất cao và PPA cân bằng tốt.

1.3. PHẠM VI NGHIÊN CỨU

Đề tài chỉ tập trung vào tập lệnh cơ bản RV32I của RISC-V và chỉ sử dụng bus nội bộ là Wishbone để kết nối các thành phần còn lại. Các ngoại vi được giới hạn bao

gồm UART, GPIO, TIMER, I2C và hệ thống ngắt. Bộ tăng tốc sử dụng trong hệ thống là Kyber Accelerator với tham số k là 2. Mà với thuật toán này, k có thể nhận những giá trị 2, 3 và 4 với ba mức độ bảo mật là Kyber-512, Kyber-768 và Kyber-1024 tương ứng. Trong số đó, $k = 4$ đảm bảo mức độ an toàn cao nhất. Nhưng vì kiến thức và khả năng chúng tôi còn hạn chế, nên chúng tôi chỉ tập trung triển khai và đánh giá phần cứng với tham số $k = 2$ trong đề án này.

1.4. BỐ CỤC CỦA ĐỀ ÁN

Báo cáo đề tài gồm 5 chương:

- **Chương 1 – Giới thiệu đề tài:** Ở chương này, chúng tôi nêu rõ lý do chọn đề tài, đối tượng và mục tiêu đề tài. Cùng với phương pháp và phạm vi nghiên cứu cũng như bố cục của đề tài.
- **Chương 2 – Cơ sở lý thuyết:** Chương này, chúng ta sẽ tìm hiểu sơ lược về hai hình thức của các thuật toán mã hoá thông tin, đồng thời lý do cần mã hoá hậu lượng tử cũng được đề cập. Sau đó, chúng tôi giải thích cơ chế hoạt động của thuật toán CRYSTALS-Kyber. Đồng thời cũng sẽ giới thiệu sơ lược về kiến trúc vi điều khiển RISC-V với các ngoại vi cơ bản, được giao tiếp qua Wishbone Bus/MMIO.
- **Chương 3 – Thiết kế và triển khai đề tài:** Trong chương này, chúng tôi trình bày chi tiết sơ đồ khối và mô tả chức năng và hành vi của từng module từ lớn đến nhỏ trong hệ thống.
- **Chương 4 – Kiểm tra và đánh giá kết quả:** Chúng tôi trình bày các bài kiểm tra cho từng module phần cứng và toàn bộ vi điều khiển không chỉ qua mô phỏng mà còn đánh giá chức năng khi triển khai lên FPGA. Sau đó, đánh giá hành vi hệ thống với kịch bản mong đợi, trong đó cũng đảm bảo kết quả mã hoá của bộ tăng tốc cho ra trùng khớp với phần mềm tương ứng. Cuối cùng, tiến hành đánh giá mức độ cải thiện ADP với các nghiên cứu khác.
- **Chương 5 – Kết luận và hướng phát triển:** Chương cuối, chúng tôi đưa ra nhận xét khách quan về kết quả đạt được và mục tiêu phát triển đề tài trong tương lai.

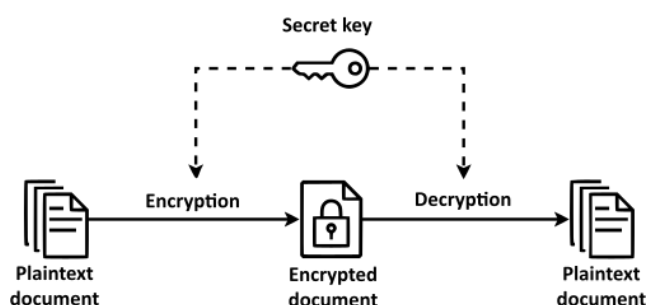
CHƯƠNG 2

CƠ SỞ LÝ THUYẾT

2.1. TỔNG QUAN VỀ MÃ HOÁ

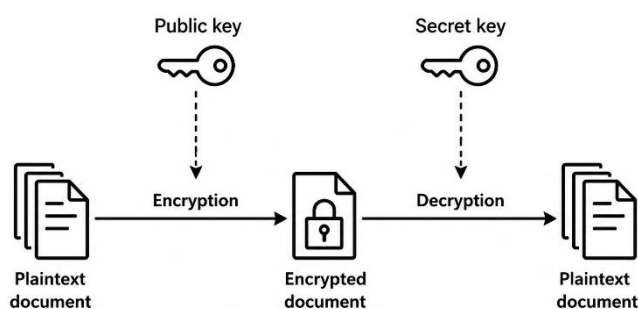
Mã hóa – một phương pháp biến đổi thông tin để đảm bảo rằng chỉ có các bên liên quan mới có thể giải mã dữ liệu đã được mã hoá với khoá mật mã đã được bắt tay dùng chung giữa hai bên. Do đó, chỉ những người có đúng khóa tương ứng mới có thể khôi phục và đọc được nội dung ban đầu. Về cơ bản, mã hóa được chia thành hai nhóm chính là mã hóa đối xứng và mã hóa bất đối xứng.

Hình 2.1 minh họa mô hình mã hóa đối xứng, mô hình chỉ sử dụng một khóa duy nhất cho cả mã hóa và giải mã. Vì vậy mà người gửi và người nhận cần có được khóa này thông qua một kênh an toàn trước khi trao đổi thông tin.



Hình 2.1. Mô hình mã hoá đối xứng

Hình 2.2 lại thể hiện mô hình mã hóa bất đối xứng, mô hình sử dụng hai khóa riêng biệt. Cụ thể, khóa công khai được dùng để mã hóa dữ liệu, còn khóa bí mật được sử dụng để giải mã và khôi phục lại nội dung ban đầu.



Hình 2.2. Mô hình mã hoá bất đối xứng

2.2. TỔNG QUAN VỀ MẬT MÃ HẬU LƯỢNG TỬ

Điện toán lượng tử đang tạo ra một nguy cơ lớn đối với các hệ thống mật mã truyền thống, bởi loại máy tính này có khả năng giải quyết hiệu quả những bài toán như phân tích số nguyên ra thừa số nguyên tố và logarit rời rạc. Mà đây vốn là các phương cách an toàn của hệ mật mã phổ biến như RSA và ECC. Thuật toán Shor, một tiêu biểu cho thuật toán lượng tử, có thể tiêu diệt tính an toàn hiện tại khi máy tính lượng tử đủ mạnh xuất hiện trong tương lai. Đứng trước rủi ro đó mà đến năm 2024, NIST đã lựa chọn được các thuật toán mật mã hậu lượng tử (PQC) tiêu biểu có thể chống lại nguy cơ trên sau đây:

- Chức năng đóng gói khoá: CRYSTALS-Kyber, HQC.
- Chức năng chữ ký số: CRYSTALS-Dilithium, FALCON, SPHINCS+.

Về mặt toán học, các thuật toán PQC dựa trên những bài toán toán khó giải, máy tính lượng tử cũng không ngoại lệ. Trong rất nhiều nền tảng nổi bật, mật mã dựa trên mạng lưới (lattice-based), được quan tâm nhiều nhờ có nền tảng lý thuyết chặt chẽ, mức độ an toàn tốt và hiệu năng phù hợp cho triển khai thực tế. Một đại diện tiêu biểu của nhóm thuật toán lattice-based là CRYSTALS-Kyber, một cơ chế đóng gói khóa, Key Encapsulation Mechanism (KEM), được NIST chuẩn hoá qua nhiều vòng lựa chọn cho mật mã hậu lượng tử.

Mật mã dựa trên mạng lưới khai thác độ khó của bài toán trên không gian nhiều chiều. Mà CRYSTALS-Kyber là một lược đồ mã hóa bất đối xứng, trong đó độ an toàn được xây dựng dựa trên bài toán Module Learning With Errors (MLWE), là một dạng mở rộng của bài toán Learning With Errors (LWE). Về bản chất, bài toán LWE liên quan đến việc tìm nghiệm của một hệ phương trình tuyến tính có thêm nhiễu, và chính độ khó này giúp bảo vệ trước các hình thức tấn công trong tương lai. CRYSTALS-Kyber hướng đến việc cung cấp cơ chế mã hóa khóa công khai và trao đổi khóa an toàn có tính bảo mật cao.

Cơ sở lý thuyết sẽ trình bày sau đây của CRYSTALS-Kyber chủ yếu được lấy từ nghiên cứu của Avanzi và các cộng sự [8]. Lược đồ mã hoá này gồm hai phần chính: Kyber.CPAPKE, dùng cho mã hóa khóa công khai, và Kyber.CCAKEM, dùng cho cơ chế đóng gói khóa. Cả hai thành phần đều bao gồm nhiều phép nhân đa thức

có độ phức tạp cao trên trường hữu hạn và hàm băm (Hash) theo chuẩn FIPS-202, do đó các thuật toán và kiến trúc thực thi phải xử lý hiệu quả các phép toán này.

2.3. GIẢI THUẬT CỦA CRYSTALS-KYBER

2.3.1. Các đặc điểm chính trong CRYSTALS-Kyber

CRYSTALS-Kyber được thiết kế để đáp ứng các yêu cầu bao gồm:

- **An toàn:** Chống lại cả tấn công cổ điển và tấn công lượng tử, cung cấp các bảo đảm an toàn mạnh dựa trên MLWE.
- **Hiệu quả:** Cung cấp các phép mã hóa và giải mã nhanh, phù hợp với các môi trường bị giới hạn tài nguyên.
- **Đơn giản:** Có thiết kế dạng module, dễ triển khai cả phần cứng lẫn phần mềm.

Kyber được tham số hóa thành ba mức an toàn khác nhau, mỗi mức có mức hiệu quả khác nhau. Kyber-512 cung cấp mức an toàn 128 bit, Kyber-768 cung cấp mức an toàn 192 bit, và Kyber-1024 cung cấp mức an toàn 256 bit. Bảng 2.1 tóm tắt các tham số được sử dụng ở cả ba mức an toàn. Các tham số này bao gồm kích thước đa thức (k), giá trị modulo (q), tham số nhiễu (η_1, η_2), mức nén (d_u, d_v) và xác suất lỗi (δ), mà tất cả đều ảnh hưởng đến độ an toàn và hiệu quả của thuật toán.

Bảng 2.1: Các tham số của CRYSTALS-Kyber KEM

Version	n	k	q	η_1, η_2	d_u, d_v	Xác suất lỗi (δ)
Kyber-512	256	2	3329	3, 2	10, 4	2^{-139}
Kyber-768	256	3	3329	3, 2	10, 4	2^{-164}
Kyber-1024	256	4	3329	3, 2	10, 4	2^{-174}

2.3.2. Các thành phần lõi của CRYSTALS-Kyber

1) Kyber.CPAPKE:

Có nhiều điểm tương đồng với lược đồ LPR, tên viết tắt ba nhà nghiên cứu Vadim Lyubashevsky, Chris Peikert và Oded Regev. Lược đồ LPR phát triển sơ đồ mã hóa sử dụng LWE do Regev đề xuất, đồng thời khai thác cấu trúc vành đa thức trong mật mã NTRU. Mà điểm khác biệt là không sử dụng Ring-LWE mà dựa trên Module-LWE. Các thao tác chính trong lược đồ này gồm:

- **Sinh khóa:** Quá trình này tạo ra một cặp khóa gồm khóa công khai và khóa bí mật dựa trên bài toán MLWE. Các khóa thường được biểu diễn dưới dạng ma trận hoặc đa thức, được lấy mẫu từ những phân phối xác định trong vành đa thức.
- **Mã hóa:** Thông điệp ban đầu được mã hóa thành bản mã bằng khóa công khai. Trong quá trình này, một lượng nhiễu có kiểm soát được thêm vào nhằm tăng mức độ an toàn cho hệ mật mã.
- **Giải mã:** Thông điệp được khôi phục lại từ bản mã thông qua khóa bí mật. Độ chính xác của quá trình giải mã phụ thuộc vào việc phần nhiễu được giữ ở mức đủ nhỏ để không gây ra sai lệch trong kết quả.

Lược đồ này được thiết kế để chống lại đối thủ lượng tử bằng cách dựa trên độ khó của MLWE, kết hợp với các sai số ngẫu nhiên nhỏ.

2) Kyber.CCAKEM:

Để nâng cao mức độ an toàn, Kyber áp dụng biến đổi Fujisaki-Okamoto nhằm chuyển lược đồ Kyber.CPAPKE, vốn an toàn ở mức CPA, thành một cơ chế đóng gói khóa có khả năng chống lại tấn công chọn bản mã (IND-CCA2). Cách chuyển đổi này giúp đảm bảo tính an toàn mạnh hơn, kể cả trong những trường hợp kẻ tấn công có thể truy cập vào cả cơ chế giải mã bản mã.

2.3.3. Các khái niệm nền tảng và ký hiệu trong CRYSTALS-Kyber

Để hiểu chính xác về hoạt động bên trong của thuật toán Kyber, chúng ta cần hiểu một hệ thống ký hiệu chung và định nghĩa của một số cấu trúc toán học. Mà các ký hiệu và khái niệm sau đây chính là nền tảng:

1) Byte và mảng byte

- B : Biểu diễn tập giá trị byte có thể có, tức các số nguyên không dấu 8 bit:
 $\{0, 1, \dots, 255\}$
- B^k : Biểu diễn tập tất cả các mảng byte có độ dài k .
- B^* : Biểu diễn tập các mảng byte có độ dài tùy ý, hay còn gọi là luồng byte.
- $\parallel$: Ký hiệu phép nối hai mảng byte.
- $a + k$: Với một mảng byte a , ký hiệu này chỉ vị trí bắt đầu từ byte thứ k trong mảng a .

2) Vành đa thức

- R : Biểu diễn vành đa thức: $R = \mathbb{Z}[X]/(X^n + 1)$, trong đó $\mathbb{Z}$: tập số nguyên.
- R_q : Biểu diễn vành đa thức: $R_q = \mathbb{Z}_q[X]/(X^n + 1)$, trong đó $\mathbb{Z}_q$ là tập số nguyên modulo q . Các số sử dụng nhất quán: $n = 256$ và $q = 3329$.

3) Vector và ma trận

- Các chữ in đậm như $\mathbf{v}$, biểu diễn các vector có hệ số trong R hoặc R_q .
- Các chữ hoa in đậm như $\mathbf{A}$, biểu diễn ma trận.
- $\mathbf{v}^T$ hoặc $\mathbf{A}^T$ là biểu diễn chuyển vị của vector $\mathbf{v}$ hoặc ma trận $\mathbf{A}$.
- $\mathbf{v}[i]$: Truy cập phần tử thứ i của vector $\mathbf{v}$, theo chỉ số bắt đầu từ 0.
- $\mathbf{A}[i][j]$: Chỉ phần tử ở hàng thứ i và cột thứ j của ma trận $\mathbf{A}$.

4) Chuẩn của phần tử và đa thức

- Với: $w \in \mathbb{Z}_q$, ta có: $\|w\|_\infty = |w \bmod \pm q|$
- Với một đa thức: $w = w_0 + w_1X + \dots + w_{n-1}X^{n-1} \in R$; ta có:

$$\|w\|_\infty = \max_i \|w_i\|_\infty \quad (2.1)$$

$$\|w\| = \sqrt{\|w_0\|_\infty^2 + \dots + \|w_{n-1}\|_\infty^2} \quad (2.2)$$

- Các chuẩn này cũng mở rộng tương tự cho các vector trong: R^k và R_q^k

5) Tập hợp và phân phối

- $s \leftarrow S$: ám chỉ s được lấy mẫu ngẫu nhiên đều từ tập S .
- $s \leftarrow D$: ám chỉ s được lấy mẫu theo phân phối xác suất D .

6) Lấy mẫu đồng nhất trong R_q

Kyber lấy mẫu theo cách tắt định trong R_q , sao cho các phần tử được lấy gần với phân phối ngẫu nhiên đều về mặt thống kê. Điều này thực hiện bằng hàm $\text{Parse}: B^* \rightarrow R_q$, hàm này chuyển một luồng byte: $B = \{b_0, b_1, b_2, \dots\} \in B^*$ thành biểu diễn NTT: $\hat{a} \in R_q$ của $a \in R_q$. Hàm hoạt động như sau:

- Khởi tạo: $i \leftarrow 0, j \leftarrow 0$
- Trong khi $j < n$:

$$d_1 = b_i + 256 \cdot (b_{i+1} \bmod 16)$$

$$d_2 = \lfloor b_{i+1}/16 \rfloor + 16 \cdot b_{i+2}$$

Nếu: $d_1 < q$, thì gán:

$$\hat{a}_j = d_1, j \leftarrow j + 1$$

Nếu: $d_2 < q$, và $j < n$, thì gán:

$$\hat{a}_j = d_2, j \leftarrow j + 1$$

Cập nhật: $i \leftarrow i + 3$

- Cuối cùng, trả về:

$$\hat{a}(X) = \hat{a}_0 + \hat{a}_1 X + \hat{a}_2 X^2 + \dots + \hat{a}_{n-1} X^{n-1}$$

Trực giác của hàm Parse là: nếu mảng byte đầu vào gần với một mảng byte ngẫu nhiên đều về mặt thống kê, thì đa thức đầu ra cũng gần với một phân tử ngẫu nhiên đều trong R_q .

7) Lấy mẫu phân phối nhị thức trung tâm

Nhiều trong Kyber được lấy mẫu từ phân phối nhị thức trung tâm B_η , với tham số: $\eta = 2$ hoặc $\eta = 3$. Cụ thể:

- Lấy mẫu: $(a_1, a_2, \dots, a_\eta, b_1, b_2, \dots, b_\eta)$ một cách ngẫu nhiên đều từ $\{0,1\}^{2\eta}$
- Đầu ra là:

$$X = \sum_{i=1}^{\eta} (a_i - b_i) \quad (2.3)$$

Khi một đa thức $f \in R_q$ được lấy mẫu từ B_η , điều đó có nghĩa là mỗi hệ số của f được lấy mẫu độc lập từ B_η .

Đa thức f được lấy mẫu từ B_η có thể được biểu diễn như sau:

$$f(X) = \sum_{i=0}^{255} f_i X^i \quad (2.5)$$

trong đó mỗi hệ số f_i được định nghĩa là:

$$f_i = \sum_{j=0}^{\eta-1} \beta_{2i\eta+j} - \sum_{j=0}^{\eta-1} \beta_{2i\eta+\eta+j} \quad (2.6)$$

Mà: $(\beta_0, \beta_1, \dots, \beta_{512\eta-1})$ là các bit được suy ra từ mảng byte đầu vào B .

Và: $f(X)$ là một đa thức có $n = 256$ hệ số trong R_q .

8) Hàm nén và giải nén

Các hàm $Compress_q$ và $Decompress_q$ được sử dụng nhằm giảm kích thước của bản mã. Cách thực hiện là loại bỏ một số bit trọng số thấp trong quá trình biểu diễn dữ liệu mà không ảnh hưởng đáng kể đến độ chính xác của biểu diễn.

Ta cho một số nguyên $x \in \mathbb{Z}_q$, thì các hàm này xuất ra một số nguyên trong khoảng: $\{0, \dots, 2^d - 1\}$ với $d < \lceil \log_2(q) \rceil$.

Định nghĩa hình thức của $Compress_q$ và $Decompress_q$ như sau:

$$Compress_q(x, d) = \left\lfloor \frac{2^d}{q} \cdot x \right\rfloor \bmod 2^d \quad (2.7)$$

$$Decompress_q(x, d) = \left\lfloor \frac{q}{2^d} \cdot x \right\rfloor \quad (2.8)$$

Các hàm này được thiết kế để thỏa mãn tính chất sau:

$$x' = Decompress_q(Compress_q(x, d), d) \quad (2.9)$$

trong đó x' là một phần tử gần với x . Cụ thể:

$$|x' - x \bmod \pm q| \leq B_q := \left\lfloor \frac{q}{2^{d+1}} \right\rfloor \quad (2.10)$$

Ở đây: B_q biểu diễn cận trên của sai số được tạo ra bởi quá trình nén và giải nén. Mức chịu sai số này cân bằng giữa hiệu quả nén và độ chính xác.

Ngoài mục đích nén, các hàm này cũng được sử dụng để hỗ trợ việc hiệu chỉnh sai số trong bài toán LWE trong quá trình mã hóa và giải mã.

9) Các phép toán cốt lõi trong hàm băm

- **PRF:** Hàm giả ngẫu nhiên ($PRF: B^{32} \times B \rightarrow B^*$), được sử dụng cho tạo khóa và tạo nhiễu.
- **XOF:** Hàm đầu ra mở rộng ($XOF: B^* \times B \times B \rightarrow B^*$), được sử dụng để mở rộng ma trận công khai A từ một seed.
- **H:** Hàm băm ($H: B^* \rightarrow B^{32}$).
- **G:** Hàm băm ($G: B^* \rightarrow B^{32} \times B^{32}$).
- **KDF:** Hàm tạo khóa ($KDF: B^* \rightarrow B^*$).

10) NTT, Number Theoretic Transform

Để thực hiện phép nhân trong R_q , người ta đã phát triển một cách rất hiệu quả là sử dụng biến đổi số học, Number Theoretic Transform (NTT). Ta thấy với số nguyên tố $q = 3329$ và $q - 1 = 2^8 \cdot 13$, thì trường cơ sở $\mathbb{Z}_q$ chứa các căn nguyên thủy bậc 256, nhưng không chứa căn nguyên thủy bậc 512. Do đó, đa thức $X^{256} + 1$ của R được phân tích thành 128 đa thức bậc 2 modulo q , và NTT của một đa thức $f \in R_q$ là một vector gồm 128 đa thức bậc 1.

Triển khai NTT tính toán tại chỗ đơn giản, nếu không sắp xếp lại đầu ra, các đa thức thu được sẽ xuất hiện theo thứ tự đảo bit và ta định nghĩa NTT theo cách này. Cụ thể, đặt $\zeta = 17$ là căn nguyên thủy bậc 256 đầu tiên của đơn vị modulo q , và $\{\zeta, \zeta^3, \zeta^5, \dots, \zeta^{255}\}$ là tập tất cả các căn bậc 256 của đơn vị. Khi đó, đa thức $X^{256} + 1$ có thể được viết là:

$$X^{256} + 1 = \prod_{i=0}^{127} (X^2 - \zeta^{2i+1}) = \prod_{i=0}^{127} (X^2 - \zeta^{2br_7(i)+1}) \quad (2.11)$$

trong đó, $br_7(i)$, với $i = 0, \dots, 127$ là đảo bit của số nguyên không dấu 7 bit thứ i . Việc sắp xếp các nhân tử theo dạng này giúp tính toán tương thích tốt hơn với đặc điểm của các lệnh AVX. Khi đó, NTT của $f \in R_q$ được cho bởi:

$$(f \bmod X^2 - \zeta^{2br_7(0)+1}, \dots, f \bmod X^2 - \zeta^{2br_7(127)+1}) \quad (2.12)$$

Vector gồm các đa thức tuyến tính này được tuần tự hóa thành một vector trong $\mathbb{Z}_q^{256}$ theo cách chính tắc. Hơn nữa, để hỗ trợ các triển khai NTT tính toán tại chỗ, ta định nghĩa:

$$NTT(f) = \hat{f} = \hat{f}_0 + \hat{f}_1 X + \dots + \hat{f}_{255} X^{255} \quad (2.13)$$

$$\text{Với: } \hat{f}_{2i} = \sum_{j=0}^{127} f_{2j} \zeta^{(2br_7(i)+1)j} ; \quad \hat{f}_{2i+1} = \sum_{j=0}^{127} f_{2j+1} \zeta^{(2br_7(i)+1)j} \quad (2.14)$$

Mặc dù $\hat{f}$ được viết dưới dạng một đa thức trong R_q , nhưng không nên hiểu nó như một đa thức thông thường về mặt đại số. Cách hiểu tự nhiên hơn của $NTT(f) = \hat{f}$ là 128 đa thức bậc 1 như ở dạng vector ở trên.

Khi sử dụng NTT và phép biến đổi ngược của nó là NTT^{-1} , tích $f \cdot g$ của hai phân tử $f, g \in R_q$ có thể được tính một cách hiệu quả như sau:

$$NTT^{-1}(NTT(f) \circ NTT(g))$$

trong đó $\circ$ biểu diễn phép nhân từng thành phần, gồm 128 phép nhân cơ sở:

$$\hat{h}_{2i} + \hat{h}_{2i+1}X = (\hat{f}_{2i} + \hat{f}_{2i+1}X)(\hat{g}_{2i} + \hat{g}_{2i+1}X) \bmod X^2 - \zeta^{2br_7(i)+1}$$

11) Mã hóa và giải mã

Trong Kyber, có hai kiểu dữ liệu cần được chuyển đổi tuần tự thành mảng byte, bao gồm các mảng byte và các đa thức, hoặc vector nhiều đa thức. Hàm $Decode_\ell$ giải mã một mảng byte có độ dài 32ℓ byte thành một đa thức:

$$f = f_0 + f_1X + \dots + f_{255}X^{255},$$

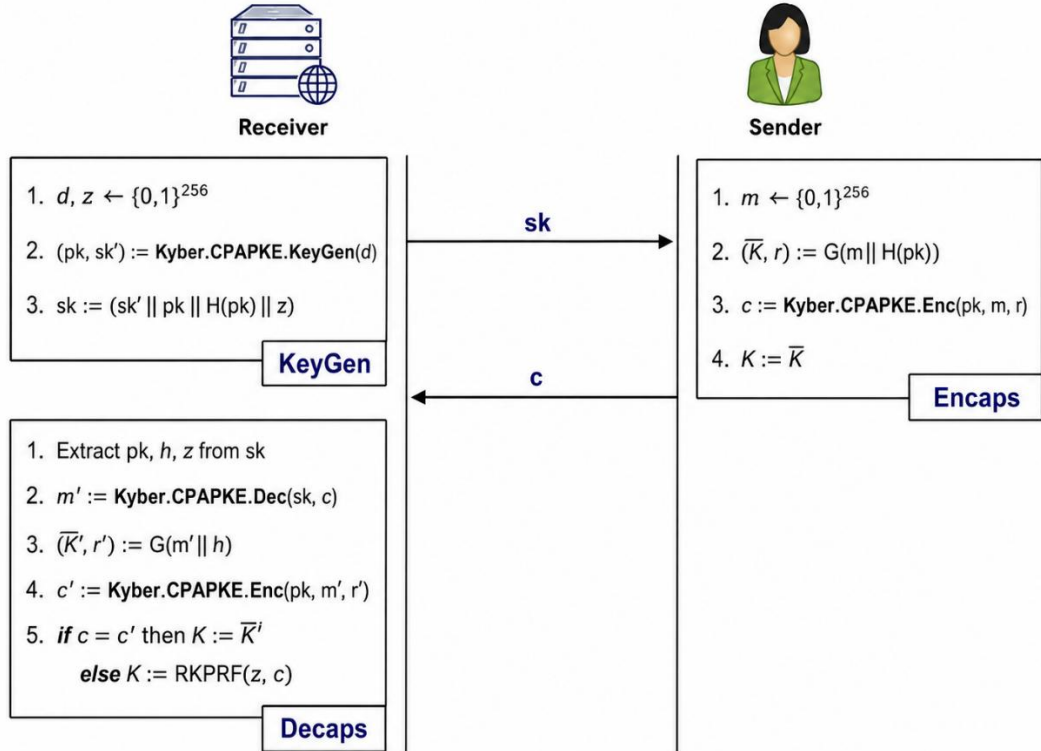
trong đó mỗi hệ số:

$$f_i \in \{0, \dots, 2^\ell - 1\}$$

Hàm $Encode_\ell$ thì được định nghĩa là hàm nghịch đảo của $Decode_\ell$.

2.3.4. Các bước hoạt động của CRYSTALS-Kyber

Hình 2.3 minh họa cơ chế sinh khóa, mã hóa và giải mã của Kyber trong mô hình client-server, giả sử Alice và Bob thực hiện quá trình truyền thông mã hóa.



Hình 2.3. Quá trình tạo shared key dùng chung giữa Sender và Receiver

Trong một kịch bản giao tiếp client-server sử dụng CRYSTALS-Kyber, Alice, tức server (receiver), bắt đầu quá trình bằng cách tạo một cặp khóa gồm khóa công khai và khóa bí mật. Khóa công khai sau đó được chia sẻ với Bob, tức client (sender), trong khi Alice giữ khóa bí mật một cách an toàn. Bob sử dụng khóa công khai nhận được để thực hiện quá trình đóng gói khóa, từ đó tạo ra một bản mã và một khóa chia sẻ. Bản mã sau đó được gửi trở lại Alice. Sau khi nhận bản mã, Alice sử dụng khóa bí mật của mình để thực hiện giải đóng gói, qua đó tạo ra cùng khóa chia sẻ với Bob. Luồng xử lý này cho thấy khả năng thiết lập khóa dùng chung an toàn của thuật toán Kyber, đồng thời thể hiện vai trò cần thiết của khóa công khai, khóa bí mật và bản mã trong quá trình trao đổi khóa.

2.3.4.1. Sinh khoá (*KeyGen – Key Generation*)

Được thực hiện bởi sender, ban đầu của quá trình là tạo ma trận A thông qua lấy mẫu đồng nhất trong miền $\mathbb{Z}_q$. Ma trận nhiễu e và ma trận s được lấy mẫu từ phân phối nhị thức cũng trong miền này. Có thể thấy rằng nếu biết ma trận A và ma trận t , việc tìm ma trận s là trực tiếp bằng cách nhân nghịch đảo của A với t :

Được thực hiện bởi sender, quá trình sinh khóa là bước đầu tiên trong thuật toán. Bước này nhằm tạo ra một cặp khóa gồm khóa công khai pk và khóa bí mật sk . Trong đó, khóa công khai pk được gửi cho bên người nhận sử dụng để đóng gói khóa, còn khóa bí mật sk phải được giữ kín và cũng được sử dụng ở phía nhận trong quá trình giải gói khóa.

Trong Kyber, thì quá trình sinh khóa được xây dựng nhờ bài toán MLWE. Thị ý tưởng chính là tạo ra một quan hệ gần tuyến tính giữa ma trận công khai A , vector bí mật s , vector nhiễu e , và vector công khai t :

$$t = A \cdot s + e \quad (2.15)$$

Có thể hiểu rằng nếu biểu thức chỉ có dạng là $t = A \cdot s$ thì kẻ tấn công có thể dễ dàng khôi phục lại s . Tuy nhiên, nếu ta thêm nhiễu e nhỏ, bài toán đã trở nên khó hơn rất nhiều. Vì kẻ tấn công biết A và t , nhưng lại không thể dễ dàng tìm ra chính xác s vì kết quả đã bị làm lệch bởi nhiễu nhỏ. Đây chính là nền tảng an toàn của Kyber. Lược đồ mã hóa Kyber phụ thuộc mạnh vào tính toán biểu thức $As + e$, trong đó

phép toán tốn tài nguyên nhất là phép nhân đa thức. Người ta đã tối ưu hoá phép toán này bằng sử dụng Number-Theoretic Transform (NTT) và biến đổi ngược của nó (INTT), đã giúp tăng cường đáng kể các phép toán đa thức và giảm tính toán.

➤ **Kyber.CCAKEM.KeyGen()**

Ta thấy Thuật toán 1 dưới đây không xây dựng khóa từ đầu mà gọi lại hàm sinh khóa lớp bên dưới là `Kyber.CPAKEM.KeyGen()`. Qua đó, hàm này tạo ra khóa công khai pk và khóa bí mật tạm thời sk_0 . Sau đó, `Kyber.CCAKEM` mở rộng khóa bí mật bằng cách ghép thêm một số thành phần khác:

$$sk = sk_0 \parallel pk \parallel H(pk) \parallel z$$

Thuật toán 1 `Kyber.CCAKEM.KeyGen()`: Sinh khoá

Đầu ra: khoá công khai $pk \in B^{12 \cdot k \cdot n/8+32}$

Đầu ra: khóa bí mật $sk \in B^{24 \cdot k \cdot n/8+96}$

1: $z \leftarrow B^{32}$

2: $(pk, sk_0) := \text{Kyber.CPAKEM.KeyGen}()$

3: $sk \leftarrow (sk_0 \parallel pk \parallel H(pk) \parallel z)$

4: Return (pk, sk)

Trong đó, sk_0 là khóa bí mật của CPAKEM, pk là khóa công khai, $H(pk)$ là giá trị băm của khóa công khai, còn z là một chuỗi ngẫu nhiên. Việc lưu thêm pk và $H(pk)$ giúp bước giải gói khóa có đủ dữ liệu để mã hóa lại và kiểm tra bản mã. Thành phần z được dùng trong trường hợp bản mã không hợp lệ, nhằm tạo ra khóa thay thế mà không làm lộ thông tin về khóa bí mật.

➤ **Kyber.CPAPKE.KeyGen()**

Đối với thuật toán trong Thuật toán 2, minh hoạ quá trình sinh khóa bắt đầu bằng việc sinh một giá trị ngẫu nhiên d . Giá trị này được đưa qua hàm băm mở rộng G để tạo ra hai seed là ρ và σ . Seed ρ được dùng để sinh ma trận công khai A , còn seed σ được dùng để sinh vector bí mật s và vector nhiễu e .

Ma trận A không được lưu trữ trực tiếp toàn bộ trong khóa công khai. Thay vào đó, Kyber chỉ lưu seed ρ , sau đó khi cần có thể sinh lại các phần tử của ma trận A bằng hàm XOF . Ta thấy, cách làm này giúp giảm kích thước khóa công khai thay vì lưu một ma trận lớn, hệ thống chỉ cần lưu một seed ngắn.

Thuật toán 2 Kyber.CPAKEM.KeyGen()

Đầu ra: Khóa bí mật $sk_0 \in B^{12 \cdot k \cdot n/8}$

Đầu ra: Khóa công khai $pk \in B^{12 \cdot k \cdot n/8+32}$

```
1:  $d \leftarrow B^{32}$ 
2:  $(\rho, \sigma) \leftarrow G(d \parallel k)$ , với  $k = 2$ 
3:  $N \leftarrow 0$ 
4: for  $i$  from 0 to  $k - 1$  do                                 $\triangleright$  Sinh  $\hat{A} \in R_q^{k \times k}$  trong miền NTT
5:   for  $j$  from 0 to  $k - 1$  do
6:      $\hat{A}[i][j] \leftarrow \text{Parse}(\text{XOF}(\rho, j, i))$ 
7:   end for
8: end for
9: for  $i$  from 0 to  $k - 1$  do                                 $\triangleright$  Lấy mẫu  $s \in R_q^k$  từ phân bố  $B_{\eta_1}$ 
10:    $s[i] \leftarrow \text{CBD}_{\eta_1}(\text{PRF}(\sigma, N))$ 
11:    $N \leftarrow N + 1$ 
12: end for
13: for  $i$  from 0 to  $k - 1$  do                                 $\triangleright$  Lấy mẫu  $e \in R_q^k$  từ phân bố  $B_{\eta_1}$ 
14:    $e[i] \leftarrow \text{CBD}_{\eta_1}(\text{PRF}(\sigma, N))$ 
15:    $N \leftarrow N + 1$ 
16: end for
17:  $\hat{s} \leftarrow \text{NTT}(s)$ 
18:  $\hat{e} \leftarrow \text{NTT}(e)$ 
19:  $\hat{t} \leftarrow \hat{A} \circ \hat{s} + \hat{e}$ 
20:  $pk \leftarrow \text{Encode}_{12}(\hat{t} \bmod^+ q) \parallel \rho$              $\triangleright pk := A \circ s + e$ 
21:  $sk_0 \leftarrow \text{Encode}_{12}(\hat{s} \bmod^+ q)$                  $\triangleright sk_0 := s$ 
22: Return  $(pk, sk_0)$ 
```

Sau khi có ρ và σ , thuật toán sinh các vector s và e bằng hàm CBD . Đây là hàm lấy mẫu theo phân phối nhị thức trung tâm, tạo ra các hệ số nhỏ quanh giá trị 0. Các hệ số nhỏ này rất quan trọng, vì nhiều quá lớn thì quá trình giải mã sau này có thể bị sai. Ngược lại, nếu không có nhiều thì không đủ an toàn.

Tiếp theo, vector s và e chuyển sang miền NTT: $\hat{s} = \text{NTT}(s)$, $\hat{e} = \text{NTT}(e)$. Lý do sử dụng NTT là vì trong Kyber, các phần tử không chỉ là số nguyên đơn lẻ mà là các đa thức. Phép nhân giữa ma trận và vector thực chất bao gồm nhiều phép nhân đa thức. Khi đưa sang miền NTT, phép nhân đa thức lại có thể thay bằng phép nhân theo từng thành phần, giúp thuật toán chạy nhanh hơn và thân thiện hơn khi triển khai phần cứng.

Sau đó, thuật toán tính: $\hat{t} = \hat{A} \circ \hat{s} + \hat{e}$, mà ký hiệu $\circ$ thể hiện phép nhân trong miền NTT. Còn kết quả $\hat{t}$ là thành phần chính của khóa công khai. Mà cuối cùng, khóa công khai và khóa bí mật của CPAPKE được tạo bởi: $pk = \text{Encode}_{12}(\hat{t}) \parallel \rho$ và $sk' = \text{Encode}_{12}(\hat{s})$.

Như vậy, có thể hiểu đơn giản rằng pk chứa thông tin công khai để bên gửi có thể mã hóa, còn sk_0 chứa vector bí mật dùng để giải mã. Người ngoài biết A và t , nhưng không thể khôi phục lại s do sự xuất hiện của nhiễu e .

2.3.4.2. Đóng gói khoá (*Enc – Encapsulation*)

Quá trình đóng gói khoá được mô tả sau đây, được thực hiện bởi receiver sử dụng khóa công khai pk nhận được để sinh bản mã ciphertext c và shared key K .

➤ **Kyber.CCAKEM.Enc(pk)**

Đóng gói khoá là quá trình được thực hiện bởi bên gửi. Đầu vào của quá trình này là khóa công khai pk của bên nhận. Sau khi thực hiện, thuật toán tạo ra hai giá trị là bản mã c và khóa chia sẻ K . Bản mã c sẽ được gửi cho bên nhận, còn khóa chia sẻ K được bên gửi giữ lại để sử dụng cho các bước mã hóa đối xứng hoặc giao tiếp bảo mật tiếp theo.

Thuật toán 3 Kyber.CCAKEM.Enc(pk): Đóng gói khoá

Đầu vào: Khóa công khai $pk \in B^{12 \cdot k \cdot n/8 + 32}$

Đầu ra: Bản mã $c \in B^{d_u \cdot k \cdot n/8 + d_v \cdot n/8}$

Đầu ra: Khóa chia sẻ $K \in B^*$

- 1: $m \leftarrow B^{32}$
 - 2: $(\bar{K}, r) \leftarrow G(m \parallel H(pk))$
 - 3: $c := \text{Kyber.CPAPKE.Enc}(pk, m, r)$
 - 4: $K := \bar{K}$
 - 5: **Return** (c, K)
-

Thuật toán 3 minh họa thuật toán trước hết sinh một thông điệp ngẫu nhiên m . Sau đó, m được kết hợp với giá trị băm của khóa công khai $H(pk)$ để tạo ra hai giá trị là $(\bar{K}, r) = G(m \parallel H(pk))$. Việc sử dụng $H(pk)$ giúp ràng buộc kết quả sinh ra với khóa công khai đang sử dụng. Ở đây, $\bar{K}$ là giá trị trung gian dùng để tạo khóa chia sẻ cuối cùng, còn r là giá trị ngẫu nhiên dùng trong quá trình

mã hóa CPAPKE. Việc đưa $H(pk)$ vào đầu vào của hàm G giúp ràng buộc kết quả với khóa công khai cụ thể. Điều này có nghĩa là cùng một thông điệp m , nhưng nếu dùng khóa công khai khác nhau thì khóa sinh ra cũng khác nhau. Sau đó, thuật toán gọi hàm mã hóa của CPAPKE để tiến hành tính $c = \text{Kyber.CPAPKE.Enc}(pk, m, r)$. Hàm này dùng khóa công khai pk , thông điệp m , và giá trị ngẫu nhiên r để tạo bản mã c . Sau khi có bản mã, khóa chia sẻ cuối cùng được gán trực tiếp từ giá trị $\bar{K}$, tức: $K = \bar{K}$. Tóm lại, hàm này trả về một cặp kết quả bản mã và khóa chia sẻ (c, K) .

Thuật toán 4 $\text{Kyber.CPAPKE.Enc}(pk, m, r)$

Đầu vào: Khóa công khai $pk \in B^{12 \cdot k \cdot n/8 + 32}$

Đầu vào: Thông điệp $m \in B^{32}$ và seed ngẫu nhiên $r \in B^{32}$

Đầu ra: Bản mã $c \in B^{d_u \cdot k \cdot n/8 + d_v \cdot n/8}$

```

1:  $N \leftarrow 0$ 
2:  $\hat{t} \leftarrow \text{Decode}_{12}(pk)$ 
3:  $\rho \leftarrow pk + 12 \cdot k \cdot n/8$ 
4: for  $i$  from 0 to  $k - 1$ , do  $\triangleright$  Sinh  $\hat{A} \in R_q^{k \times k}$  trong miền NTT
5:   for  $j$  from 0 to  $k - 1$ , do
6:      $\hat{A}^T[i][j] \leftarrow \text{Parse}(\text{XOF}(\rho, i, j))$ 
7:   end for
8: end for
9: for  $i$  from 0 to  $k - 1$ , do  $\triangleright$  Lấy mẫu  $r \in R_q^k$  từ phân bố  $B_{\eta_1}$ 
10:    $r[i] \leftarrow \text{CBD}_{\eta_1}(\text{PRF}(r, N))$ 
11:    $N \leftarrow N + 1$ 
12: end for
13: for  $i$  from 0 to  $k - 1$ , do  $\triangleright$  Lấy mẫu  $e_1 \in R_q^k$  từ phân bố  $B_{\eta_2}$ 
14:    $e_1[i] \leftarrow \text{CBD}_{\eta_2}(\text{PRF}(r, N))$ 
15:    $N \leftarrow N + 1$ 
16: end for
17:  $e_2 \leftarrow \text{CBD}_{\eta_2}(\text{PRF}(r, N))$   $\triangleright$  Lấy mẫu  $e_2 \in R_q^k$  từ phân bố  $B_{\eta_2}$ 
18:  $\hat{r} \leftarrow \text{NTT}(r)$ 
19:  $u \leftarrow \text{NTT}^{-1}(\hat{A}^T \circ \hat{r}) + e_1$ 
20:  $v \leftarrow \text{NTT}^{-1}(\hat{t}^T \circ \hat{r}) + e_2 + \text{Decompress}_q(\text{Decode}_1(m), 1)$ 
21:  $c_1 \leftarrow \text{Encode}_{d_u}(\text{Compress}_q(u, d_u))$ 
22:  $c_2 \leftarrow \text{Encode}_{d_v}(\text{Compress}_q(v, d_v))$ 
23: Return  $c \leftarrow c_1 \parallel c_2$ 

```

Trong cơ chế Kyber, quá trình $CPAPKE.Enc$ là bước mã hóa lõi, sử dụng khóa công khai và các giá trị ngẫu nhiên để tạo ra bản mã. Bản mã này không chỉ mang thông tin đã được mã hóa mà còn chứa các thành phần nhiễu, giúp tăng khả năng chống lại các phương pháp phân tích mật mã.

Thuật toán 4 mô tả chi tiết $Kyber.CPAPKE.Enc(pk, m, r)$, thuật toán bắt đầu bằng cách giải mã khóa công khai pk để lấy ra $\hat{t}$ và seed ρ . Trong đó, $\hat{t}$ là một phần của khóa công khai ở miền NTT, còn ρ được dùng để sinh lại ma trận công khai A . Từ seed ρ , thuật toán sinh lại ma trận $\hat{A}$. Tuy nhiên, trong quá trình mã hóa, Kyber sử dụng ma trận chuyển vị $\hat{A}^T$. Điều này phù hợp với công thức tạo bản mã, vì bên gửi cần nhân ma trận công khai với vector ngẫu nhiên để tạo ra thành phần u .

Sau đó, từ giá trị ngẫu nhiên r , thuật toán sinh ra ba thành phần: vector ngẫu nhiên r , vector nhiễu e_1 , và nhiễu e_2 . Các giá trị này cũng được lấy mẫu bằng hàm CBD , nên có hệ số nhỏ. Vector ngẫu nhiên r đóng vai trò giống như một “khóa tạm thời” trong quá trình mã hóa. Các nhiễu e_1 và e_2 được thêm vào để làm cho bản mã khó bị phân tích ngược. Vector r sau đó được đưa sang miền NTT: $\hat{r} = NTT(r)$. Tiếp theo, thuật toán tính thành phần thứ nhất của bản mã $u = NTT^{-1}(\hat{A}^T \circ \hat{r}) + e_1$. Có thể hiểu u là phần bản mã được tạo từ ma trận công khai A và vector ngẫu nhiên r . Thành phần này không chứa trực tiếp thông điệp, nhưng nó giúp bên nhận có thể dùng khóa bí mật để khôi phục lại phần cần thiết khi giải mã. Sau đó, thuật toán tính thành phần thứ hai: $v = NTT^{-1}(\hat{t}^T \circ \hat{r}) + e_2 + Decompress_q(Decode_1(m), 1)$.

Trong biểu thức này, phần $NTT^{-1}(\hat{t}^T \circ \hat{r})$ tạo ra một giá trị phụ thuộc vào khóa công khai. Thành phần e_2 là nhiễu được thêm vào để tăng độ an toàn. Phần cuối cùng $Decompress_q(Decode_1(m), 1)$ là cách Kyber đưa thông điệp m vào trong đa thức. Nói cách khác, thông điệp không được đặt trực tiếp vào bản mã, mà được biến đổi sang dạng phù hợp rồi cộng vào v .

Sau khi có u và v , Kyber không gửi trực tiếp hai giá trị này ở dạng đầy đủ, vì như vậy bản mã sẽ lớn. Thay vào đó, thuật toán dùng hàm nén: $c_1 = Encode_{d_u}(Compress_q(u, d_u))$, $c_2 = Encode_{d_v}(Compress_q(v, d_v))$.

Rồi cuối cùng, bản mã được tạo bằng cách ghép hai phần: $c = c_1 \parallel c_2$. Trong đó, c_1 là phần mã hóa của u , còn c_2 là phần mã hóa của v . Việc nén giúp giảm kích thước bản mã khi truyền đi. Tuy nhiên, quá trình nén cũng tạo ra một lượng sai số nhỏ, nên các tham số d_u và d_v phải được chọn sao cho sai số này không làm quá trình giải mã bị lỗi.

2.3.4.3. Giải gói khoá (Dec – Decapsulation)

Giải gói khoá là quá trình được thực hiện bởi bên nhận. Đầu vào của bước này là bản mã c nhận được từ bên gửi và khóa bí mật sk . Mục tiêu của quá trình này là khôi phục lại khóa chia sẻ K . Nếu bản mã hợp lệ và không bị thay đổi, thì khóa K mà bên nhận tính được sẽ giống với khóa K mà bên gửi đã tạo.

➤ **Kyber.CCAKEM.Dec(c, sk)**

Thuật toán 5 Kyber.CCAKEM.Dec(c, sk): Giải gói khoá

Đầu vào: Bản mã $c \in B^{d_u \cdot k \cdot n/8 + d_v \cdot n/8}$

Đầu vào: Khóa bí mật $sk \in B^{24 \cdot k \cdot n/8 + 96}$

Đầu ra: Khóa chia sẻ $K \in B^{32}$

```

1:  $pk := sk[12 \cdot k \cdot n/8]$ 
2:  $H(pk) := sk[24 \cdot k \cdot n/8 + 32] \in B^{32}$ 
3:  $z := sk[24 \cdot k \cdot n/8 + 64]$ 
4:  $m' := \text{Kyber.CPAPKE.Dec}(sk_0, c)$ 
5:  $(\bar{K}, r) := G(m' \parallel H(pk))$ 
6:  $c' := \text{Kyber.CPAPKE.Enc}(pk, m', r)$ 
8: if  $c = c'$  then
9:   Return  $K := \bar{K}$ 
10: else
11:   Return  $K := \text{RKPRF}(z, c)$ 
12: end if
```

Thuật toán 5 minh họa quá trình giải gói khoá, đầu tiên quá trình này tách khóa bí mật sk thành các thành phần đã được ghép trong bước sinh khóa. Các thành phần này gồm sk_0 , pk , $H(pk)$, và z . Trong đó, sk_0 được dùng để giải mã CPAPKE, pk được dùng để mã hóa lại kiểm tra bản mã, $H(pk)$ được dùng để tính lại giá trị ngẫu nhiên, còn z được dùng nếu bản mã không hợp lệ.

Trước hết, thuật toán giải mã bản mã bằng khóa bí mật với CPAPKE $m' = \text{Kyber.CPAPKE.Dec}(sk', c)$. Giá trị m' là thông điệp được khôi phục tạm thời từ bản mã. Tuy nhiên, ở mức CCAKEM, thuật toán chưa tin ngay rằng m' là đúng. Lý do là kẻ tấn công có thể cố ý gửi một bản mã giả hoặc bản mã đã bị sửa đổi. Vì vậy, Kyber cần thêm bước kiểm tra.

Từ m' và $H(pk)$, thuật toán tính lại $(\bar{K}, r) = G(m' \parallel H(pk))$. Nếu bản mã ban đầu là hợp lệ, thì giá trị m' này phải đúng với thông điệp mà bên gửi đã dùng. Khi đó, giá trị r được tính lại cũng sẽ trùng với giá trị ngẫu nhiên mà bên gửi đã dùng trong quá trình mã hóa.

Tiếp theo, thuật toán mã hóa lại m' bằng khóa công khai pk và giá trị r : $c' = \text{Kyber.CPAPKE.Enc}(pk, m', r)$. Bản mã c' này được dùng để kiểm tra. Nếu c' giống với bản mã nhận được c , thuật toán kết luận rằng bản mã hợp lệ. Khi đó, khóa chia sẻ được tạo bằng: $K = \bar{K}$.

Ngược lại, nếu c' khác c , nghĩa là bản mã nhận được có thể đã bị sửa đổi hoặc không được tạo đúng theo thuật toán Kyber. Trong trường hợp này, Kyber không trả lỗi trực tiếp. Thay vào đó, thuật toán dùng giá trị ngẫu nhiên z đã lưu trong khóa bí mật để tạo khóa thay thế: $K = \text{RKPRF}(z, c)$. Trong đó, RKPRF tạo một khóa 32 byte từ $z \parallel c$, thực hiện bằng SHAKE256.

Cách xử lý này được gọi là implicit rejection. Ý nghĩa của nó là hệ thống vẫn trả ra một khóa, nhưng khóa này không phải khóa đúng mà bên gửi mong đợi. Nhờ vậy, kẻ tấn công không thể dựa vào thông báo lỗi hoặc phản hồi khác nhau để suy đoán thông tin về khóa bí mật.

➤ **Kyber.CPAPKE.Dec(sk, c)**

Thuật toán 6 Kyber.CPAPKE.Dec(sk, c)

Đầu vào: Khóa bí mật $sk_0 \in B^{12 \cdot k \cdot n/8}$

Đầu vào: Bản mã $c \in B^{d_u \cdot k \cdot n/8 + d_v \cdot n/8}$

Đầu ra: Thông điệp $m' \in B^{32}$

- 1: $u \leftarrow \text{Decompress}_q(\text{Decode}_{d_u}(c), d_u)$
 - 2: $v \leftarrow \text{Decompress}_q(\text{Decode}_{d_v}(c + d_u \cdot k \cdot n/8), d_v)$
 - 3: $\hat{s} \leftarrow \text{Decode}_{12}(sk)$
 - 4: $m' \leftarrow \text{Encode}_1(\text{Compress}_q(v - \text{NTT}^{-1}(\hat{s}^T \circ \text{NTT}(u)), 1))$
 - 5: **Return** m'
-

Thuật toán 6 thực hiện việc giải mã bản mã ở CPAPKE. Trước hết, bản mã c được tách thành hai phần: $c = c_1 \parallel c_2$. Trong đó, c_1 tương ứng với thành phần u , còn c_2 tương ứng với thành phần v . Vì trong quá trình mã hóa, u và v đã được nén, nên khi giải mã cần giải nén lại:

$$u = \text{Decompress}_q(\text{Decode}_{d_u}(c_1), d_u)$$

$$v = \text{Decompress}_q(\text{Decode}_{d_v}(c_2), d_v)$$

Sau đó, khóa bí mật sk_0 được giải mã để lấy lại vector bí mật $\hat{s}$. Vector này đang ở miền NTT, phù hợp để thực hiện phép nhân nhanh với $NTT(u)$. Thuật toán tính: $NTT^{-1}(\hat{s}^T \circ NTT(u))$. Giá trị này chính là phần mà bên nhận có thể tính được nhờ khóa bí mật. Trong quá trình mã hóa, v được tạo bằng cách cộng thông điệp với một thành phần liên quan đến khóa công khai và vector ngẫu nhiên. Khi giải mã, bên nhận dùng khóa bí mật để tính lại thành phần che giấu đó, rồi lấy v trừ đi nó: $v - NTT^{-1}(\hat{s}^T \circ NTT(u))$.

Kết quả còn lại gần với thông điệp ban đầu. Do trong quá trình mã hóa có thêm nhiễu và nén dữ liệu, kết quả này có thể không giống tuyệt đối ở dạng số, nhưng nếu sai số nằm trong giới hạn cho phép thì hàm Compress_q vẫn có thể đưa nó về đúng bit thông điệp ban đầu:

$$m' = \text{Encode}_1(\text{Compress}_q(v - NTT^{-1}(\hat{s}^T \circ NTT(u)), 1))$$

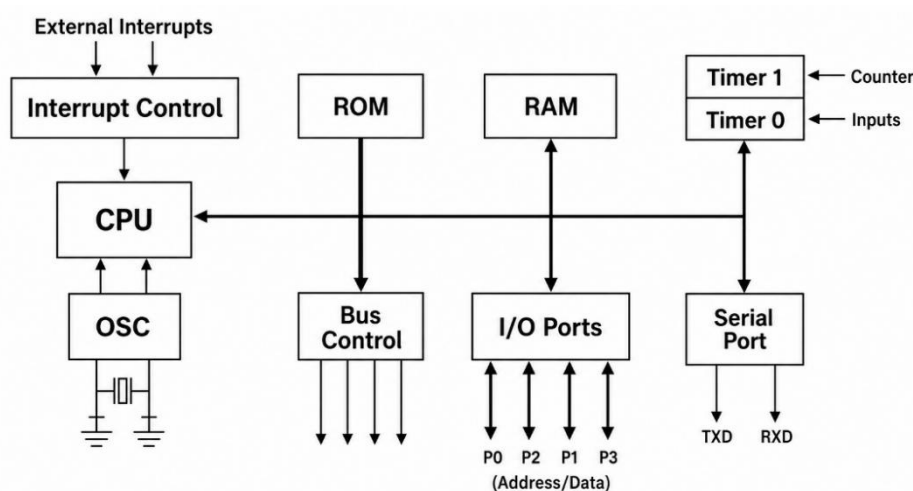
*Tóm lại, có thể hiểu đơn giản quá trình giải mã như sau: khi mã hóa, thông điệp m được giấu trong thành phần v . Bên nhận dùng khóa bí mật s và thành phần u để tính lại phần che giấu, sau đó loại bỏ phần này khỏi v . Phần còn lại chính là thông điệp ban đầu. Sau khi khôi phục được m' , thuật toán CCAKEM tiếp tục dùng m' để kiểm tra lại bản mã và sinh khóa chia sẻ cuối cùng.

2.4. VI ĐIỀU KHIỂN (MCU)

Vi điều khiển là một hệ thống xử lý nhúng, trong đó các thành phần như bộ xử lý trung tâm CPU, bộ nhớ và các module ngoại vi được tích hợp chung trên một chip, như minh họa ở Hình 2.4. Vi điều khiển thường được sử dụng để thực hiện các tác vụ điều khiển trong thời gian thực. Trong hệ thống này, lõi xử lý có nhiệm vụ giải mã và thực thi lệnh, điều khiển luồng hoạt động của chương trình, đồng thời xử lý dữ liệu trong quá trình vận hành.

Bộ nhớ Flash hoặc ROM được dùng để lưu chương trình firmware và các dữ liệu cố định. RAM được sử dụng trong quá trình chương trình chạy, chẳng hạn như lưu biến toàn cục, biến cục bộ, ngăn xếp và các vùng đệm tạm thời. Trong một số vi điều khiển, EEPROM cũng được tích hợp để lưu các tham số cấu hình hoặc dữ liệu cần được giữ lại ngay cả khi hệ thống mất nguồn.

Bên cạnh CPU và bộ nhớ, vi điều khiển còn có nhiều module ngoại vi như GPIO, UART, SPI, I2C, CAN, timer, bộ điều khiển ngắt, ADC, DAC,... Các module này giúp vi điều khiển giao tiếp với các thiết bị bên ngoài và thực hiện các chức năng điều khiển khác nhau. Các thành phần bên trong chip được kết nối với nhau thông qua hệ thống bus nội bộ, bao gồm bus địa chỉ, bus dữ liệu và bus điều khiển. Ngoài ra, nguồn xung nhịp, có thể đến từ thạch anh ngoài hoặc bộ dao động nội, sẽ quyết định tần số hoạt động và ảnh hưởng trực tiếp đến hiệu năng xử lý của vi điều khiển.



Hình 2.4. Sơ đồ mô tả kiến trúc vi điều khiển 8051

2.4.1. Bộ xử lý trung tâm - lõi RISC-V

RISC-V là một kiến trúc tập lệnh mở, không yêu cầu phí bản quyền, được phát triển ban đầu tại Đại học Berkeley vào khoảng năm 2010. Một đặc điểm quan trọng của RISC-V là cấu trúc thiết kế theo dạng module. Tập lệnh cơ bản RV32I chỉ bao gồm một số lệnh nền tảng cần thiết, sau đó có thể mở rộng thêm bằng các phần mở rộng tùy chọn như M cho nhân/chia, A cho thao tác nguyên tử, F/D cho số thực dấu phẩy động, C cho lệnh nén và V cho xử lý vector. RISC-V cũng tuân theo mô hình Load/Store, nghĩa là các phép toán của ALU chủ yếu được thực hiện trên thanh ghi,

còn việc truy cập bộ nhớ chỉ được thực hiện thông qua các lệnh LOAD như LD, LW, LB,... và STORE như SD, SW, SB,...

Đối với RV32, mỗi lệnh có độ dài cố định 32 bit và được chia thành nhiều trường như opcode, rd, rs1, rs2, funct3, funct7 và immediate. Trong đó, opcode dùng để xác định nhóm lệnh, rd là thanh ghi đích, rs1 và rs2 là các thanh ghi nguồn, còn funct3 và funct7 hỗ trợ xác định cụ thể loại thao tác cần thực hiện. Cách tổ chức lệnh cố định này giúp phần cứng giải mã lệnh đơn giản hơn, đồng thời thuận lợi cho việc thiết kế bộ xử lý. Một lõi RISC-V thông thường có 32 thanh ghi tổng quát từ x0 đến x31, trong đó thanh ghi x0 luôn có giá trị bằng 0. Các thanh ghi còn lại được sử dụng theo quy ước ABI để truyền tham số, lưu giá trị tạm thời và chứa kết quả trả về.

Về chế độ hoạt động, RISC-V hỗ trợ các mức đặc quyền khác nhau. Tối thiểu, hệ thống có thể có User mode và Machine mode. Trong các hệ thống phức tạp hơn, đặc biệt là những hệ thống cần chạy hệ điều hành, kiến trúc có thể hỗ trợ thêm Supervisor mode. Các chế độ đặc quyền này giúp phân tách quyền truy cập giữa chương trình người dùng, hệ điều hành và phần mềm mức thấp.

Quá trình thực thi một lệnh trong bộ xử lý RISC-V thường có thể được chia thành năm giai đoạn chính: Fetch để lấy lệnh từ bộ nhớ, Decode để giải mã lệnh và đọc thanh ghi, Execute để thực hiện phép toán, MEM để truy cập bộ nhớ hoặc xử lý các thao tác liên quan, và WB để ghi kết quả trở lại thanh ghi. Nhờ tập lệnh đơn giản và cách tổ chức rõ ràng, RISC-V rất phù hợp để triển khai kiến trúc pipeline, đồng thời cũng có thể mở rộng sang các kỹ thuật xử lý hiệu năng cao hơn như superscalar. Bên cạnh đó, RISC-V hiện có hệ sinh thái phần mềm ngày càng phát triển, với các công cụ như GCC, LLVM, Spike, QEMU, FreeRTOS và Zephyr hỗ trợ quá trình biên dịch, mô phỏng, kiểm thử và triển khai hệ thống.

2.4.2. GPIO (General-Purpose Input/Output)

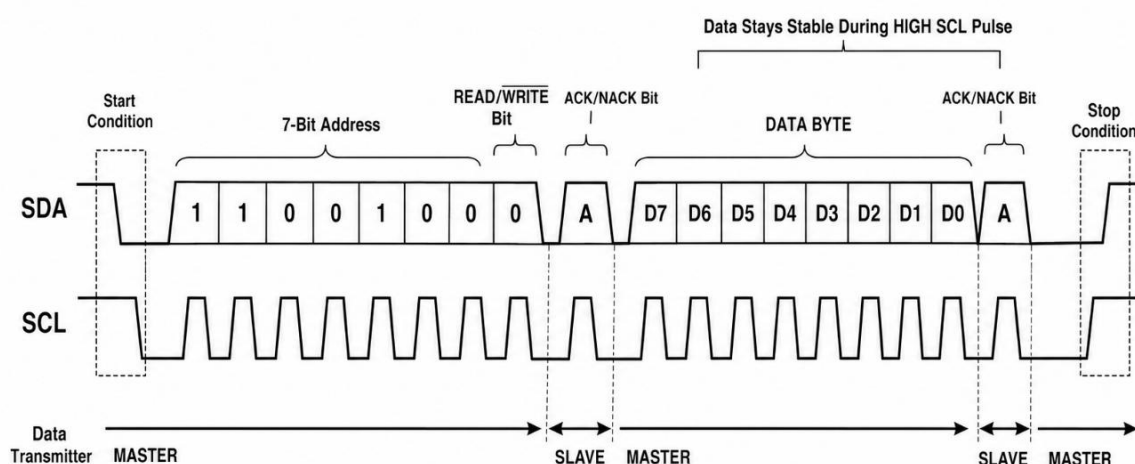
GPIO là các chân vào ra đa dụng trên vi điều khiển, mỗi chân có thể được cấu hình riêng thành ngõ vào hoặc ngõ ra số. Khi đặt ở chế độ ngõ vào, GPIO dùng để đọc mức logic từ các thiết bị bên ngoài, ví dụ như nút nhấn, công tắc hoặc cảm biến số. Còn khi ở chế độ ngõ ra, chân GPIO có thể xuất tín hiệu để điều khiển LED, transistor, relay hoặc một số mạch ngoại vi khác.

2.4.3. I2C (Inter-Integrated Circuit)

I2C là chuẩn giao tiếp nối tiếp hai dây, được Philips, hiện nay là NXP, phát triển vào năm 1982. Giao tiếp này dùng hai đường chính là SDA để truyền dữ liệu và SCL để truyền xung clock. I2C thường được dùng để nối vi điều khiển với nhiều thiết bị ngoại vi như cảm biến, LCD, RTC, ADC hoặc DAC. Ưu điểm của I2C là chỉ cần hai dây nhưng vẫn có thể kết nối nhiều thiết bị trên cùng một bus. Giao tiếp này hỗ trợ nhiều master và nhiều slave, trong đó mỗi slave được nhận biết bằng địa chỉ 7 bit hoặc 10 bit.

Cách hoạt động của I2C được thể hiện ở Hình 2.5. Quá trình truyền bắt đầu bằng điều kiện Start, khi SDA chuyển từ mức 1 xuống mức 0 trong lúc SCL vẫn đang ở mức 1. Sau đó master gửi khung địa chỉ gồm địa chỉ slave và bit R/W, trong đó R/W bằng 1 là đọc và bằng 0 là ghi. Sau mỗi byte dữ liệu 8 bit, bên nhận sẽ gửi ACK bằng cách kéo SDA xuống mức thấp ở chu kỳ bit thứ 9. Nếu không muốn tiếp tục nhận dữ liệu thì bên nhận để SDA ở mức cao, gọi là NACK. Khi truyền xong, master tạo điều kiện Stop bằng cách cho SDA chuyển từ mức 0 lên mức 1 khi SCL đang ở mức 1. I2C có hai kiểu truyền thường gặp là:

- Master Transmitter/Slave Receiver (Master phát Start, gửi địa chỉ + R/W = 0, gửi dữ liệu rồi Stop).
- Master Receiver/Slave Transmitter (Master phát Start, gửi địa chỉ + R/W = 1, Slave gửi dữ liệu, Master gửi ACK/NACK rồi Stop).



Hình 2.5. Waveform giao tiếp của I2C

2.4.4. Interrupt

Interrupt là cơ chế phần cứng giúp vi điều khiển tạm dừng chương trình đang chạy để xử lý một sự kiện quan trọng hơn, có thể đến từ ngoại vi hoặc từ bên trong hệ thống. Sau khi xử lý xong, vi điều khiển sẽ quay lại tiếp tục chương trình chính tại vị trí trước đó. Cơ chế này giúp hệ thống phản hồi nhanh hơn và tiết kiệm năng lượng, vì CPU có thể ở trạng thái chờ cho đến khi có ngắt xảy ra. Khi nhận ngắt, vi điều khiển sẽ lưu lại ngữ cảnh hiện tại rồi chuyển sang chương trình phục vụ ngắt ISR. Sau khi ISR kết thúc, ngữ cảnh khôi phục và chương trình chính thực thi bình thường.

2.4.5. Timer

Bộ đếm/timer là một ngoại vi thường có trong vi điều khiển, dùng để tạo trễ, đếm xung, tạo PWM hoặc hỗ trợ đồng hồ thời gian thực RTC. Timer có thể lấy xung từ clock hệ thống hoặc từ external clock, sau đó đếm tăng hoặc đếm giảm tùy cấu hình. Một timer cơ bản thường gồm các phần chính sau:

- Thanh ghi điều khiển dùng để cấu hình chế độ hoạt động như one-shot, periodic, PWM, input capture hoặc output compare. Thanh ghi này cũng có thể cấu hình tự nạp lại, prescaler để chia tần số clock, và cho phép bật/tắt ngắt khi timer tràn hoặc đạt giá trị so sánh.
- Thanh ghi đếm lưu giá trị hiện tại của timer trong lúc đang hoạt động.
- Thanh ghi nạp lại hoặc so sánh dùng để đặt giá trị mốc. Khi counter đạt đến giá trị này thì timer có thể phát ngắt, tự nạp lại trong chế độ periodic, hoặc dùng để xác định duty cycle trong chế độ PWM.
- Thanh ghi prescaler dùng để chia nhỏ tần số clock đầu vào, giúp tạo được khoảng thời gian dài hơn.

Ở chế độ one-shot, timer chỉ đếm một lần từ giá trị ban đầu đến giá trị đặt trước, sau đó phát ngắt hoặc tín hiệu rồi dừng lại, thường dùng để tạo trễ cố định. Với chế độ periodic, timer đếm đến giá trị so sánh, phát ngắt, tự nạp lại và tiếp tục đếm để tạo chu kỳ lặp. Chế độ PWM tạo sóng vuông với duty cycle phụ thuộc vào giá trị compare, dùng để điều khiển độ sáng LED hoặc tốc độ động cơ. Ngoài ra, input capture dùng để ghi lại thời điểm có xung ngoài đưa vào để đo tần số hoặc độ rộng

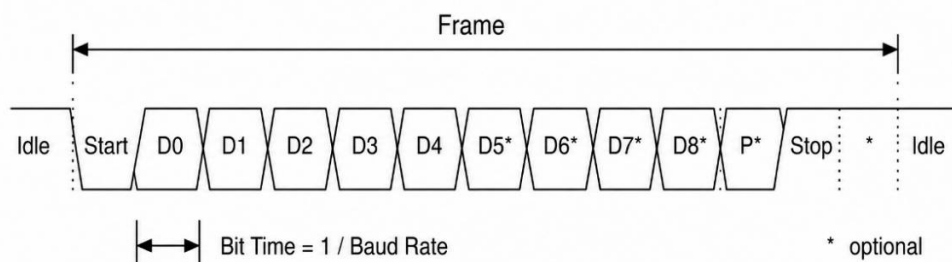
xung, còn output compare dùng để đổi trạng thái ngõ ra hoặc phát ngắt khi counter đạt giá trị đã cấu hình.

2.4.6. UART (Universal Asynchronous Receiver Transmitter)

UART là một chuẩn giao tiếp nối tiếp bất đồng bộ, thường dùng để truyền dữ liệu giữa các hệ thống máy tính. Nguyên lý của UART như Hình 2.6 là không dùng chung đường clock, mà hai bên phải cấu hình cùng baud rate để truyền và nhận đúng dữ liệu. Giao tiếp UART thường có hai đường chính là TX để truyền và RX để nhận. Một frame dữ liệu thường gồm bit start ở mức thấp để báo bắt đầu, 5 đến 8 bit dữ liệu, thông dụng nhất là 8 bit, có thể có thêm bit parity để kiểm tra lỗi chẵn lẻ, và 1 hoặc 2 bit stop ở mức cao để báo kết thúc frame.

Trong nhiều vi điều khiển, UART có tích hợp FIFO cho cả truyền và nhận để tạm lưu dữ liệu, nhờ vậy CPU không cần xử lý từng byte quá gấp và giảm khả năng mất dữ liệu. Các thanh ghi điều khiển của UART cho phép cấu hình số bit dữ liệu, parity, số bit stop, baud rate, cũng như bật hoặc tắt ngắt khi bộ truyền rỗng hoặc bộ nhận có dữ liệu.

- **Quá trình truyền:** CPU ghi một byte dữ liệu vào thanh ghi truyền TX Holding Register. Sau đó UART tự thêm bit start, parity nếu có, bit stop rồi gửi dữ liệu ra chân TX theo baud rate đã cấu hình. Khi thanh ghi truyền trống, UART có thể tạo ngắt TX Empty để CPU nạp byte tiếp theo.
- **Quá trình nhận:** UART bắt đầu nhận khi phát hiện tín hiệu start, tức đường RX chuyển từ mức 1 xuống 0. Sau đó UART lần lượt đọc các bit dữ liệu, bit parity nếu có và bit stop. Nếu frame hợp lệ, byte nhận được sẽ được đưa vào RX FIFO, đồng thời cờ RX Ready hoặc RX Full được bật và có thể phát ngắt cho CPU xử lý.



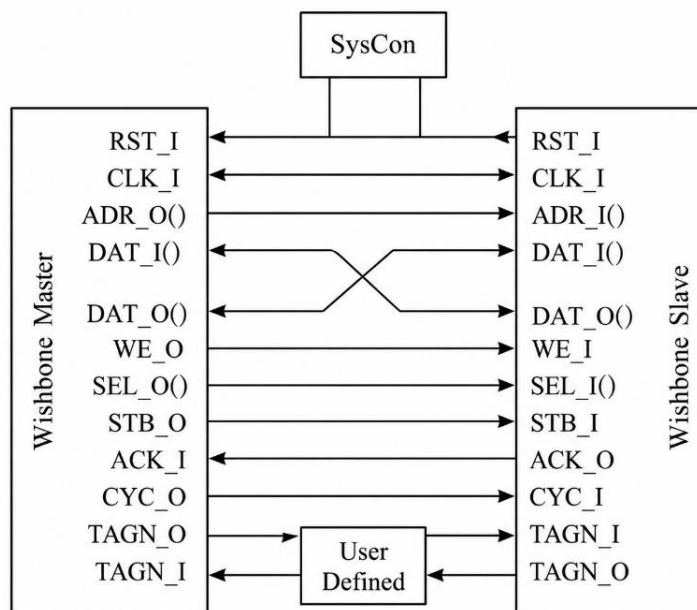
Hình 2.6. Waveform của giao tiếp UART

2.4.7. Bus Wishbone

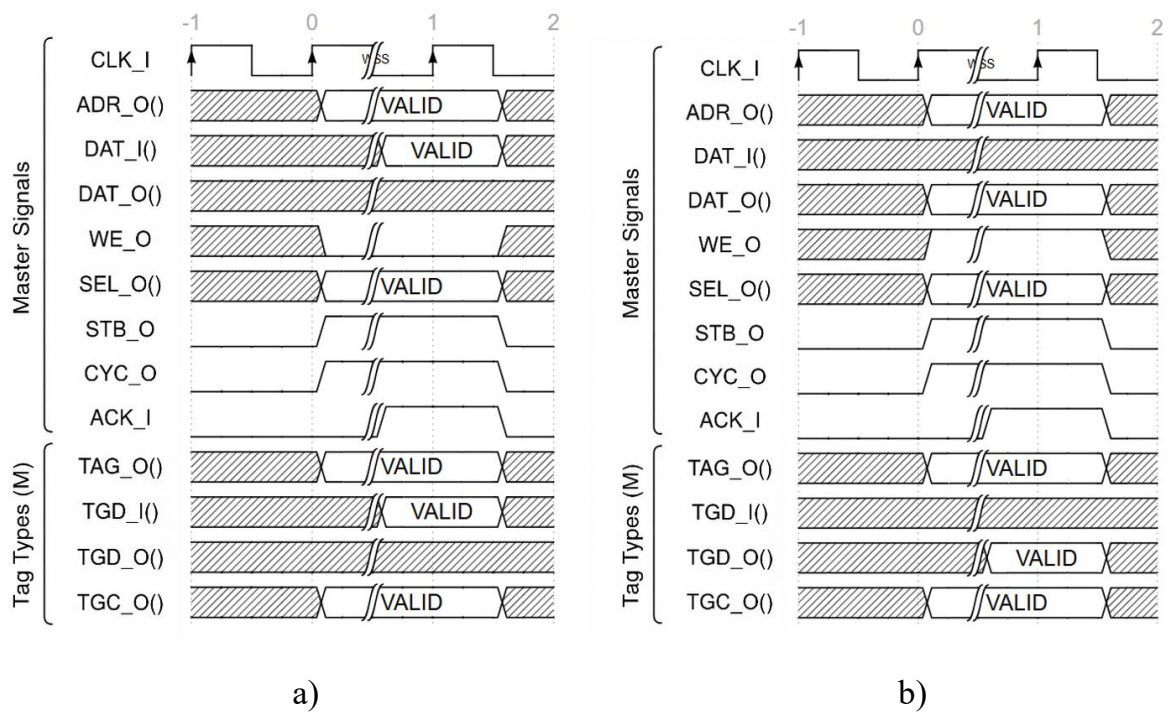
Wishbone là một kiến trúc bus mã nguồn mở do Silicore Corporation phát triển, dùng để chuẩn hóa cách giao tiếp giữa các IP core trong thiết kế SoC. Mục đích chính của bus này là giúp các khối IP có thể tái sử dụng dễ hơn nhờ dùng chung một kiểu giao diện, từ đó giảm thời gian tích hợp hệ thống và làm thiết kế linh hoạt hơn.

Wishbone không yêu cầu phí bản quyền và cũng không cần đăng ký sử dụng, nên phù hợp cho dự án học thuật. Bus này có thể hỗ trợ nhiều độ rộng dữ liệu khác nhau như 8, 16, 32 hoặc 64 bit, tùy yêu cầu. Các tín hiệu trong Wishbone thường đồng bộ theo cùng một xung nhịp, nên việc thiết kế và kiểm tra timing đơn giản hơn. Ngoài ra, Wishbone chỉ quy định các tín hiệu logic và giao thức truyền nhận, chứ không ràng buộc lớp vật lý, vì vậy có thể triển khai trên nhiều nền tảng khác nhau.

Một số tín hiệu cơ bản của bus Wishbone gồm cyc, stb, we, sel, adr, dat_i, dat_o, ack và err. Các tín hiệu này dùng để master và slave trao đổi địa chỉ, dữ liệu và điều khiển quá trình đọc ghi. Wishbone phù hợp cho cả hệ thống đơn giản lẫn hệ thống có nhiều master và slave. Nhờ thiết kế đơn giản, dễ dùng và miễn phí bản quyền, Wishbone được dùng khá nhiều trong các thiết kế FPGA, ASIC và các hệ thống nhúng. Hình 2.7 và Hình 2.8 minh họa cách kết nối cũng như quá trình đọc/ghi giữa master và slave trong hệ thống Wishbone bus.



Hình 2.7. Minh họa kết nối giữa Master và Slave

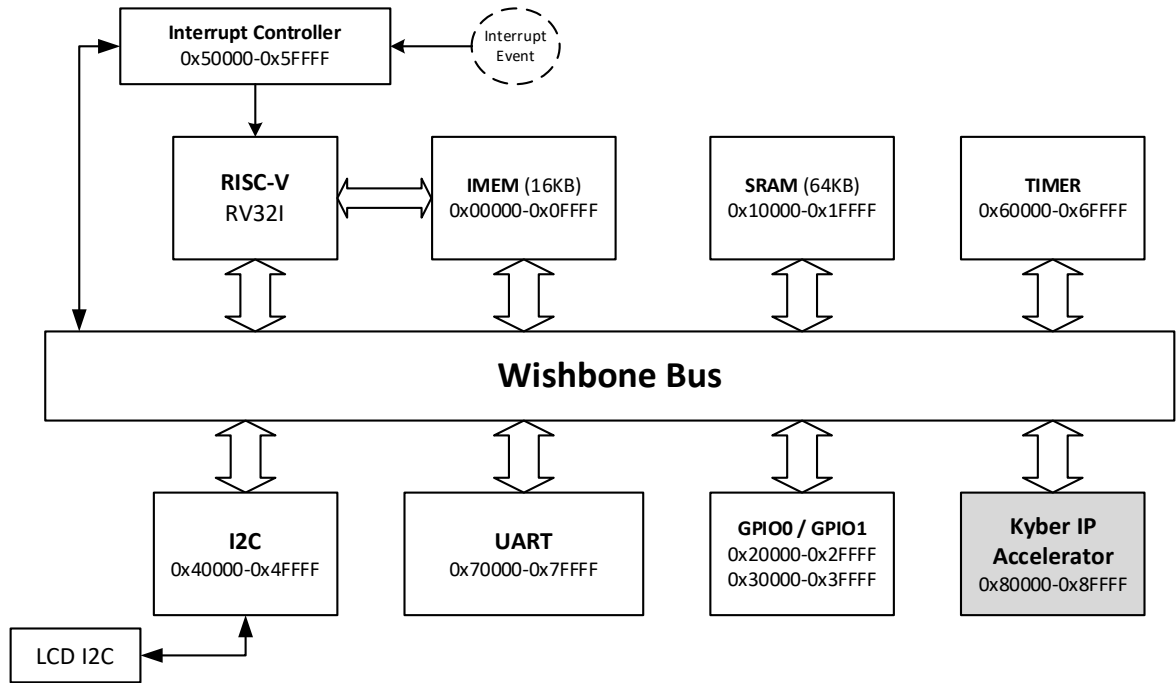


Hình 2.8. a) Quá trình Read của Wishbone b) Quá trình Write của Wishbone

CHƯƠNG 3

THIẾT KẾ VÀ TRIỂN KHAI ĐỀ TÀI

3.1. VI ĐIỀU KHIỂN RISC-V



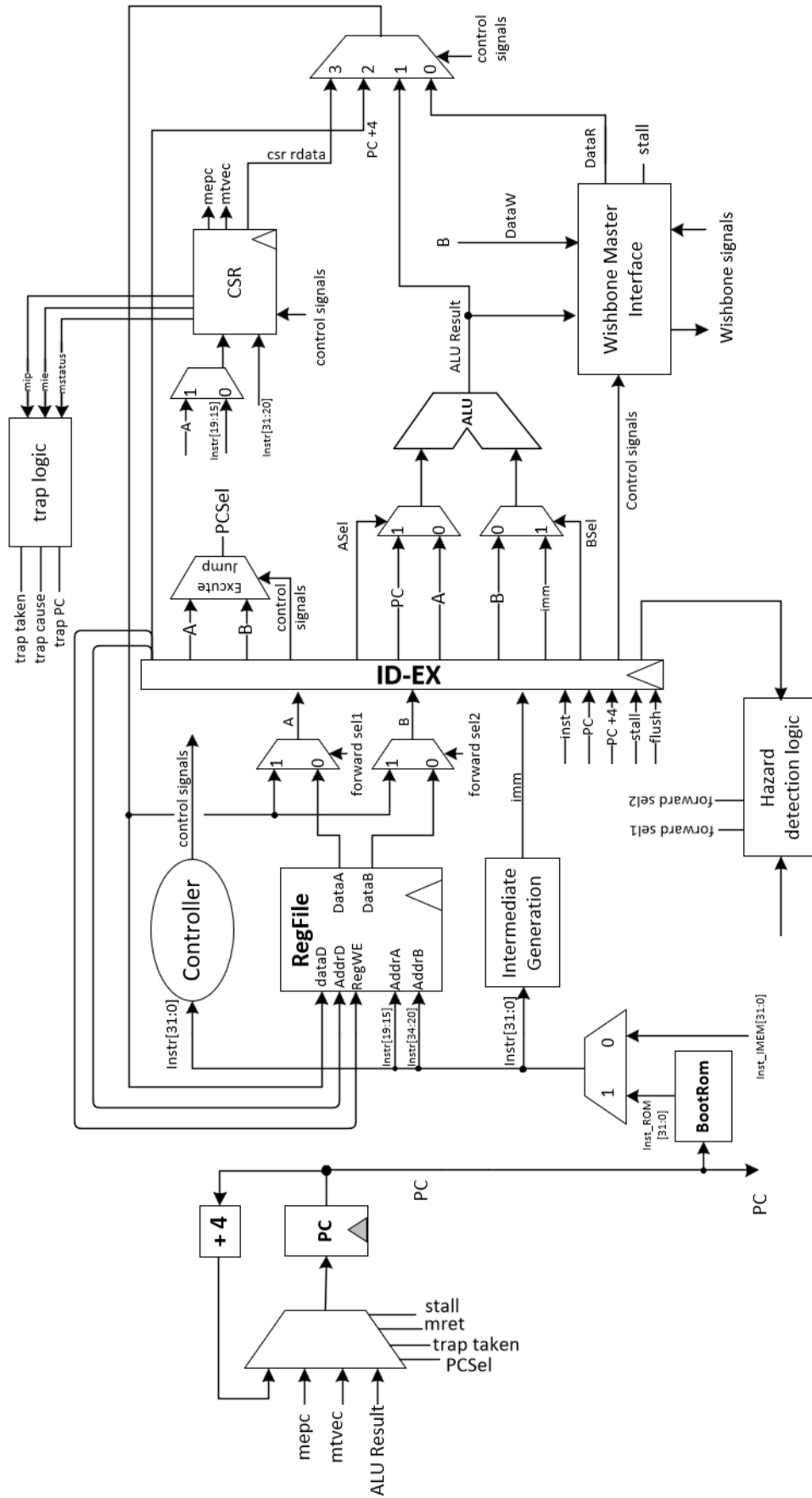
Hình 3.1. Kiến trúc tổng quan hệ thống SoC

Hình 3.1 trình bày kiến trúc tổng quan của hệ thống vi điều khiển được xây dựng dựa trên lõi xử lý RISC-V RV32I. Đồng thời được tích hợp giao diện Wishbone B4 để kết nối với bộ nhớ và các ngoại vi trong hệ thống. Bên cạnh tập lệnh cơ sở RV32I, lõi xử lý hỗ trợ các lệnh CSR thuộc phần mở rộng Zicsr và tập thanh ghi điều khiển.

Hệ thống được trang bị các khối ngoại vi và bộ tăng tốc như sau:

- **UART:** Hỗ trợ cấu hình tốc độ baud, độ dài dữ liệu, parity và stop bit.
- **GPIO 0/1:** 16 kênh, cấu hình linh hoạt cho các thiết bị ngoại vi.
- **TIMER:** Cung cấp chức năng đếm thời gian và tạo ngắt định kỳ.
- **I2C:** Giao tiếp với thiết bị ngoại vi, cảm biến hoặc bộ nhớ ngoài.
- **Interrupt Controller:** Quản lý và ưu tiên hóa ngắt từ các ngoại vi.
- **Kyber IP:** Sinh khoá, đóng gói khoá và giải gói khoá (chi tiết ở mục 3.2).

3.1.1. Kiến trúc vi xử lý



Hình 3.2. Kiến trúc lõi RISC-V RV32 [9]

Hình 3.2 mô tả thiết kế của lõi RISC-V RV32I, được xây dựng dựa theo tài liệu RISC-V [9]. Lõi có thêm phần mở rộng Zicsr để hỗ trợ các thanh ghi điều khiển và trạng thái, đồng thời dùng Machine-mode privileged architecture cho xử lý ngắt và bẫy. Kiến trúc hướng đến ứng dụng nhúng, nên luồng dữ liệu và điều khiển được tổ chức khá rõ.

Tổng quan lõi RISC-V RV32I được thiết kế theo pipeline 2 giai đoạn. Giai đoạn 1 gồm Instruction Fetch (IF) và Instruction Decode (ID). Ở IF, lệnh được lấy từ IMEM hoặc BootROM; còn ở ID, lệnh được giải mã, đọc dữ liệu từ RegFile, đồng thời sinh tín hiệu điều khiển và immediate. Giai đoạn 2 gồm Execute (EX), Memory Access (MEM) và Write Back (WB). EX thực hiện phép toán số học/logic, MEM truy cập bộ nhớ qua Wishbone hoặc xử lý CSR, còn WB ghi kết quả về RegFile. Thanh ghi ID_EX đồng bộ dữ liệu giữa hai giai đoạn, giúp giảm độ trễ và hỗ trợ tăng tần số xung nhịp. Các xung đột dữ liệu, điều khiển và ngắt được xử lý qua CSR, trap_logic và tín hiệu stall/flush.

Các thành phần chính của lõi gồm Program Counter (PC), Controller, RegFile, Immediate Generation, Forwarding Logic, ID_EX, ALU, Jump_Execute, CSR, Trap Logic và Hazard Detection Logic. PC lưu địa chỉ lệnh hiện tại và cập nhật dựa trên ALU_Result, mepc, mtvec, PCSel, mret, trap_taken và stall. Lệnh từ IMEM hoặc BootROM được đưa sang ID để giải mã, Controller tạo tín hiệu điều khiển, RegFile xuất DataA/DataB, còn Immediate Generation tạo giá trị hằng số. Forwarding Logic dùng MUX để xử lý xung đột dữ liệu khi lệnh sau cần dùng kết quả của lệnh trước. Trong EX, ALU thực hiện phép toán, Jump_Execute tạo tín hiệu cập nhật PC; MEM truy cập Wishbone hoặc CSR, còn WB chọn DataD ghi vào RegFile. Trap Logic tạo trap_taken, trap_cause, trap_pc, còn Hazard Detection Logic sinh stall hoặc flush.

Về luồng dữ liệu và điều khiển, dữ liệu đi từ IF và ID, qua ID_EX, rồi đến EX, MEM và WB. Khi có ngắt, hệ thống lưu địa chỉ lệnh hiện tại vào mepc, nhảy đến địa chỉ xử lý ngắt trong mtvec và flush ID_EX. Khi gặp lệnh mret, trạng thái được khôi phục và mepc được đưa vào PC để tiếp tục chương trình trước ngắt. Với lệnh nhảy, hệ thống cập nhật PC và flush ID_EX để tránh thực thi sai lệnh. Khi truy cập Wishbone bus mà chưa có ack, lõi sẽ tạo stall để tạm dừng.

Phần xử lý ngắt được thực hiện thông qua trap_logic và các thanh ghi CSR. Trap được phát hiện dựa trên trap_logic, CSR và trạng thái các bit MIP, MIE, trong đó ngoại lệ được ưu tiên trước ngắt. Khi trap xảy ra, trap_logic xuất trap_taken, trap_cause và trap_pc. CSR lưu trap_pc vào mepc, ghi nguyên nhân vào mcause và cập nhật mstatus. Khi kết thúc bằng lệnh mret, MIE được khôi phục từ MPIE, đồng thời PC được cập nhật từ mepc để chương trình tiếp tục chạy.

3.1.2. Kiến trúc bộ nhớ

Lỗi xử lý truy xuất lệnh trực tiếp từ IMEM. Dữ liệu thực thi được lưu trong SRAM, chương trình chính được biên dịch và truyền từ PC qua UART dưới dạng payload, và BootROM chứa bootloader nạp firmware từ máy tính và ghi payload vào vùng Upload IMEM. Chiến lược phân vùng bộ nhớ được trình bày trong Bảng 3.1.

- **Firmware source:** được truyền từ PC vào thông qua UART Bootloader.
- **IMEM:** 16 KiB, chứa chương trình thực thi sau khi upload.
- **SRAM:** 16 KiB, chứa .data, .bss, heap và stack.
- **BootROM:** 16 KB, chứa UART Bootloader, reset vector và mã khởi động.

Bảng 3.1: Phân vùng bộ nhớ

IMEM (16 KB)

Vùng	Địa chỉ bắt đầu	Kích thước	Nội dung
Firmware	0x0000_4000	16 KB	Chương trình chính nạp từ PC qua UART Bootloader

SRAM (16 KB)

Vùng	Địa chỉ bắt đầu	Kích thước	Nội dung
.data(run)	0x0001_0000	0 KB	Bản sao biến khởi tạo trong bộ nhớ chương trình
.bss	0x0001_0000	3.344 KB	Biến chưa khởi tạo, được xoá về 0 khi khởi động
Heap	0x0001_0D60	0 KB	Vùng cấp phát động
Stack	0x0001_0D60	12.656 KB	Lưu tạm ngăn xếp

BootROM (16 KB)

Vùng	Địa chỉ bắt đầu	Kích thước	Nội dung
Bootloader	0x0000_0000	16 KB	UART Bootloader, mã khởi động và reset vector

3.1.3. Hệ thống Bus (Interconnect)

Trong hệ thống vi điều khiển của đồ án này, CPU, các khối nhớ, ngoại vi và bộ tăng tốc Kyber trao đổi dữ liệu thông qua bus Wishbone B4 rộng 32 bit, một bus nội bộ mã nguồn mở, miễn phí và có cấu trúc đơn giản dễ tùy chỉnh. Hậu tố B4 hiện là phiên bản mới nhất của Wishbone bus hỗ trợ vi xử lý kiến trúc pipeline. Vi điều khiển hiện tại sử dụng mô hình một master và nhiều slave. Lõi RISC-V là bus master duy nhất, phát các giao dịch lấy lệnh, load và store. Boot ROM/IMEM, SRAM, GPIO0, GPIO1, I2C, PIC, Timer, UART và bộ gia tốc Kyber là các slave, mỗi slave chiếm một slot địa chỉ như Bảng 3.2, đảm bảo tránh xung đột và hỗ trợ mở rộng hiệu quả.

Bảng 3.2: Sơ đồ ánh xạ địa chỉ (Address Map)

Ngoại vi	Địa chỉ Bus
BootROM / IMEM	0x0000_0000 – 0x0000_FFFF
SRAM	0x0001_0000 – 0x0001_FFFF
GPIO0	0x0002_0000 – 0x0002_FFFF
GPIO1	0x0003_0000 – 0x0003_FFFF
I2C	0x0004_0000 – 0x0004_FFFF
PIC	0x0005_0000 – 0x0005_FFFF
Timer	0x0006_0000 – 0x0006_FFFF
UART	0x0007_0000 – 0x0007_FFFF
Kyber Accelerator	0x0008_0000 – 0x0008_FFFF

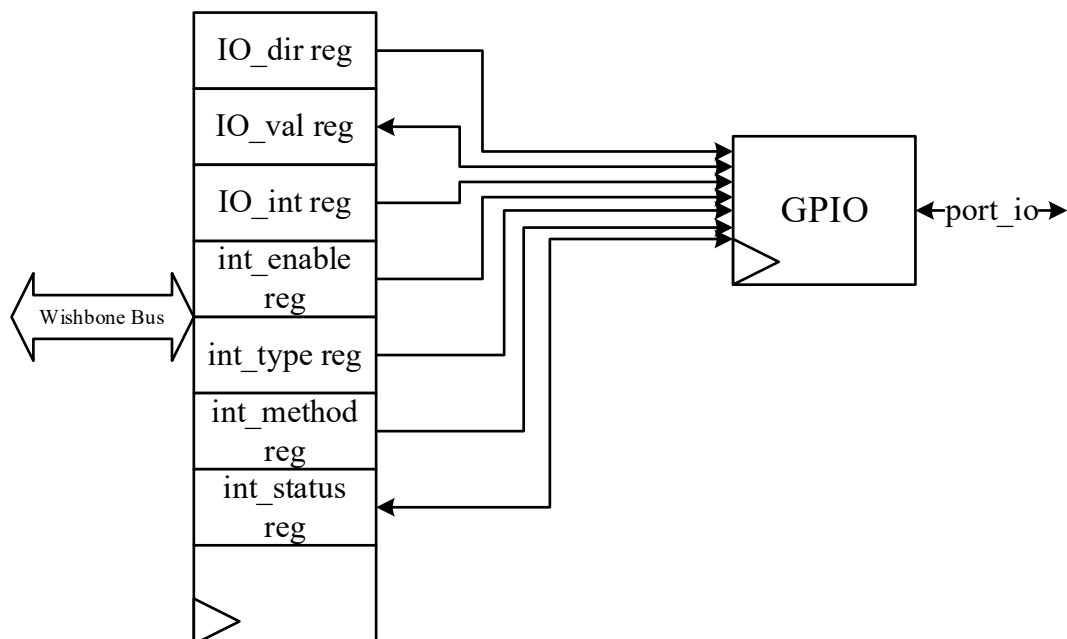
Bảng 3.3 trình bày các tín hiệu chính của giao thức Wishbone, thực hiện giải mã song song địa chỉ do master cung cấp. Khi địa chỉ thuộc một vùng hợp lệ, tín hiệu CYC và STB của slave tương ứng khởi tạo phiên truy cập, WE xác định thao tác đọc (0), ghi (1), ADR chỉ định địa chỉ, DAT_MISO, DAT_MOSI truyền dữ liệu vào/ra,

ACK xác nhận giao dịch hoàn tất, còn SEL chọn byte/halfword/word cụ thể. Quy trình hoạt động bắt đầu khi CPU (Master) kích hoạt CYC và STB, đặt địa chỉ ADR, dữ liệu ghi DAT_MOSI nếu cần và WE. Bộ giải mã địa chỉ chọn Slave và Slave xử lý yêu cầu và trả về ACK cho Master nhận kết quả và kết thúc giao dịch [10].

Bảng 3.3: Các tín hiệu chính của giao thức Wishbone

Tín hiệu	Hướng theo master	Chức năng
ADR[31:0]	Ra	Địa chỉ byte của giao dịch
DAT_MOSI[31:0]	Ra	Dữ liệu ghi từ CPU tới slave
DAT_MISO[31:0]	Vào	Dữ liệu đọc từ slave về CPU
SEL[3:0]	Ra	Chọn từng byte hợp lệ trong từ dữ liệu 32 bit
WE	Ra	0 là đọc, 1 là ghi
CYC	Ra	Cho biết một chu kỳ bus đang hoạt động
STB	Ra	Cho biết yêu cầu hiện tại hợp lệ
ACK	Vào	Slave xác nhận giao dịch hoàn tất
ERR	Vào	Slave hoặc interconnect báo giao dịch lỗi

3.1.4. GPIO



Hình 3.3. Sơ đồ khối tổng quan GPIO

Hình 3.3 minh hoạ sơ đồ khối của GPIO_WB, hệ thống có 2 instance gồm GPIO0 và GPIO1. Mỗi instance có 8 chân vào/ra, do đó toàn hệ thống cung cấp tổng cộng 16 kênh GPIO. Về giao diện, module GPIO_WB giao tiếp với vi xử lý cũng phải tuân theo nghiêm ngặt giao diện Wishbone đã được trình bày ở mục 3.1.3. Mỗi chân được cấu hình hướng độc lập bằng thanh ghi IO_DIR; ngõ ra dữ liệu và ngõ ra cho phép tương ứng được cung cấp qua các tín hiệu gpio_o[7:0] và gpio_oe[7:0]. Dữ liệu từ chân ngoài đi qua mạch đồng bộ hai tầng trước khi được CPU đọc hoặc đưa vào logic phát hiện ngắt.

Bảng 3.4: Địa chỉ các thanh ghi của GPIO0 và GPIO1

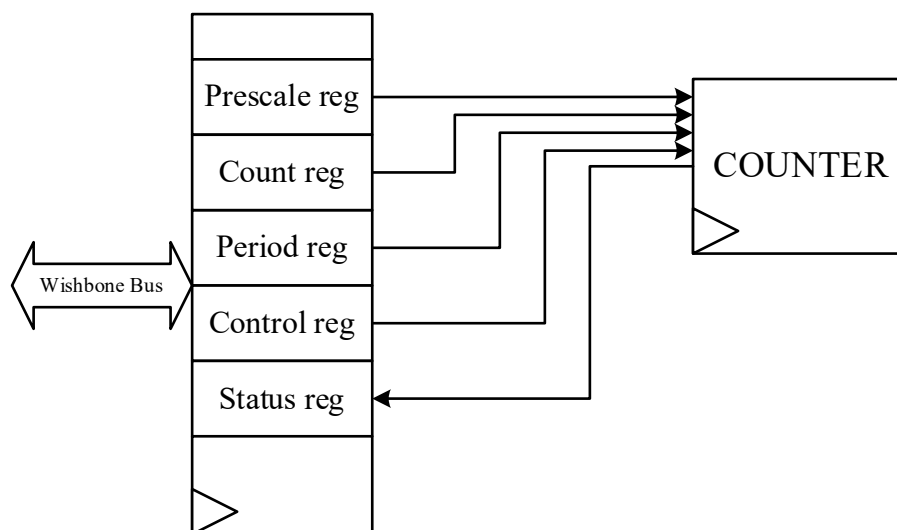
Tên thanh ghi	Địa chỉ	Tương tác	Chức năng
IO_DIR[7:0]	0x00	R/W	Chọn hướng từng chân: 0 input, 1 output
IO_VAL[7:0]	0x04	R/W	Ghi latch ngõ ra; khi đọc trả ngõ ra cho chân output và mẫu đồng bộ cho chân input
INT_ENABLE[7:0]	0x08	R/W	Cho phép ngắt theo từng chân
INT_TYPE[7:0]	0x0C	R/W	0 ngắt theo mức, 1 ngắt theo cạnh
INT_METHOD[7:0]	0x10	R/W	0 mức thấp/cạnh xuống, 1 mức cao/cạnh lên
INT_STATUS[7:0]	0x14	R/W	Trạng thái ngắt: 1 khi có ngắt, 0 khi không có ngắt; ghi 1 để xóa bit tương ứng
GPIO_SET[7:0]	0x1C	W	Đặt các bit ngõ ra tương ứng lên 1
GPIO_CLEAR[7:0]	0x20	W	Xóa các bit ngõ ra tương ứng về 0

Bảng 3.4 tóm tắt các thanh ghi điều khiển chủ chốt thuộc khối GPIO_WB, cung cấp chi tiết về độ lệch địa chỉ (offset) so với địa chỉ gốc cùng chức năng cụ thể của từng thanh ghi. Thông qua hệ thống thanh ghi này, bộ vi xử lý có thể thiết lập chiều dữ liệu (vào/ra), thực hiện các thao tác đọc/ghi dữ liệu, đồng thời kiểm soát tiến trình ngắt cho từng chân GPIO, giúp tối ưu hóa việc điều khiển các ngoại vi một cách linh hoạt.

Để ngăn chặn hiện tượng bất ổn định trạng thái (metastability) và duy trì tính ổn định của dữ liệu trước khi đưa vào xử lý, module GPIO_WB áp dụng phương thức đồng bộ tín hiệu ngõ vào thông qua cặp thanh ghi `sync_port_io_1` và `sync_port_io_2`. Việc cấu hình hướng cho các chân GPIO được thực hiện bởi thanh ghi `IO_DIR`: trong đó giá trị 0 quy định chân là ngõ vào, còn giá trị 1 định nghĩa chân là ngõ ra. Đối với các chân được cấu hình làm đầu ra, lỗi xử lý sẽ kiểm soát mức logic thông qua thanh ghi `IO_VAL`, hoặc có thể sử dụng các thanh ghi `GPIO_SET_BIT` và `GPIO_CLEAR_BIT` để can thiệp vào từng bit riêng lẻ. Đối với các chân ngõ vào, trạng thái sau khi đồng bộ sẽ được lỗi xử lý đọc trực tiếp từ thanh ghi `IO_VAL`.

Bên cạnh đó, module GPIO_WB còn tích hợp sẵn tính năng quản lý ngắt. Quá trình kích hoạt ngắt được điều hành bởi thanh ghi `INT_ENABLE`, với việc ghi giá trị 1 để cho phép ngắt trên chân GPIO tương ứng. Thanh ghi `INT_TYPE` chịu trách nhiệm phân loại dạng ngắt: giá trị 0 dành cho ngắt theo mức và giá trị 1 dành cho ngắt theo cạnh. Trong khi đó, thanh ghi `INT_METHOD` sẽ chỉ định cụ thể mức kích hoạt (thấp/cao) hoặc cạnh kích hoạt (xuống/lên). Khi xuất hiện sự kiện ngắt, bit tương ứng trong thanh ghi `INT_STATUS` sẽ tự động chuyển sang mức 1; lỗi xử lý thực hiện xóa cờ ngắt bằng cách ghi giá trị 1 vào bit này. Tín hiệu `int_o` sẽ chuyển lên mức cao để thông báo sự kiện ngắt khi cả hai bit `INT_STATUS[i]` và `INT_ENABLE[i]` đồng thời bằng 1.

3.1.5. TIMER



Hình 3.4. Sơ đồ khối TIMER

Hình 3.4 trình bày sơ đồ khối của module TIMER_WB, một ngoại vi định thời được tích hợp trong SoC để hỗ trợ các tác vụ liên quan đến thời gian, tạo sự kiện định kỳ và phát sinh ngắt cho CPU. Module này giao tiếp với RISC-V thông qua bus Wishbone và được ánh xạ vào không gian địa chỉ MMIO của hệ thống. Trong thiết kế SoC này, ngoại vi TIMER được đặt tại vùng địa chỉ nền 0x0006_0000; các thanh ghi bên trong module được truy cập thông qua địa chỉ offset so với địa chỉ nền này.

Sơ đồ khối TIMER có thể được biểu diễn gồm hai phần chính như Hình 3.4: khối thanh ghi Wishbone ở bên trái và khối Counter ở bên phải. Các thanh ghi CTRL_REG, COUNT_REG, PERIOD_REG và PRESCALER_REG cung cấp tín hiệu cấu hình cho Counter. Ngược lại, Counter cập nhật trạng thái hoạt động về COUNT_REG và tạo các cờ sự kiện trong STATUS_REG. Khi ngắt được bật và có sự kiện hợp lệ, module phát tín hiệu irq_o về bộ điều khiển ngắt của SoC.

Bảng 3.5. Địa chỉ các thanh ghi của TIMER

Tên thanh ghi	Offset	Truy cập	Chức năng
REG_CTRL[7:0]	0x00	R/W	*Thanh ghi điều khiển TIMER. - Bit 0 dùng để bật/tắt bộ đếm; - Bit 1 chọn chế độ đếm lên / xuống; - Bit 2 cho phép ngắt; - Bit 3 bật chế độ tự động nạp lại; - Bit 4 bật bộ chia tần prescaler; - Các bit còn lại không sử dụng và giữ giá trị 0.
REG_COUNT[15:0]	0x04	R/W	Thanh ghi đếm hiện tại của TIMER: CPU có thể đọc giá trị hiện tại của bộ đếm hoặc ghi giá trị ban đầu để khởi tạo bộ đếm.
REG_PERIOD[15:0]	0x08	R/W	Thanh ghi giá trị so sánh: Khi COUNT_REG đạt tới giá trị này, TIMER tạo cờ period trong

			STATUS_REG và có thể phát sinh ngắt nếu ngắt được bật.
REG_STATUS[7:0]	0x0C	R/W1C	*Thanh ghi trạng thái sự kiện: - Bit 0 là cờ đạt giá trị so sánh (period); - Bit 1 là cờ tràn trên (overflow); - Bit 2 là cờ tràn dưới (underflow); - CPU xóa cờ bằng cách ghi 1 vào bit tương ứng.
REG_PRESCALER [15:0]	0x10	R/W	Thanh ghi cấu hình bộ chia tần: Khi prescaler được bật, TIMER chỉ cập nhật bộ đếm sau khi bộ đếm prescaler đạt giá trị cấu hình.

Bảng 3.5 trình bày các thanh ghi điều khiển và trạng thái chính của Module TIMER_WB. Nó hoạt động dựa trên thanh ghi CTRL_REG. Khi bit enable trong CTRL_REG[0] được đặt lên 1, bộ đếm bắt đầu hoạt động. Nếu CTRL_REG[1] bằng 0, TIMER hoạt động ở chế độ đếm lên; giá trị COUNT_REG tăng dần sau mỗi nhịp đếm hợp lệ. Nếu CTRL_REG[1] bằng 1, TIMER hoạt động ở chế độ đếm xuống; giá trị COUNT_REG giảm dần sau mỗi nhịp đếm hợp lệ.

Thanh ghi PERIOD_REG được dùng làm giá trị so sánh. Trong quá trình hoạt động, nếu giá trị bộ đếm đạt tới giá trị trong PERIOD_REG, bit period trong STATUS_REG được set. Nếu chế độ tự động nạp lại được bật thông qua CTRL_REG[3], bộ đếm sẽ được nạp lại giá trị ban đầu để tiếp tục tạo sự kiện định kỳ. Chế độ này phù hợp cho các tác vụ cần tạo tick thời gian lặp lại, ví dụ tạo delay, đo chu kỳ hoặc phát sinh ngắt định kỳ cho firmware.

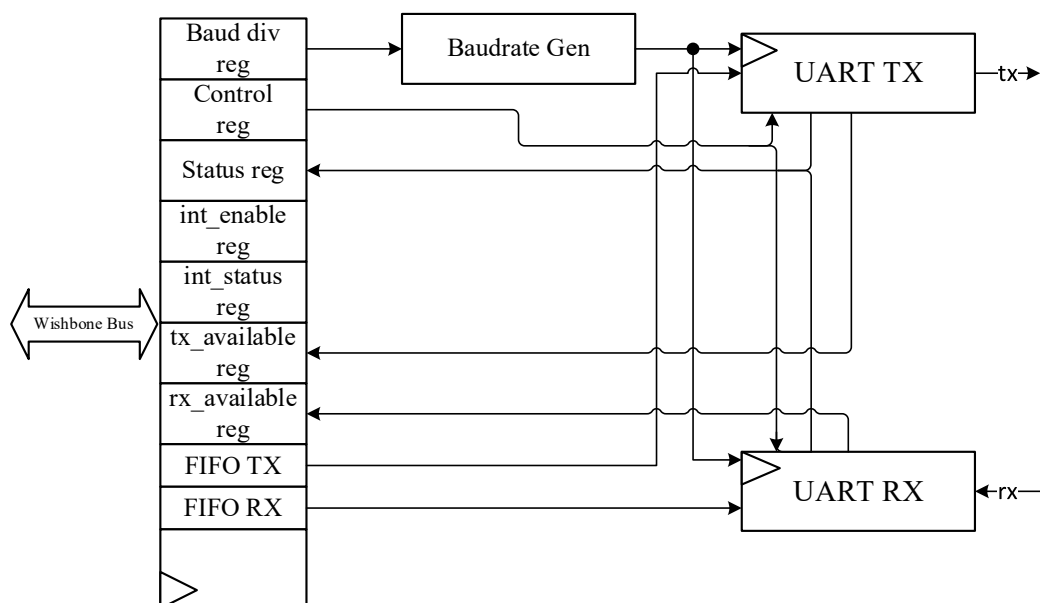
Ngoài sự kiện đạt period, TIMER còn có thể ghi nhận trạng thái tràn. Trong chế độ đếm lên, nếu bộ đếm vượt quá giá trị lớn nhất thì cờ overflow trong STATUS_REG được set. Trong chế độ đếm xuống, nếu bộ đếm giảm qua giá trị 0 thì cờ underflow được set. Các cờ trạng thái này giúp CPU nhận biết những sự kiện thời gian khác nhau trong quá trình chạy chương trình.

Để điều chỉnh tốc độ đếm, module TIMER tích hợp bộ chia tần prescaler. Khi bit CTRL_REG[4] được bật, bộ đếm chính không cập nhật ở mọi chu kỳ clock hệ thống mà chỉ cập nhật sau khi bộ đếm prescaler đạt tới giá trị trong PRESCALER_REG. Nhờ đó, firmware có thể tạo ra các khoảng thời gian dài hơn mà không cần dùng bộ đếm chính quá rộng. Khi prescaler không được bật, TIMER đếm trực tiếp theo clock hệ thống.

Cơ chế ngắt của TIMER được điều khiển bởi bit interrupt enable trong CTRL_REG[2]. Khi bit này được bật, nếu có bất kỳ cờ sự kiện nào trong STATUS_REG được set, module sẽ kích hoạt tín hiệu irq_o. Tín hiệu này được đưa tới bộ điều khiển ngắt của SoC để CPU có thể phản hồi sự kiện thời gian. CPU có thể đọc STATUS_REG để xác định nguyên nhân gây ngắt và xóa cờ bằng cách ghi 1 vào bit tương ứng.

Trong SoC, TIMER không tham gia trực tiếp vào datapath tính toán của Kyber IP, nhưng đóng vai trò là ngoại vi hệ thống hỗ trợ firmware. TIMER có thể được dùng để tạo delay, đo thời gian thực thi tương đối, kiểm tra cơ chế ngắt hoặc tạo tick định kỳ trong các chương trình demo. Việc tích hợp TIMER cùng với UART, GPIO, I2C, PIC và Kyber IP cho thấy SoC không chỉ là một accelerator rời rạc mà là một hệ thống RISC-V có đầy đủ các ngoại vi MMIO cơ bản.

3.1.6. UART



Hình 3.5. Sơ đồ khối UART

Hình 3.5 biểu diễn sơ đồ khối tổng thể của module UART_WB, là ngoại vi giao tiếp nối tiếp bất đồng bộ được tích hợp trong SoC. Module này cho phép CPU giao tiếp với máy tính thông qua đường truyền UART, phục vụ hai mục đích chính: nạp firmware qua UART bootloader và xuất thông tin debug/demo trong quá trình kiểm thử trên FPGA.

Module UART_WB giao tiếp với CPU thông qua bus Wishbone. Các thanh ghi điều khiển, trạng thái, bộ đệm truyền/nhận và bộ chia baud rate được ánh xạ vào không gian địa chỉ MMIO của SoC. Về giao diện ngoại vi, module sử dụng tín hiệu `uart_rx_i` để nhận dữ liệu nối tiếp từ máy tính và tín hiệu `uart_tx_o` để truyền dữ liệu nối tiếp ra ngoài. Ngoài ra, module còn tạo tín hiệu `irq_o` để báo ngắt khi có các sự kiện truyền/nhận hoặc lỗi UART được bật trong thanh ghi ngắt.

Bảng 3.6: Địa chỉ các thanh ghi của UART

Tên thanh ghi	Offset	Truy cập	Chức năng
TX_BUFFER [7:0]	0x00	W	Bộ đệm truyền. CPU ghi dữ liệu 8-bit vào thanh ghi này để đưa byte vào FIFO truyền.
RX_BUFFER [7:0]	0x04	R	Bộ đệm nhận. CPU đọc thanh ghi này để lấy một byte từ FIFO nhận.
CONTROL_REG [7:0]	0x08	R/W	*Thanh ghi điều khiển UART: - Bit 7 bật/tắt UART; - Bit 6 xóa FIFO truyền khi ghi 1; - Bit 5 xóa FIFO nhận và các cờ lỗi khi ghi 1; - Bit [4:3] chọn parity (00: không, 01: even, 10: odd); - Bit 2 chọn số stop bit (0: 1 bit, 1: 2 bit); - Bit [1:0] chọn số bit dữ liệu (00: 5 bit, 01: 6 bit, 10: 7 bit, 11: 8 bit).

STATUS_REG [7:0]	0x0C	R	*Thanh ghi trạng thái: - Bit 7 báo truyền đang chờ hoặc đang hoạt động (transmit_pending); - Bit 6 FIFO nhận đầy; - Bit 5 FIFO nhận rỗng; - Bit 4 FIFO truyền đầy; - Bit 3 FIFO truyền rỗng; - Bit 2 lỗi parity (parity error); - Bit 1 lỗi tràn FIFO nhận (overrun); - Bit 0 lỗi khung. Khi đọc thanh ghi này, các cờ lỗi parity, overrun và framing được xóa.
AVAILABLE_TX [5:0]	0x10	R	Số byte hiện có trong FIFO truyền, giá trị từ 0 đến 32.
AVAILABLE_RX [5:0]	0x14	R	Số byte hiện có trong FIFO nhận, giá trị từ 0 đến 32.
INT_STATUS [5:0]	0x18	R	*Trạng thái nguồn ngắt UART: - Bit 5 FIFO nhận có dữ liệu; - Bit 4 FIFO nhận đầy; - Bit 3 FIFO truyền rỗng; - Bit 2 lỗi parity; - Bit 1 lỗi tràn FIFO nhận; - Bit 0 lỗi khung.
INT_ENABLE [5:0]	0x1C	R/W	Cho phép từng nguồn ngắt UART: Các bit có ý nghĩa tương ứng với INT_STATUS[5:0]. Khi một bit trong INT_STATUS và INT_ENABLE cùng bằng 1, tín hiệu irq_o được kích hoạt.
BAUD_DIV [15:0]	0x20	R/W	Thanh ghi chia tần tạo tốc độ baud. Giá trị xác định chu kỳ xung lấy mẫu cho UART TX/RX.

Bảng 3.6 liệt kê các thanh ghi điều khiển và trạng thái của module UART_WB, gồm địa chỉ offset và chức năng của từng thanh ghi. Thông qua các thanh ghi này, lõi xử lý có thể cấu hình baud rate, định dạng khung truyền, điều khiển ngắt, gửi/nhận dữ liệu và theo dõi trạng thái hoạt động của UART.

Module UART_WB có hai chức năng chính là truyền dữ liệu TX và nhận dữ liệu RX. Ở phía TX, dữ liệu do CPU ghi vào thanh ghi TX_BUFFER sẽ được đẩy vào FIFO truyền. Khi UART được cho phép hoạt động và FIFO truyền còn dữ liệu, khối UART TX sẽ lấy từng byte từ FIFO, chuyển thành chuỗi bit nối tiếp và phát ra ngoài qua chân `uart_tx_o`. Phía RX, tín hiệu nối tiếp đi vào từ chân `uart_rx_i` sẽ được đồng bộ, lấy mẫu và giải mã bởi khối UART RX, sau đó dữ liệu nhận được được lưu vào FIFO nhận. CPU có thể đọc dữ liệu này thông qua thanh ghi RX_BUFFER [7].

Module UART_WB hỗ trợ FIFO truyền và FIFO nhận, mỗi FIFO có dung lượng 32 byte. Trong quá trình truyền, khi CPU ghi một byte vào TX_BUFFER, dữ liệu được lưu vào FIFO truyền. Nếu UART đang được bật, khối TX sẽ tự động lấy byte từ FIFO khi đang rỗi, phát start bit, các bit dữ liệu, bit parity nếu được cấu hình, và một hoặc hai stop bit tùy theo CONTROL_REG. Tín hiệu STATUS_REG[7] được dùng để cho biết UART đang còn dữ liệu chờ truyền hoặc khối TX chưa hoàn tất frame hiện tại.

Trong quá trình nhận, tín hiệu `uart_rx_i` trước hết được đồng bộ qua hai tầng thanh ghi nội bộ nhằm giảm nguy cơ metastability. Khối RX phát hiện cạnh xuống của start bit, lấy mẫu ở giữa bit start, sau đó tiếp tục lấy mẫu từng bit dữ liệu theo xung baud nội bộ. Nếu parity được bật, UART kiểm tra bit parity và set cờ `parity_error` khi phát hiện sai lệch. Nếu stop bit không ở mức cao, cờ `framing_error` được set. Khi một frame hợp lệ hoặc đã nhận xong, byte dữ liệu được đưa vào FIFO nhận. Nếu FIFO nhận đầy mà vẫn có dữ liệu mới đến, cờ `overrun_error` được set.

Tốc độ truyền UART được tạo bởi khối Baudrate Gen. Thanh ghi BAUD_DIV lưu giá trị chia tần 16-bit. Bộ tạo baud sinh xung `baud_tick` sau mỗi BAUD_DIV chu kỳ clock hệ thống. Vì khối UART sử dụng lấy mẫu 16 lần trên mỗi bit, tốc độ baud xấp xỉ được xác định bởi công thức:

$$\text{baud_div} = \text{clk}/(16*\text{baudrate}) \quad (3.1)$$

Trong firmware, giá trị BAUD_DIV được tính từ tần số clock SoC và tốc độ baud mong muốn. Với cấu hình demo, UART thường được khởi tạo ở chế độ 8N1, tức 8 bit dữ liệu, không parity và 1 stop bit.

Cơ chế ngắt của UART được điều khiển bởi hai thanh ghi INT_STATUS và INT_ENABLE. INT_STATUS phản ánh các sự kiện như FIFO nhận có dữ liệu, FIFO nhận đầy, FIFO truyền rỗng, lỗi parity, lỗi overrun và lỗi framing. INT_ENABLE cho phép bật hoặc tắt từng nguồn ngắt tương ứng. Tín hiệu irq_o được kích hoạt khi tồn tại ít nhất một bit trong INT_STATUS đồng thời được bật trong INT_ENABLE.

Trong SoC, UART là ngoại vi quan trọng vì được sử dụng bởi bootloader để nhận firmware từ máy tính. Sau khi FPGA được cấu hình và CPU bắt đầu chạy từ BootROM, firmware trên PC được gửi qua UART theo định dạng gói bootloader. Bootloader nhận dữ liệu qua RX_BUFFER, kiểm tra gói và ghi payload vào Upload IMEM. Ngoài chức năng nạp chương trình, UART còn được dùng để in log kiểm thử như trạng thái boot, kết quả chạy Kyber, chu kỳ thực thi và thông báo pass/fail trong quá trình demo trên board ZCU102.

3.1.7. Interrupt Controller

Bộ điều khiển ngắt, hay Programmable Interrupt Controller (PIC_WB), là ngoại vi trung tâm dùng để thu thập, lưu trạng thái và tổng hợp các yêu cầu ngắt từ nhiều nguồn khác nhau trong SoC. Thay vì để CPU phải kiểm tra trực tiếp từng ngoại vi, module PIC gom các nguồn ngắt lại thành một tín hiệu ngắt tổng irq_o, đồng thời cung cấp các thanh ghi trạng thái để firmware xác định nguồn gây ngắt và xóa cờ sau khi xử lý.

Trong thiết kế SoC này, PIC_WB giao tiếp với CPU thông qua bus Wishbone và được ánh xạ vào không gian địa chỉ MMIO tại địa chỉ nền 0x0005_0000. Module hỗ trợ 8 nguồn ngắt đầu vào thông qua tín hiệu irq_sources_i[7:0]. Các nguồn ngắt này bao gồm các ngoại vi nội bộ như GPIO, I2C, Timer, UART, Kyber IP và hai nguồn ngắt ngoài. Khi một nguồn ngắt được kích hoạt, PIC sẽ chốt trạng thái pending tương ứng vào thanh ghi PIC_STATUS. Nếu bit đó đồng thời được bật trong thanh ghi PIC_ENABLE, tín hiệu ngắt tổng irq_o sẽ được kích hoạt để báo cho CPU.

Bảng 3.7: Địa chỉ các thanh ghi của Interrupt Controller

Tên thanh ghi	Offset	Truy cập	Chức năng
PIC_STATUS[7:0]	0x00	R/W1C	Thanh ghi trạng thái ngắt pending: - Bit [7:0]: mỗi bit tương ứng 1 nguồn. Khi nguồn ngắt đầu vào được kích hoạt, bit tương ứng sẽ được set và giữ lại. - CPU ghi 1 vào bit tương ứng để xóa cờ ngắt.
PIC_ENABLE[7:0]	0x04	R/W	Thanh ghi cho phép ngắt/bật tắt ngắt: - Bit [7:0]: mỗi bit dùng để bật/tắt nguồn tương ứng; 1: cho phép, 0: vô hiệu hóa.
PIC_RAW[7:0]	0x08	R	Thanh ghi đọc trạng thái ngắt thô từ <code>irq_sources_i[7:0]</code>. Thanh ghi này phản ánh trực tiếp mức hiện tại của các nguồn ngắt đầu vào và không có tính sticky.

Về kiến trúc, module PIC gồm ba khối chính: cụm thanh ghi Wishbone, logic chốt trạng thái ngắt pending và logic tổng hợp ngắt. Bảng 3.7 liệt kê các thanh ghi điều khiển và trạng thái chính của module Interrupt Controller. Thanh ghi PIC_STATUS lưu các cờ ngắt đã xảy ra theo cơ chế sticky pending. Thanh ghi PIC_ENABLE đóng vai trò mask để bật hoặc tắt từng nguồn ngắt. Thanh ghi PIC_RAW cho phép CPU đọc trực tiếp trạng thái tức thời của các nguồn ngắt đầu vào trước khi qua cơ chế chốt pending và enable mask.

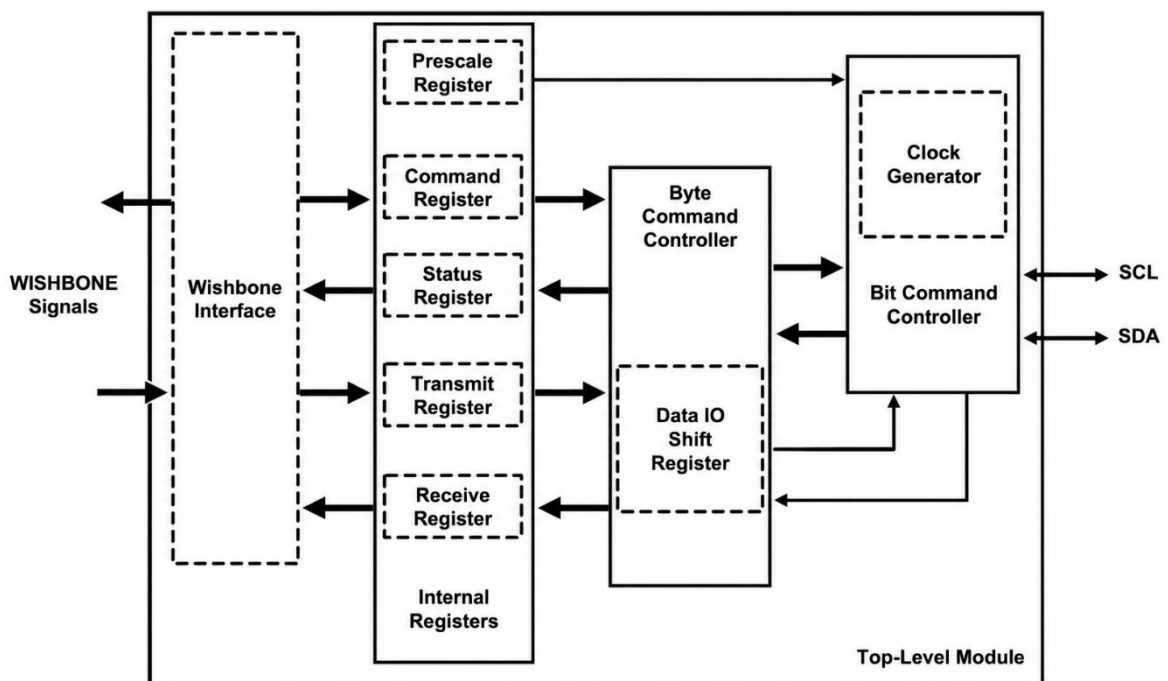
Nguyên lý hoạt động của PIC_WB dựa trên cơ chế chốt cờ ngắt. Ở mỗi chu kỳ clock, giá trị hiện tại của `irq_sources_i[7:0]` được OR vào thanh ghi PIC_STATUS. Vì vậy, khi một nguồn ngắt từng được kích hoạt, bit pending tương ứng trong PIC_STATUS sẽ giữ ở mức 1 cho đến khi CPU chủ động xóa. Cách tổ chức này tránh mất ngắt trong trường hợp xung ngắt từ ngoại vi chỉ tồn tại trong thời gian ngắn.

Thanh ghi PIC_ENABLE được dùng để quyết định nguồn ngắt nào được phép tác động tới CPU. Tín hiệu ngắt tổng irq_o được tạo bằng phép OR của các bit đã pending và đã được enable. Đồng thời, module cũng xuất ra irq_vector_o[31:0]. Vector này giúp CPU hoặc logic xử lý ngắt xác định nhanh những nguồn ngắt đang pending và được cho phép.

Sau khi CPU xử lý xong một nguồn ngắt, firmware sẽ ghi 1 vào bit tương ứng của thanh ghi PIC_STATUS để xóa cờ pending. Thanh ghi PIC_RAW khác với PIC_STATUS ở chỗ nó không lưu trạng thái pending. Khi đọc PIC_RAW, CPU nhận được giá trị tức thời của irq_sources_i[7:0]. Thanh ghi này hữu ích trong quá trình debug hoặc khi firmware cần kiểm tra xem nguồn ngắt vật lý còn đang được kích hoạt hay không. Trong khi đó, PIC_STATUS cho biết những ngắt đã từng xảy ra và đang chờ xử lý.

3.1.8. I2C

Ngoại vi I2C được sử dụng dựa trên thiết kế OpenCores trong tài liệu [11]. Hình 3.6 là sơ đồ khối của I2C_WB trong tài liệu trên đóng vai trò là một I2C master giao tiếp với CPU thông qua bus Wishbone và giao tiếp với thiết bị ngoại vi như LCD I2C trong chương trình demo thông qua hai đường tín hiệu SCL và SDA.



Hình 3.6. Sơ đồ khối I2C [11]

Bảng 3.8: Địa chỉ các thanh ghi của I2C

Offset	Tên thanh ghi	Kích thước	Truy cập	Mô tả
0x00	PRERlo	8 bit	R/W	Byte thấp của thanh ghi prescaler. Điều chỉnh tần số SCL theo công thức $\text{prescale} = (\text{f_master} / (5 \times \text{f_SCL})) - 1$ (3.2)
0x01	PRERhi	8 bit	R/W	Byte cao của thanh ghi prescaler. Hai thanh ghi PRERlo và PRERhi tạo thành giá trị prescaler 16-bit.
0x02	CTR	8 bit	R/W	*Thanh ghi điều khiển: - Bit 7 là EN, dùng để bật I2C core. - Bit 6 là IEN, dùng để cho phép ngắt. Các bit [5:0] không sử dụng.
0x03	TXR	8 bit	W	Thanh ghi truyền: Chứa byte dữ liệu (bao gồm cả bit R/W tại vị trí LSB) gửi ra I2C.
0x03	RXR	8 bit	R	Thanh ghi nhận: CPU đọc thanh ghi này để lấy byte dữ liệu vừa nhận từ bus I2C.
0x04	CR	8 bit	W	*Thanh ghi lệnh: Ghi lệnh điều khiển I2C, sau mỗi lệnh, các bit trong CR tự động xoá - Bit 7 (STA): Phát START hoặc REPEATED-START. - Bit 6 (STO): Phát STOP. - Bit 5 (RD): Gửi lệnh đọc byte. - Bit 4 (WR): Gửi lệnh ghi byte. - Bit 3 (ACK): Khi đọc, gửi ACK (0) hoặc NACK (1) - Bit 0 (IACK): Xác nhận Interrupt, xoá cờ IF

0x04	SR	8 bit	R	*Thanh ghi trạng thái: theo dõi trạng thái. - Bit 7 (RxACK): 1 là không nhận ACK từ slave, 0 là đã nhận ACK. - Bit 6 (Busy): 1 là BUSY (START đã phát), 0 là STOP đã phát. - Bit 5 (AL): Arbitration Lost: 1 là mất ưu tiên (vd: SCL bị slave giữ), 0: bình thường - Bit 1 (TIP): Transfer In Progress: 1 là đang truyền/nhận, 0 là hoàn thành. - Bit 0 (IF): Interrupt Flag: 1 là có interrupt
------	----	-------	---	--

Bảng 3.8 liệt kê các thanh ghi điều khiển chính của module I2C_WB, bao gồm địa chỉ offset, kích thước, chế độ tương tác (đọc/ghi), và chức năng từng thanh ghi, hỗ trợ cấu hình và quản lý giao tiếp I2C. Trước khi sử dụng I2C, firmware cần ghi giá trị prescaler vào PRERlo và PRERhi, sau đó bật bit EN trong thanh ghi CTR. Nếu cần sử dụng ngắt, bit IEN cũng được bật để cho phép module phát tín hiệu irq_o khi cờ IF được set.

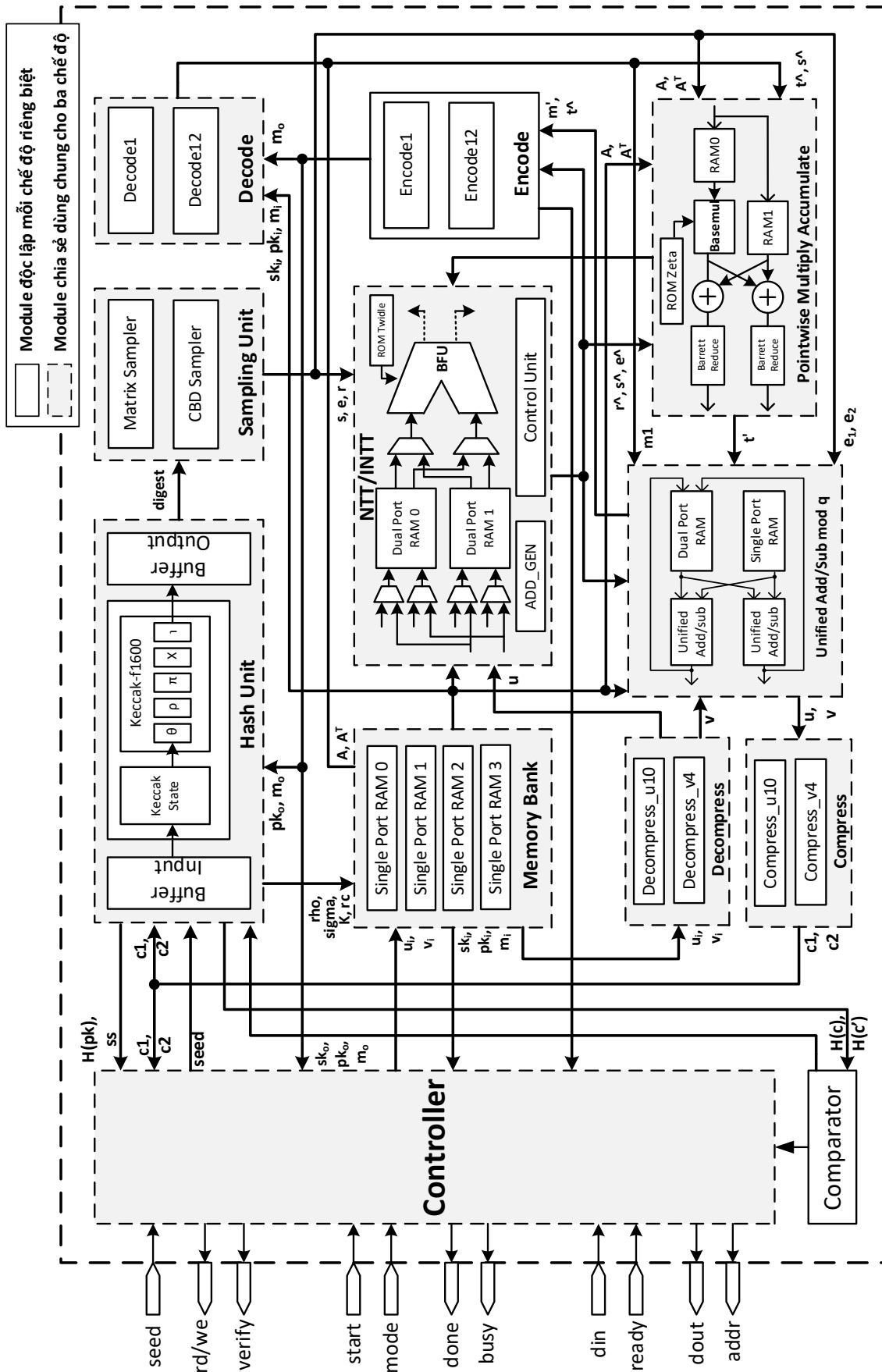
Để ghi dữ liệu đến thiết bị slave, CPU ghi địa chỉ slave kèm bit ghi vào TXR, sau đó ghi CR với lệnh START và WRITE. Sau khi bit TIP trong SR về 0, CPU tiếp tục ghi địa chỉ thanh ghi hoặc dữ liệu cần truyền vào TXR và phát lệnh WRITE. Với byte cuối cùng, CPU phát thêm lệnh STOP để kết thúc giao dịch [11].

Để đọc dữ liệu từ thiết bị slave, CPU thường thực hiện một pha ghi địa chỉ thanh ghi trước, sau đó phát repeated START với địa chỉ slave kèm bit đọc. Tiếp theo, CPU phát lệnh READ. Với các byte chưa phải cuối cùng, master gửi ACK; với byte cuối cùng, master gửi NACK và STOP để kết thúc quá trình đọc [11].

Trong SoC, I2C không tham gia trực tiếp vào quá trình tính toán mật mã, mà được dùng như một ngoại vi hệ thống phục vụ giao tiếp với thiết bị ngoài. Cụ thể, firmware có thể sử dụng I2C để điều khiển LCD, hiển thị trạng thái hệ thống hoặc thông tin demo trên board FPGA.

3.2. KYBER IP ACCELERATOR

3.2.1. Kiến trúc tổng quan của phần cứng CRYSTALS-Kyber

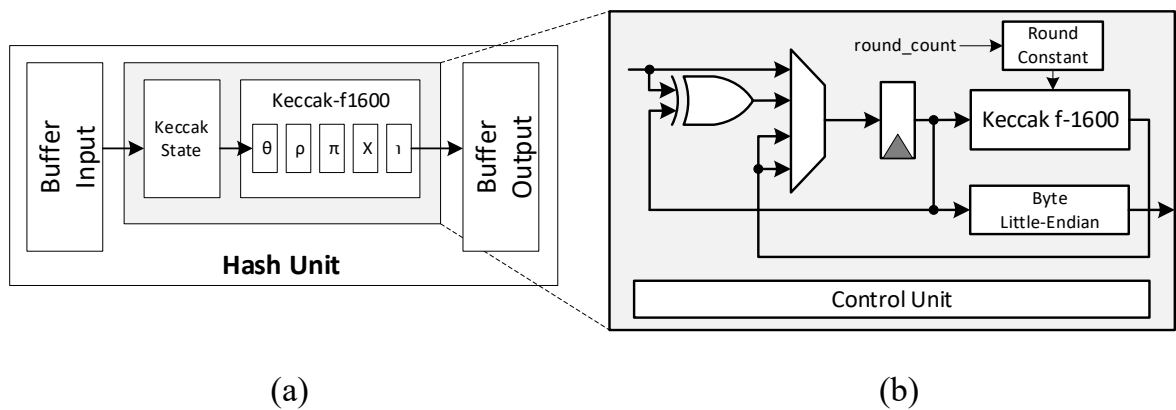


Hình 3.7. Kiến trúc tổng quan của phần cứng CRYSTALS-Kyber

Hình 3.7 minh hoạ kiến trúc tổng quan của phần cứng CRYSTALS-Kyber, trong đó Controller sẽ đảm nhận việc điều phối các đường dữ liệu theo ba chế độ của thuật toán. Để phần cứng Kyber IP hỗ trợ cả ba chế độ sinh khoá, đóng gói khoá, giải gói khoá với diện tích nhỏ và hiệu suất cao, có hai tối ưu mà khoá luận này đã thực hiện. Đầu tiên, các khối chức năng con được sử dụng trong cả ba chế độ để nhằm chia sẻ tài nguyên tối ưu diện tích. Các khối này bao gồm Decode, Sampling Unit, Hash Unit, Compress, NTT/INTT, Unified Add/Sub mod q , Pointwise Multiply Accumulate được điều khiển bởi một Controller để phù hợp với trình tự tính toán của từng chế độ hoạt động. Cụ thể hơn, Controller lựa chọn đường dữ liệu phù hợp tùy theo trạng thái thực thi của thuật toán, từ đó tái sử dụng nhiều module phần cứng cho cả ba chế độ. Thứ hai, thiết kế tách rõ các khối tính toán có độ trễ lớn và bổ sung các thanh ghi trung gian tại các đường dữ liệu tổ hợp dài để cải thiện khả năng đáp ứng timing, đồng thời hạn chế fanout của các bus lớn để đạt được tần số hoạt động tốt hơn khi hiện thực trên FPGA.

3.2.2. Thiết kế Hash Core dựa trên SHA3

Thuật toán CRYSTALS-Kyber sử dụng nhiều hàm băm và hàm mở rộng đầu vào trong quá trình tạo khóa, mã hóa và giải mã. Các hàm này được dùng để sinh ma trận công khai (A), sinh các đa thức nhiễu, băm khóa công khai và tạo các giá trị trung gian phục vụ quá trình tạo khóa chia sẻ. Vì vậy, Hash-Core là một trong những khối quan trọng trong toàn bộ Kyber IP, đặc biệt đối với các khối Sampling Unit như Rejection Uniform Sampler và Centered Binomial Distribution Sampler.



Hình 3.8: a) Kiến trúc phần cứng Hash Unit b) Phần cứng Keccak

Trong thiết kế Kyber, Hash-Unit được xây dựng dựa trên thuật toán Keccak [12] và dùng chung một lõi hoán vị Keccak f-1600 cho nhiều chức năng hash khác nhau. Lõi Keccak có trạng thái nội bộ 1600-bit, bao gồm 25 lane 64-bit được tham khảo từ [12] và tinh chỉnh để phù hợp với yêu cầu của thiết kế. Mỗi lần hoán vị Keccak thực hiện 24 vòng, với các bước biến đổi θ (theta), ρ (rho), π (pi), χ (chi) và ι (iota). Phần cứng Hash-Unit được mô tả khái quát trong Hình 3.8.

Trong Hash Unit, lõi Keccak được triển khai tuần tự phục vụ cho các chức năng SHA-3 khác nhau. Lõi Keccak này chiếm khoảng 37 % tổng số LUT trong thiết kế Kyber. Đây là một con số khá lớn, vì vậy việc tối ưu hoá sử dụng tài nguyên phần cứng cho lõi Keccak mà vẫn giữ thông lượng cao vẫn là một thách thức lớn.

Lõi Keccak tham khảo từ [12], Hình 3.2(a) đã xây dựng một thiết kế phần cứng dùng để thực hiện các hàm băm SHAKE-128, SHA-256, SHA3-256 và SHA3-512. Khối này đảm nhận việc thêm padding và xác định chiều dài dữ liệu đầu vào ứng với từng chế độ hoạt động. Trong đó sử dụng một Buffer 1344-bit nhận dữ liệu đầu vào stream 64-bit để padding hoặc thêm các bit 0 để đủ 1344-bit, vì Keccak nhận đầu vào tối đa 1344-bit. Sau đó Buffer Input đưa dữ liệu vào phần cứng Keccak để hoán vị 24 vòng, mỗi vòng thực hiện trong một chu kỳ. Vì để duy trì tốc độ cao nên lõi Keccak tồn tại trạng thái 1600-bit, đây là lý do khiến nó là phần cứng khó tối ưu diện tích nhất trong thiết kế. Cuối cùng lõi Keccak đưa 1344-bit đầu ra vào Buffer Output và tiếp tục stream từng khối 64-bit một ra ngoài làm input cho phần cứng lấy mẫu.

Trong thiết kế này, các chế độ băm được lựa chọn thông qua cổng chọn hash cmd với độ rộng 3-bit [2:0]. Bảng 3.9 dưới đây trình bày chi tiết các chế độ, chức năng và mô tả các giai đoạn sử dụng Hash Unit trong thiết kế.

Bảng 3.9: Các chế độ hoạt động của phần cứng Hash Unit

Chế độ	Chức năng	Giai đoạn sử dụng và mô tả
XOF_RQ	SHAKE128 ứng với chức năng XOF	- Trong giai đoạn tạo khóa Hình 2.5, đường XOF được gọi, sinh dữ liệu cho Uniform Sampler, làm đầu vào cho chức năng Parse để tạo ma trận A.

		- Trong giai đoạn mã hóa ở Hình 2.7, đường này tiếp tục được dùng để sinh ma trận A^T cũng qua chức năng Parse. - Ở chế độ này, đầu vào của đường XOF gồm rho 32 byte và hai chỉ số ma trận. Dữ liệu đầu ra được xuất theo từng word 64-bit cho khối lấy mẫu đồng nhất.
CMD_PRF, eta_3 = 1	SHAKE256 ứng với chức năng PRF cho $CBD_{\eta 1}$	- Trong giai đoạn tạo khóa Hình 2.5, chế độ này dùng sigma và nonce để tạo dòng bit đầu vào cho CBD Sampler, sinh các đa thức bí mật và nhiều như s và e. - Trong giai đoạn mã hóa hoặc tái mã hóa, chế độ này dùng r_seed và nonce để sinh đa thức r. - Ở chế độ này sẽ cố định một lần băm ban đầu và một lần băm mở rộng để tạo đủ đầu vào cho chức năng $CBD_{\eta 1}$. Độ dài đầu vào của chức năng $CBD_{\eta 1}$ được trình bày như Hình 3.4.
CMD_PRF, eta_3 = 0	SHAKE256 ứng với chức năng PRF cho $CBD_{\eta 2}$	- Trong giai đoạn mã hóa ở Hình 2.7, chế độ này PRF được gọi để tạo đầu vào cho chức năng $CBD_{\eta 2}$ dùng để sinh các đa thức nhiều e1 và e2. - Ở chế độ này sẽ cố định một lần băm ban đầu để tạo đủ đầu vào cho chức năng $CBD_{\eta 2}$. Độ dài đầu vào của chức năng $CBD_{\eta 2}$ được trình bày như Hình 3.4.
CMD_H, đầu vào	SHA3-256 ứng với chức năng H trong tạo và đóng gói khoá	- Trong giai đoạn tạo và đóng gói khoá Hình 2.4 và 2.6, chế độ này được gọi để tính $H(pk)$ với pk ở đây là public-key đầu vào hoặc nhận từ sender ở đóng gói khoá. pk có độ dài 800-byte hay 6144-bit. - Kết quả đầu ra là digest 256-bit, được lưu lại để phục vụ quá trình đóng và mở gói khoá.

CMD_G, đầu vào 33 byte	SHA3-512 ứng với chức năng G trong tạo khóa	- Trong giai đoạn tạo khoá Hình 2.5, chế độ này được gọi thực hiện chức năng $G(d)$ với d dài 32 byte và tham số K dài 1 byte được random ngẫu nhiên. - Kết quả với 512-bit được tách thành hai giá trị 256-bit, tương ứng rho dùng cho sinh ma trận A và sigma làm seed cho PRF sinh đa thức s và e.
CMD_G, đầu vào 64 byte	SHA3-512 ứng với chức năng G trong đóng gói khóa	- Trong giai đoạn đóng gói khoá Hình 2.6, chế độ này được gọi để thực hiện chức năng $G(m H(pk))$ nhận message m 32 byte ghép với $H(pk)$ 32 byte. - Kết quả đầu ra 512-bit được tách thành $\bar{K}$ dài 32 byte dùng làm shared secret trong đường hợp lệ của Decaps và r_seed 32 byte dùng để sinh r, $e1$ và $e2$. - Trong giai đoạn giải gói khoá Hình 2.9, sau khi khôi phục lại message, chế độ này cũng được dùng để tái tạo r_seed phục vụ quá trình tái mã hóa và so sánh ciphertext.
CMD_ RKPRF	SHAKE256 ứng với chức năng RKPRF	- Trong giai đoạn giải gói khoá Hình 2.8, nếu ciphertext tái tạo khác với ciphertext đầu vào, chế độ này được gọi để sinh khóa thay thế. - Đầu vào gồm giá trị bí mật z 32 byte ghép với ciphertext 768 byte. Kết quả đầu ra 256-bit được dùng làm shared secret trong trường hợp ciphertext không hợp lệ.

Thiết kế phần cứng này giúp hệ thống có thể chuyển đổi linh hoạt giữa nhiều chế độ băm khác nhau, tùy theo yêu cầu của từng bước trong quá trình thực hiện thuật toán. Nhờ khả năng dùng chung một kiến trúc cho nhiều hàm băm, thiết kế có thể giảm bớt phần cứng dư thừa, đồng thời vẫn đảm bảo hiệu năng xử lý phù hợp cho toàn hệ thống.

3.2.3. Thiết kế phần cứng lấy mẫu đồng nhất

Trong hệ thống mật mã Kyber, ma trận công khai A và ma trận chuyển vị A^T được tạo bằng phương pháp lấy mẫu đồng nhất, còn được gọi là Rejection Sampling hay Parse. Khối này nhận chuỗi giả ngẫu nhiên từ SHAKE128, tách chuỗi thành các ứng viên 12 bit và chỉ giữ lại các giá trị nhỏ hơn $q = 3329$. Nhờ đó, các hệ số thu được phân bố đồng đều trong miền $\mathbb{Z}_q$.

Đối với Kyber-512, ma trận có kích thước 2×2 , gồm bốn đa thức và mỗi đa thức chứa 256 hệ số. Đầu vào của SHAKE128 được tạo từ seed công khai ρ ghép với chỉ số hàng i và chỉ số cột j . Khi tạo A , thứ tự đầu vào là $\rho || j || i$; khi tạo A^T , hai chỉ số được hoán đổi thành $\rho || i || j$. Tín hiệu transposed cho phép phần cứng lựa chọn hai cách sắp xếp này mà không cần triển khai thêm một bộ lấy mẫu riêng.

Hình 3.9 mô tả kiến trúc phần cứng lấy mẫu đồng nhất với đầu vào là kết quả của Hash-Unit, cung cấp dữ liệu XOF theo từng từ 64 bit thông qua giao thức valid/ready. Mỗi khối rate của SHAKE128 gồm 168 byte, tương ứng 21 từ 64 bit. Khi các hệ số hợp lệ chưa đủ 256 phần tử, Matrix Sampler tiếp tục yêu cầu Hash-Core thực hiện hoán vị Keccak và sinh thêm dữ liệu. Vì SHAKE là hàm có đầu ra mở rộng, quá trình này có thể tiếp tục cho đến khi hoàn thành đa thức.

Dữ liệu XOF được đưa vào một buffer-1344bit vì dữ liệu được đưa vào liên tục mà phần cứng chỉ xử lý 24-bit mỗi lần lấy mẫu. Hai ứng viên được lấy đồng thời từ `sample_buf[11:0]` và `sample_buf[23:12]`. Mỗi ứng viên được đưa đến một bộ so sánh độc lập để kiểm tra điều kiện:

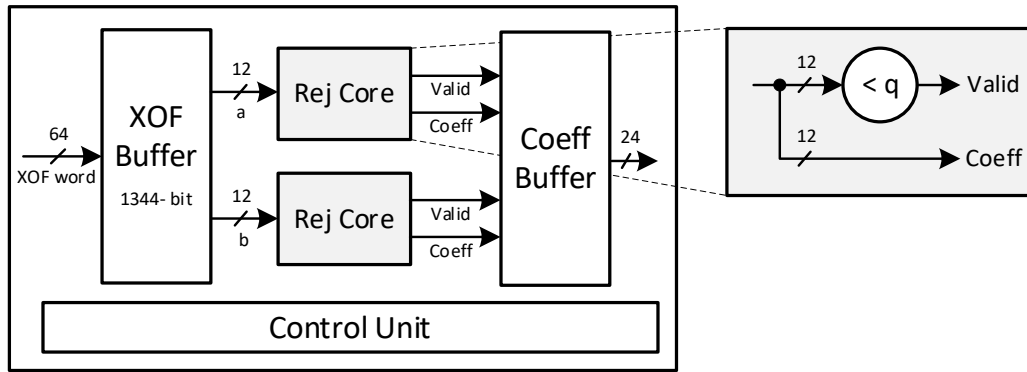
$$c_0 < q, \quad c_1 < q, \quad q = 3329.$$

Nếu cả hai ứng viên đều hợp lệ, khối lấy mẫu xuất hai hệ số trong cùng một chu kỳ thông qua hai cổng ghi. Nếu chỉ một ứng viên hợp lệ, hệ số đó được đưa ra ở cổng thứ nhất. Các giá trị không thỏa mãn điều kiện được loại bỏ và không làm tăng bộ đếm hệ số. Sau đó Coeff Buffer sẽ lưu các giá trị thỏa mãn điều kiện và đưa ra khi đủ hai giá trị 12-bit. Ở đây chọn hai giá trị vì ở nghiên cứu [7] cho thấy khi xử lý nhiều hơn hai giá trị sẽ rất phức tạp, do đó chọn xử lý với hai giá trị mỗi lần lấy mẫu. Trong luồng sinh khóa, các hệ số của ma trận A có thể được chuyển trực tiếp sang PWMA

để thực hiện phép nhân với vector bí mật. Đồng thời, các hệ số này được lưu vào những vùng còn trống của Memory Bank để tái sử dụng, tránh phải gọi lại SHAKE128.

Kiến trúc phần cứng lấy mẫu đồng nhất gồm các thành phần chính sau:

- **XOF Request Controller:** tạo đầu vào ρ , i , j và điều khiển SHAKE128.
- **XOF Buffer:** lưu tối đa 128 bit dữ liệu XOF và quản lý số bit hợp lệ.
- **Hai Rejection Core:** so sánh đồng thời hai ứng viên 12 bit với $q = 3329$.
- **Coeff Buffer:** đầu ra lưu các giá trị thoả mãn điều kiện, đưa ra khi đủ 24-bit.



Hình 3.9. Kiến trúc tổng quan phần cứng lấy mẫu đồng nhất

3.2.4. Thiết kế phần cứng lấy mẫu phân phối nhị thức trung tâm

Hình 3.10 mô tả đường dẫn dữ liệu của phần cứng lấy mẫu phân phối nhị thức trung tâm trong bộ tăng tốc mật mã Kyber. Phần cứng này được sử dụng để tạo các đa thức có hệ số nhỏ phục vụ quá trình sinh khóa, đóng gói khóa và mở gói khóa trong CRYSTALS-Kyber. Dữ liệu đầu vào của khối được tạo bởi hàm PRF dựa trên SHAKE256. Mỗi hệ số đầu ra được tính bằng hiệu giữa tổng của hai nhóm bit, nhờ đó tạo ra phân phối đối xứng quanh giá trị 0.

Phần cứng hỗ trợ hai tham số η (eta) là 2 và 3. Mỗi chế độ có độ dài đầu vào khác nhau, mà kết quả đầu vào lấy từ hàm băm SHAKE256 hay Pseudo-Random Function (PRF) trong quá trình lấy mẫu nhị thức trung tâm. Cụ thể:

- Với $\eta = 2$, dữ liệu đầu vào dài 128 byte, tương ứng 1024 bit.
- Với $\eta = 3$, dữ liệu đầu vào dài 192 byte, tương ứng 1536 bit.

Khác với kiến trúc chờ toàn bộ dữ liệu PRF được tạo xong, CBD Sampler hiện tại được kết nối trực tiếp với Hash-Core bằng giao diện streaming 64 bit qua Buffer Input. Khi Hash-Unit phát một từ dữ liệu hợp lệ, CBD Sampler có thể tiếp nhận và xử lý ngay. Do phần cứng chỉ xử lý 12-bit hoặc 8-bit mỗi lần lấy mẫu, Buffer này sẽ lấy ra 12-bit hoặc 8-bit mỗi lần, tùy thuộc chế độ đang tính toán:

- Với $\eta = 3$, sử dụng 12-bit.
- Với $\eta = 2$, sử dụng 8-bit và chèn thêm 4-bit 0.

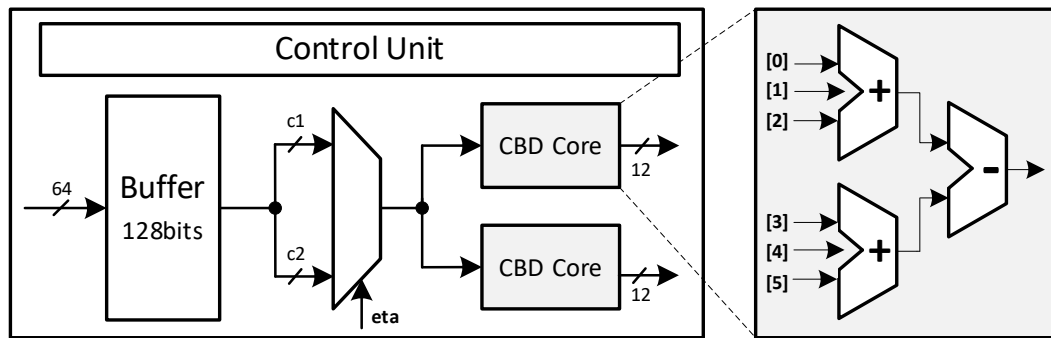
Việc chèn thêm các bit 0 được mô tả cụ thể như sau:

- Khi $\eta = 3$: `buffer_dout[11:0]`.
- Khi $\eta = 2$: `buffer_dout[11:10]`, `1'b0`, `buffer_dout[9:8]`, `1'b0`, `buffer_dout[7:6]`, `1'b0`, `buffer_dout[5:4]`, `1'b0`.

Cách chèn bit 0 cho phép hai chế độ $\eta = 2$ và $\eta = 3$ sử dụng chung hai bộ cộng ba đầu vào và hai bộ trừ như mô tả Hình 3.10. Vì vậy, phần cứng không cần triển khai riêng hai datapath CBD, giúp tiết kiệm và tái sử dụng phần cứng.

Trong quá trình sinh khóa, hệ số nhiều s và e được tạo với $\eta_1 = 3$. Kết quả CBD được ghi trực tiếp vào hai cổng của RAM đầu vào NTT để chuẩn bị cho biến đổi tiếp theo. Trong quá trình đóng gói khóa, vector r sử dụng $\eta_1 = 3$, còn các đa thức e_1 và e_2 sử dụng $\eta_2 = 2$. Các hệ số này được chuyển trực tiếp vào Memory Bank.

Ngoài ra, phần cứng này trong hệ thống được thiết kế để tạo mẫu nhị thức hiệu quả với việc sử dụng hai CBD-Core để tăng hiệu suất. Thiết kế nhỏ gọn giúp tiết kiệm diện tích, hỗ trợ đa chế độ cho xử lý đầu vào có độ dài khác nhau.



Hình 3.10. Kiến trúc tổng quan phần cứng lấy mẫu phân phối nhị thức trung tâm

3.2.5. Thiết kế phần cứng NTT/INTT

Number Theoretic Transform (NTT) là một biến thể của Discrete Fourier Transform (DFT), trong đó các phép tính được thực hiện trên trường hữu hạn thay vì trên số phức. NTT chuyển đa thức từ miền hệ số sang miền biến đổi, cho phép thay phép nhân đa thức có độ phức tạp $O(n^2)$ giảm xuống bằng phép nhân điểm có độ phức tạp $(O(n \log n))$. Trong CRYSTALS-Kyber, các đa thức được xác định trên vành: $R_q = \mathbb{Z}_q[X]/(X^{256} + 1)$ với $q = 3329$. Nên tồn tại căn nguyên thủy bậc 256 trong $\mathbb{Z}_q$, thường chọn $\zeta = 17$. Tuy nhiên, không tồn tại căn nguyên thủy bậc 512, vì vậy $X^{256} + 1$ không thể phân tích hoàn toàn thành các nhân tử bậc nhất. Kyber sử dụng NTT không đầy đủ, phân tích đa thức thành 128 nhân tử bậc hai:

$$X^{256} + 1 = \prod_{i=0}^{127} (X^2 - \zeta^{2i+1}) = \prod_{i=0}^{127} (X^2 - \zeta^{2br_7(i)+1}) \quad (3.3)$$

Trong đó $br_7(i)$ là phép đảo thứ tự bảy bit của i . Vì vậy, NTT trong Kyber chỉ thực hiện bảy tầng biến đổi thay vì tám tầng đầy đủ. Sau NTT, 256 hệ số được tổ chức thành 128 cặp và được xử lý bởi khối Pointwise Base-Multiply.

Giải thuật của các phép biến đổi NTT và biến đổi ngược INTT được minh họa trong giải thuật 7 và giải thuật 8 dưới đây.

Thuật toán 7 Biến đổi Number Theoretic Transform

Đầu vào: Mảng hệ số của đa thức $a(x) \in R_q$, ký hiệu là A , với $q = 3329$

Đầu ra: $\hat{A} \leftarrow \text{NTT}(A)$ trong $\mathbb{Z}_q$

```

1:  $k \leftarrow 1$ 
2: for  $len \leftarrow 128$ ;  $len \geq 2$ ;  $len \leftarrow len/2$  do
3:   for  $start \leftarrow 0$ ;  $start < 256$ ;  $start \leftarrow start + 2 \cdot len$  do
4:      $z \leftarrow \zeta^{br_7(k)}$ 
5:      $k \leftarrow k + 1$ 
6:     for  $j \leftarrow start$ ;  $j < start + len$ ;  $j \leftarrow j + 1$  do
7:        $t \leftarrow \text{MontgomeryReduce}(z \cdot A[j + len])$ 
8:        $A[j + len] \leftarrow (A[j] - t) \bmod q$ 
9:        $A[j] \leftarrow (A[j] + t) \bmod q$ 
10:    end for
11:  end for
12: end for
13: Return  $\hat{A} \leftarrow \text{NTT}(A)$ 

```

Thuật toán 7 trình bày thuật toán NTT dùng để chuyển đa thức từ miền hệ số sang miền NTT. Đầu vào của thuật toán là mảng hệ số A của đa thức $a(x) \in R_q$, cùng với căn nguyên thủy bậc 256 của đơn vị ζ . Thuật toán thực hiện theo nhiều lớp, với giá trị len giảm dần từ 128 xuống 2, mà mỗi lớp, các cặp hệ số được xử lý bằng phép butterfly, trong đó hệ số phía sau được nhân với hệ số xoay ζ , sau đó cộng và trừ modulo q . Sau vòng lặp, mảng A được biến đổi sang dạng $NTT(A)$ trong $\mathbb{Z}_q$.

Thuật toán 8 Inverse Number Theoretic Transform

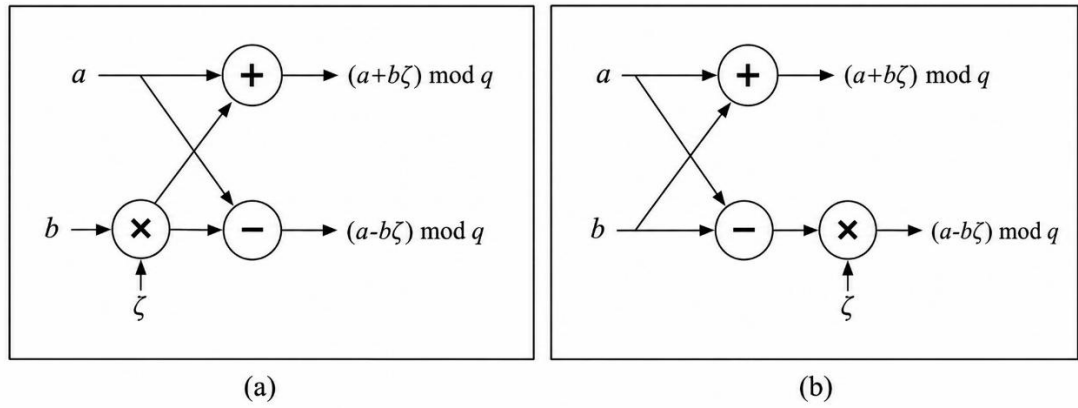
Đầu vào: Mảng hệ số NTT của đa thức $\hat{A}$, với $\hat{A} \in R_q$
Đầu ra: $A \leftarrow INTT(\hat{A})$ trong $\mathbb{Z}_q$

```

1:  $k \leftarrow 127$ 
2: for  $len \leftarrow 2$ ;  $len \leq 128$ ;  $len \leftarrow 2 \cdot len$  do
3:   for  $start \leftarrow 0$ ;  $start < 256$ ;  $start \leftarrow start + 2 \cdot len$  do
4:      $z \leftarrow \zeta^{br_7(k)}$ 
5:      $k \leftarrow k - 1$ 
6:     for  $j \leftarrow start$ ;  $j < start + len$ ;  $j \leftarrow j + 1$  do
7:        $t \leftarrow \hat{A}[j]$ 
8:        $\hat{A}[j] \leftarrow \text{BarrettReduce}(t + \hat{A}[j + len])$ 
9:        $\hat{A}[j + len] \leftarrow \text{MontgomeryReduce}(z \cdot (\hat{A}[j + len] - t))$ 
10:    end for
11:  end for
12: end for
13: for  $j \leftarrow 0$ ;  $j < 256$ ;  $j \leftarrow j + 1$  do
14:    $\hat{A}[j] \leftarrow \text{MontgomeryReduce}(1441 \cdot \hat{A}[j])$ 
15: end for
16: Return  $A \leftarrow INTT(\hat{A})$ 
```

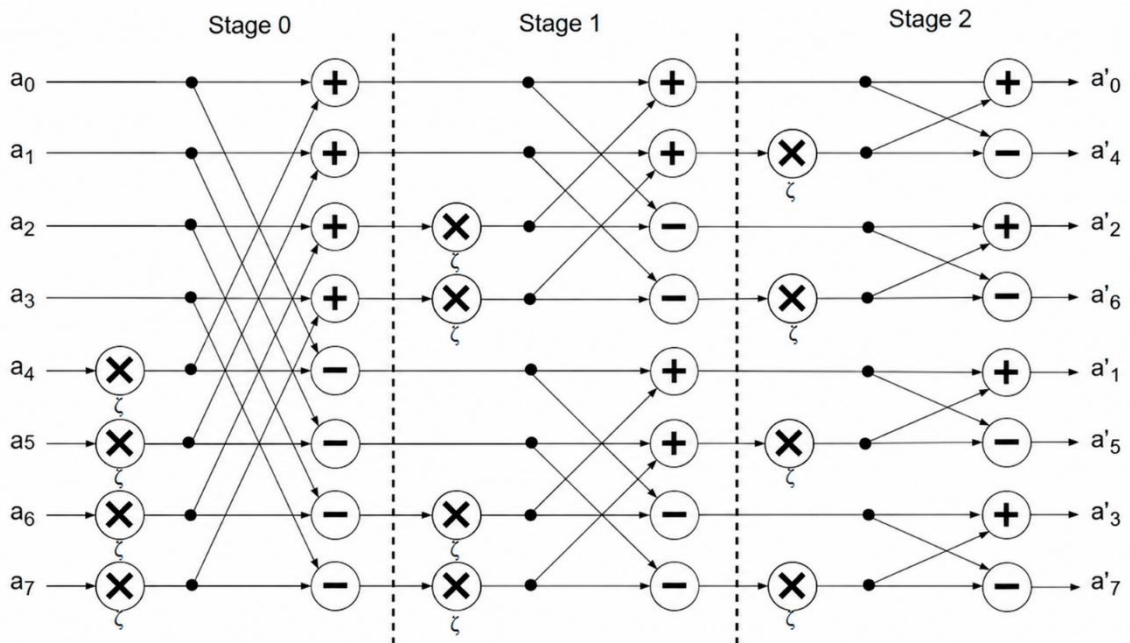
Thuật toán 8 mô tả thuật toán INTT hay phép biến đổi ngược của NTT. Thuật toán này nhận đầu vào là mảng hệ số đang ở miền NTT và đưa nó trở lại miền hệ số ban đầu. Khác với NTT, quá trình INTT thực hiện theo chiều ngược lại, với len tăng dần từ 2 đến 128, mà mỗi bước, các cặp hệ số cũng được xử lý theo dạng butterfly ngược, kết hợp phép cộng, phép trừ và nhân với hệ số ζ theo modulo q . Kết quả cuối cùng là mảng A sau khi được biến đổi ngược, tương ứng với $INTT(\hat{A})$ trong $\mathbb{Z}_q$.

Trong Kyber, phép biến đổi NTT thuận sử dụng phương pháp Butterfly Cooley-Tukey (CT), còn INTT sử dụng Butterfly Gentleman-Sande (GS) được thể hiện trong Hình 3.11. Hai thuật toán này đều chia quá trình biến đổi thành nhiều tầng, trong đó mỗi tầng đều lặp lại phép toán butterfly, nghĩa là xử lý n hệ số theo từng cặp tuần tự.



Hình 3.11. (a) Butterfly Cooley-Tukey, (b) Butterfly Gentleman-Sande

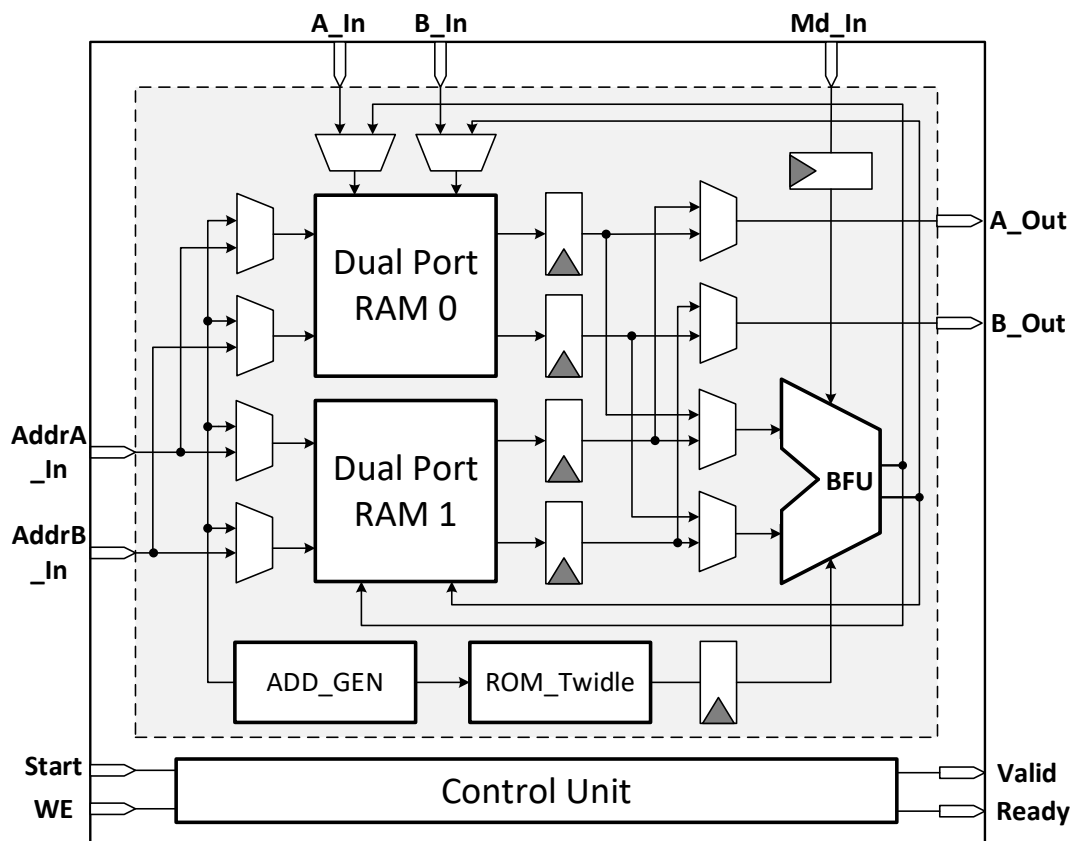
Cụ thể, trong Kyber ta có $n = 256$, nên xét theo cách NTT thông thường thì quá trình biến đổi sẽ gồm $\log_2 256 = 8$ tầng. Tuy vậy, với modulo $q = 3329$, trường này chỉ có căn nguyên thủy bậc 256 mà không có căn nguyên thủy bậc 512. Vì lý do đó, NTT trong Kyber không thể tách đa thức hoàn toàn thành các nhân tử bậc 1, mà chỉ dừng lại ở các nhân tử bậc 2. Do đó, phép NTT của Kyber trên thực tế chỉ thực hiện 7 tầng biến đổi, thường được gọi là NTT không đầy đủ, hay incomplete NTT. Mô hình NTT được rút gọn với trường hợp $n = 8$ được minh họa trong Hình 3.12.



Hình 3.12. Mô hình NTT với $n = 8$.

Kiến trúc của khối NTT được minh họa trong Hình 3.13. Thiết kế này gồm một số module chính như hai bộ nhớ RAM Dual Port 0 và 1, ROM Twiddle dùng để lưu

hệ số xoay, khối Address Generator để tạo địa chỉ đọc/ghi, khối Butterfly Unit để thực hiện phép tính butterfly và khối Controller để điều khiển toàn bộ quá trình. Kiến trúc này được thiết kế để thực hiện bảy tầng biến đổi NTT theo dạng butterfly. Các tầng từ 0 đến 6 được xử lý lần lượt bằng cách tái sử dụng cùng một khối Butterfly Unit, nhờ đó giảm số lượng phần cứng cần dùng và tối ưu tài nguyên triển khai.



Hình 3.13. Kiến trúc tổng quan phần cứng NTT/INTT

Bộ nhớ Dual Port RAM 0 và RAM 1

Trong khối NTT, dữ liệu trung gian không được đọc ghi trực tiếp liên tục từ bộ nhớ hệ thống, mà được lưu tạm trong hai bộ nhớ cục bộ là RAM 0 và RAM 1. Mỗi bộ nhớ có tổ chức 256 từ, mỗi từ rộng 16 bit, phù hợp với 256 hệ số của một đa thức Kyber. Hai RAM này hoạt động theo cơ chế luân phiên để phục vụ các tầng biến đổi butterfly. Ở một tầng, một RAM đóng vai trò là bộ nhớ nguồn cung cấp hai hệ số đầu vào cho Butterfly Unit, trong khi RAM còn lại nhận kết quả tính toán. Sang tầng kế tiếp, vai trò của hai RAM được hoán đổi. Cách tổ chức ping-pong này giúp lưu lại kết quả trung gian sau từng tầng NTT/INTT mà không cần đưa dữ liệu quay lại bộ nhớ ngoài. Nhờ vậy, quá trình xử lý các tầng biến đổi được thực hiện liên tục hơn,

giảm số lần truy cập bộ nhớ hệ thống và hạn chế độ trễ do truyền dữ liệu. Ở tầng đầu tiên, dữ liệu đầu vào được nạp vào RAM nội bộ trước khi đưa đến Butterfly Unit. Từ các tầng sau, đầu vào của Butterfly Unit chính là các kết quả trung gian đã được ghi lại ở tầng trước đó.

Address Generator

Address Generator có nhiệm vụ tạo địa chỉ truy cập bộ nhớ và địa chỉ hằng số twiddle trong suốt quá trình thực hiện NTT/INTT. Với mỗi tầng biến đổi, các hệ số của đa thức được xử lý theo từng cặp butterfly. Do đó, khối sinh địa chỉ phải tạo chính xác địa chỉ của hai hệ số cần đọc, địa chỉ ghi kết quả và địa chỉ truy xuất hệ số ζ (zeta) trong ROM Twiddle. Trong thiết kế này, Address Generator hoạt động dựa trên thông tin về tầng hiện tại, bộ đếm butterfly và chế độ biến đổi. Khi thực hiện NTT, thứ tự địa chỉ được tạo theo luồng Cooley-Tukey. Khi thực hiện INTT, thứ tự truy xuất được điều chỉnh để phù hợp với luồng Gentleman-Sande. Việc tách riêng khối sinh địa chỉ giúp bộ điều khiển chính đơn giản hơn, đồng thời đảm bảo việc đọc dữ liệu, ghi dữ liệu và lấy twiddle được đồng bộ với pipeline tính toán.

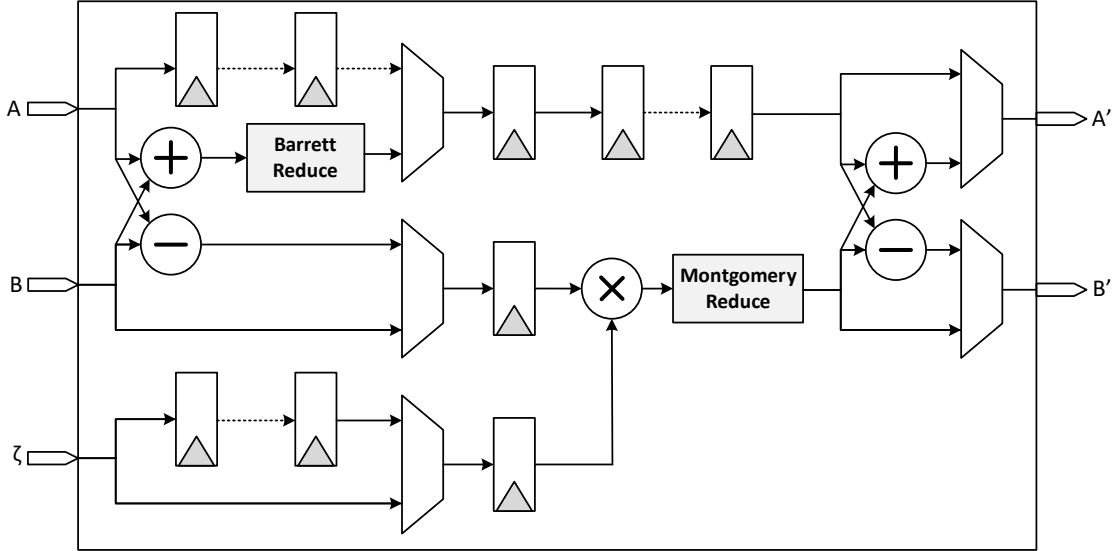
ROM Twiddle

ROM Twiddle lưu các hằng số căn nguyên ζ (zeta) cần thiết cho các phép butterfly trong NTT và INTT. Các giá trị này được chuẩn bị trước và được truy cập bằng địa chỉ do Address Generator tạo ra. Ở mỗi phép butterfly, một giá trị twiddle tương ứng được đọc từ ROM và đưa vào Butterfly Unit để thực hiện phép nhân modulo. Việc lưu twiddle trong ROM giúp giảm logic tính toán hằng số tại thời gian chạy, đồng thời đảm bảo các giá trị sử dụng trong từng tầng biến đổi là cố định và nhất quán với thuật toán Kyber.

Butterfly Unit

Butterfly Unit là khối tính toán trung tâm trong module NTT/INTT. Khối này nhận vào hai hệ số và một hệ số twiddle ζ , sau đó thực hiện các phép toán cộng, trừ và nhân theo modulo q . Tùy vào chế độ được chọn, Butterfly Unit có thể xử lý theo dạng Cooley-Tukey khi chạy NTT hoặc theo dạng Gentleman-Sande khi chạy INTT. Cụ thể, khối này có ba đầu vào chính gồm hai hệ số cần tính toán và hằng số căn

nguyên ζ tương ứng với từng tầng butterfly. Bên cạnh đó, Butterfly Unit còn nhận thêm tín hiệu điều khiển mode để xác định đang thực hiện phép biến đổi NTT hay INTT. Kiến trúc tổng quát của Butterfly Unit được trình bày trong Hình 3.14.



Hình 3.14. Kiến trúc phần cứng Pipelined Butterfly Unit

Quá trình hoạt động BFU hiện các phép tính toán cộng, trừ và nhân modulo q . Và ta có kết quả đầu ra lần lượt là:

- NTT: $A' = (A + B \cdot \zeta) \bmod q$ và $B' = (A - B \cdot \zeta) \bmod q$ (3.4)

- INTT: $A' = (A + B) \bmod q$ và $B' = (A - B) \cdot \zeta \bmod q$ (3.5)

Để rút gọn các kết quả về miền modulo ($q = 3329$), thiết kế sử dụng hai kỹ thuật giảm modulo là Montgomery Reduction và Barrett Reduction, được trình bày trong giải thuật 9 và 10 và kiến trúc tương ứng ở Hình 3.15 và 3.16.

Thuật toán 9 Montgomery Reduction

Đầu vào: Số nguyên có dấu 32 bit a

Đầu ra: Số nguyên có dấu 16 bit t

- 1: $u \leftarrow (a \times QINV)$
 - 2: $t \leftarrow u \times KYBER_Q$
 - 3: $t \leftarrow a - t$
 - 4: $t \leftarrow t \ggg 16$
 - 5: **Return** t
-

Thuật toán Montgomery Reduction ở trên dùng để rút gọn một giá trị 32 bit a về dạng 16 bit trong miền modulo q . Đầu tiên, thuật toán tính giá trị trung gian u

bằng cách nhân a với hằng số $QINV$. Hằng số này được chọn trước sao cho hỗ trợ việc khử phần dư theo modulo q . Sau đó, u được nhân tiếp với $KYBER_Q$ để tạo ra một giá trị gần với a theo modulo. Thuật toán lấy a trừ đi giá trị này, rồi dịch phải 16 bit để thu được kết quả cuối cùng. Nhờ cách làm này, phép chia modulo phức tạp được thay bằng các phép nhân, trừ và dịch bit, nên phù hợp hơn khi triển khai phần cứng.

Thuật toán 10 Barrett Reduction

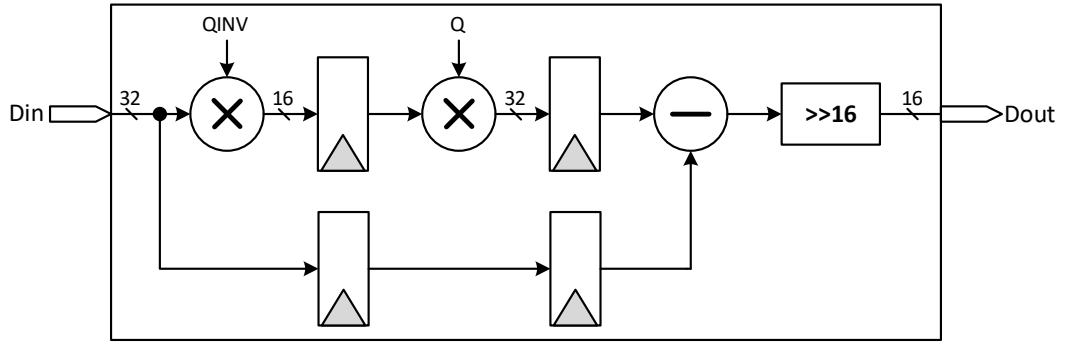
Đầu vào: Số nguyên có dấu 16 bit a

Đầu ra: Số nguyên có dấu 16 bit t

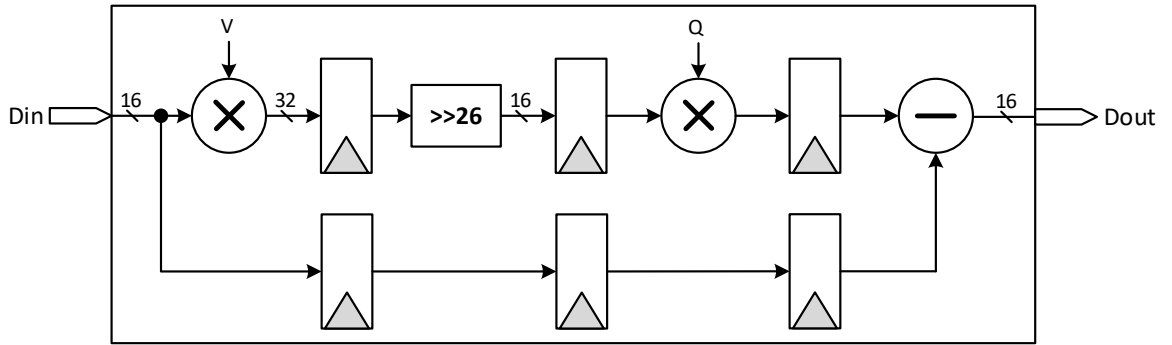
- 1: **const** $v \leftarrow (1 \ll 26 + KYBER_Q/2)/KYBER_Q$
 - 2: $t \leftarrow (v \cdot a) \gg 26$
 - 3: $t \leftarrow t \cdot KYBER_Q$
 - 4: **Return** $a - t$
-

Thuật toán Barrett Reduction ở trên cũng dùng để đưa một giá trị a về miền modulo q , nhưng cách làm là ước lượng trước thương của phép chia cho q . Đầu tiên, thuật toán tính một hằng số v , đây là giá trị xấp xỉ của $\frac{2^{26}}{q}$. Sau đó, v được nhân với a và dịch phải 26 bit để ước lượng xem a chứa khoảng bao nhiêu lần q . Giá trị ước lượng này tiếp tục được nhân với $KYBER_Q$, rồi lấy a trừ đi kết quả đó để thu được phần dư gần đúng.

Tóm lại, Montgomery Reduction thường dùng sau phép nhân lớn, còn Barrett Reduction thường dùng để rút gọn nhanh các giá trị sau cộng/trừ. Cả hai đều tránh dùng phép chia trực tiếp cho q , nên giúp khối NTT/INTT chạy nhanh và dễ thiết kế phần cứng hơn. Các phép nhân với hằng số căn nguyên ζ , chẳng hạn như trong biểu thức $(A - B) \cdot \zeta \bmod q$, tạo ra kết quả có độ rộng lớn, đòi hỏi kỹ thuật Montgomery Reduction, thường được dùng sau các phép nhân với twiddle, vì kết quả phép nhân có độ rộng lớn hơn và cần được đưa về miền modulo một cách hiệu quả. Trong khi đó, Barrett Reduction, một phương pháp giảm modulo dựa trên xấp xỉ chia, được dùng để chuẩn hóa các kết quả sau cộng hoặc trừ, do các giá trị này thường chỉ lệch khỏi miền modulo trong phạm vi nhỏ hơn. Hình 3.19 và 3.20 trình bày phần cứng Montgomery Reduce và Barrett Reduce.



Hình 3.15. Kiến trúc khối Montgomery Reduce



Hình 3.16. Kiến trúc khối Barrett Reduce

3.2.6. Thiết kế phần cứng Pointwise Multiply Accumulate

Trong CRYSTALS-Kyber, các phép toán chính được thực hiện trên đa thức có hệ số thuộc trường hữu hạn Z_q . Các dữ liệu như khóa công khai, khóa bí mật và bản mã đều được biểu diễn dưới dạng vector đa thức. Trong đó, phép nhân đa thức là phép toán tốn nhiều thời gian nếu thực hiện trực tiếp, vì độ phức tạp có thể đạt $O(n^2)$.

Giả sử có hai đa thức:

$$A(x) = a_0 + a_1x + a_2x^2 + \dots + a_{n-1}x^{n-1}$$

$$B(x) = b_0 + b_1x + b_2x^2 + \dots + b_{n-1}x^{n-1}$$

Khi nhân trực tiếp, mỗi hệ số của đa thức kết quả được tính theo dạng tích chập:

$$c_k = \sum_{i+j=k} a_i b_j \quad (3.6)$$

Cách tính này yêu cầu nhiều phép nhân và phép cộng, do đó không phù hợp với các thiết kế cần hiệu năng cao. Vì vậy, Kyber sử dụng biến đổi NTT để đưa đa thức từ miền hệ số sang miền NTT:

$$\hat{a} = NTT(a), \hat{b} = NTT(b) \quad (3.7)$$

Trong miền NTT, phép nhân đa thức được thay bằng phép nhân từng điểm:

$$c = INTT(\hat{a} \circ \hat{b}) = INTT(NTT(a) \circ NTT(b)) \quad (3.8)$$

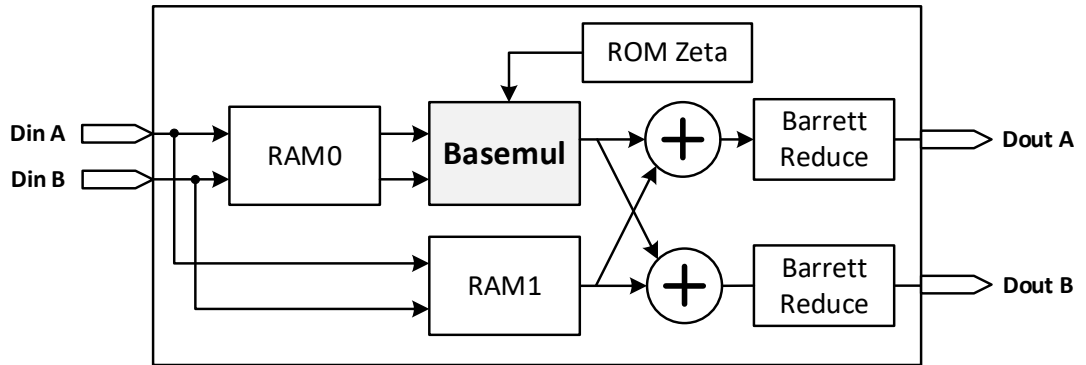
Nhờ đó, phép nhân đa thức được thực hiện hiệu quả hơn so với phép nhân trực tiếp. Trong Kyber, với $n = 256$, mỗi đa thức sau biến đổi NTT được biểu diễn dưới dạng 128 cặp hệ số. Các cặp này tương ứng với các đa thức bậc 1 có dạng: $\hat{a}_{2i} + \hat{a}_{2i+1}X, i = 0, \dots, 127$.

Khi nhân hai đa thức trong miền NTT, từng cặp hệ số được nhân theo modulo $X^2 - \zeta^{2br7(i)+1}$. Kết quả của mỗi cặp được tính như sau:

$$\hat{c}_{2i} = \hat{a}_{2i}\hat{b}_{2i} + \hat{a}_{2i+1}\hat{b}_{2i+1}\zeta^{2br7(i)+1} \quad (3.9)$$

$$\hat{c}_{2i+1} = \hat{a}_{2i}\hat{b}_{2i+1} + \hat{a}_{2i+1}\hat{b}_{2i} \quad (3.10)$$

Trong đó, $br7(i)$ là giá trị đảo bit 7-bit được sử dụng trong thứ tự truy xuất của thuật toán NTT. Với các phép nhân vector đa thức trong quá trình tạo khóa hoặc mã hóa, từng cặp đa thức trong miền NTT sẽ được nhân và cộng dồn để tạo ra kết quả cuối cùng, sau đó được rút gọn modulo q .



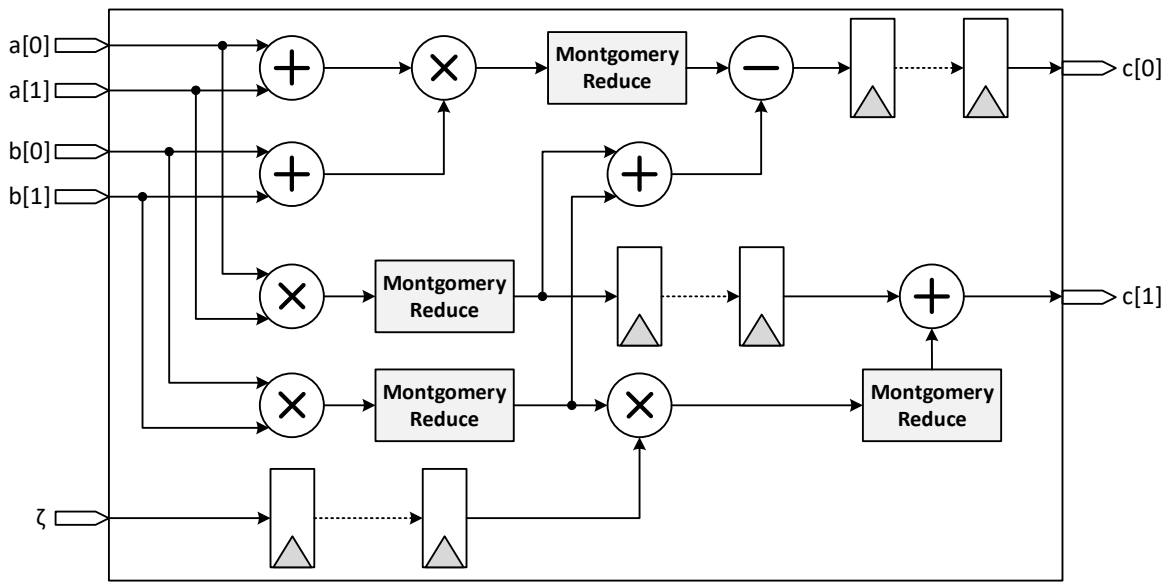
Hình 3.17. Kiến trúc tổng quan phần cứng Pointwise Multiply Accumulate

Mô hình tổng quan của mô-đun Pointwise Multiply Accumulate (PWMA) được trình bày trong Hình 3.17. Chức năng chính của khối này là thực hiện phép nhân từng điểm giữa các cặp hệ số đa thức trong miền NTT, sau đó cộng dồn các kết quả trung gian để tạo ra một đa thức kết quả duy nhất. Đây là phép toán quan trọng trong các bước như tính $\hat{A} \circ \hat{s}$, $\hat{A}^T \circ \hat{r}$ hoặc $\hat{s}^T \circ \hat{u}$ trong Kyber.

Trong quá trình hoạt động, các hệ số tương ứng của hai vector đa thức đầu vào a và b được nạp tuần tự vào bộ nhớ nội bộ. Ở mỗi chu kỳ xử lý, bộ nhớ cung cấp cho

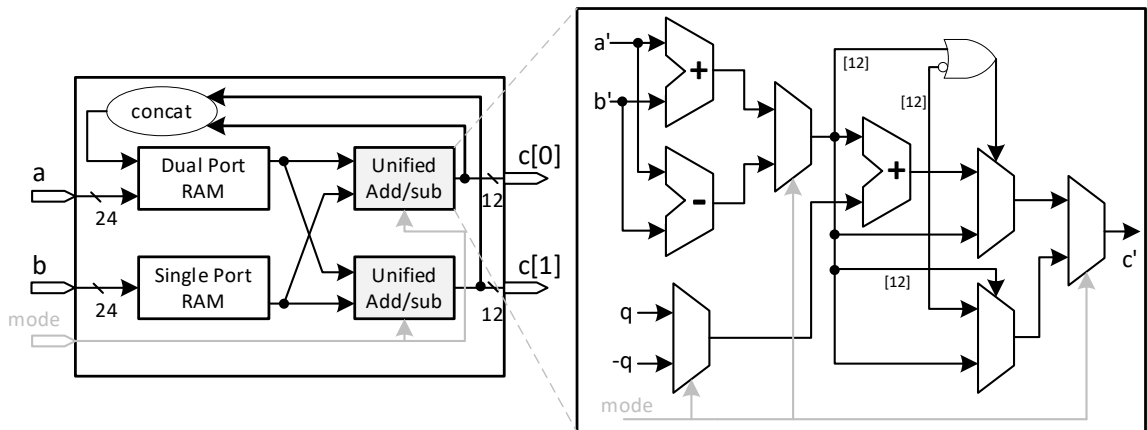
khối Basemul hai cặp hệ số trong miền NTT, ví dụ $(\hat{a}_{2i}, \hat{a}_{2i+1})$ và $(\hat{b}_{2i}, \hat{b}_{2i+1})$. Đồng thời, hằng số $\zeta^{2br7(i)+1}$ được đọc từ ROM Zeta để phục vụ phép nhân theo cấu trúc của Kyber trong miền NTT.

Basemul là khối tính toán chính của mô-đun PWMA, được minh họa trong Hình 3.18. Khối này thực hiện phép nhân giữa từng cặp hệ số tương ứng của hai đa thức đã được biến đổi sang miền NTT. Các phép nhân trong Basemul là phép nhân modulo q , do đó kỹ thuật Montgomery Reduction được sử dụng để rút gọn kết quả sau nhân và đảm bảo giá trị đầu ra nằm trong miền hợp lệ của Z_q .



Hình 3.18. Kiến trúc khối Basemul

3.2.7. Thiết kế phần cứng Unified Add/Sub mod q



Hình 3.19. Kiến trúc phần cứng Unified Add/Sub mod q

Hình 3.19 trình bày kiến trúc phần cứng của khối Unified Add/Sub mod q , dùng để thực hiện các phép cộng trên vector đa thức như $\hat{A} \circ \hat{s} + \hat{e}$, cũng như các phép cộng dồn đa thức như $NTT^{-1}(\hat{t}^T \circ \hat{r}) + e_2 + Decompress_q(Decode_1(m), 1)$. Sau khi cộng hai đa thức, kết quả sẽ được lưu vào Simple Dual Port RAM để tiếp tục dùng cho các phép cộng với đa thức khác. Ngoài phép cộng, khối này cũng hỗ trợ phép trừ đa thức, ví dụ như $v - NTT^{-1}(\hat{s}^T \circ NTT(u))$.

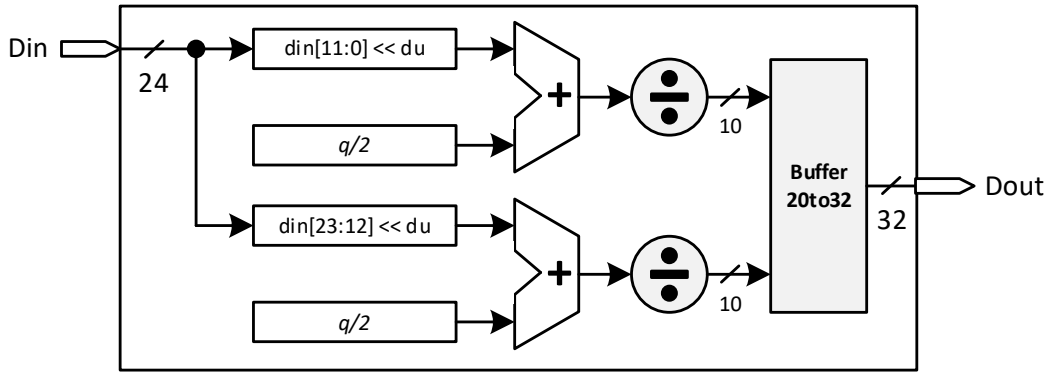
Việc thiết kế Unified Add/Sub mod q Core theo hướng dùng chung cho cả cộng và trừ giúp phần cứng linh hoạt hơn trong quá trình xử lý. Kiến trúc ở Hình 3.17 cho phép tái sử dụng các thành phần lưu trữ, đặc biệt là RAM, khi các phép toán không cần dùng bộ nhớ cùng lúc. Điều này khá quan trọng trong triển khai CRYSTALS-Kyber, vì thuật toán phải xử lý nhiều phép toán trên đa thức, nên nếu tận dụng lại RAM hợp lý thì có thể giảm bớt tài nguyên phần cứng.

Các kết quả đầu ra của khối đều được đưa về modulo q để phù hợp với yêu cầu tính toán trong Kyber. Nhờ dùng Unified Add/Sub mod q Core, cả phép cộng và phép trừ modulo đều có thể được xử lý trong cùng một kiến trúc, thay vì phải tách thành nhiều khối riêng. Cách làm này giúp giảm phần cứng dư thừa và vẫn đáp ứng được các bước tính toán modulo cần thiết trong thuật toán.

3.2.8. Thiết kế phần cứng khối Encode, Decode, Compress, Decompress

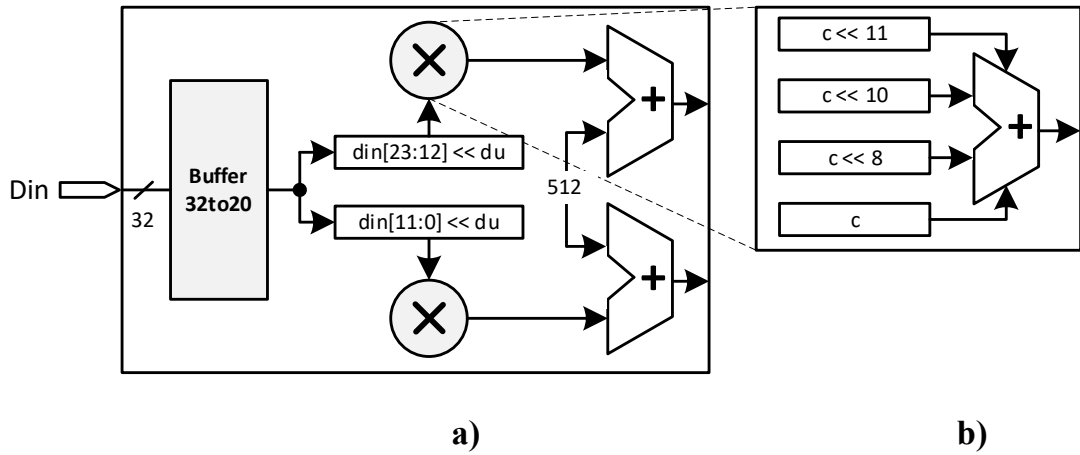
Hình 3.20 trình bày kiến trúc phần cứng của khối $Compress_{d_u}$ với $d_u = 10$, được dùng để nén vector đa thức u trong quá trình tạo bản mã. Cụ thể, quá trình này tương ứng với biểu thức $c_1 = Encode_{d_u}(Compress_{d_u}(u, d_u))$. Trong Kyber, ngoài việc nén vector u , còn có quá trình nén đa thức v để tạo ra thành phần bản mã thứ hai $c_2 = Encode_{d_v}(Compress_{d_v}(v, d_v))$. Kiến trúc phần cứng của khối $Compress_{d_v}$ cũng được tổ chức tương tự như $Compress_{d_u}$. Điểm khác nhau chủ yếu chính nằm ở số bit sau nén, trong đó u dùng giá trị $d_u = 10$, còn v dùng giá trị $d_v = 4$.

Khối Divider trong Hình 3.20 dùng để phục vụ quá trình nén. Thay vì chia trực tiếp cho q , thiết kế dùng cách xấp xỉ theo Barrett Reduction để giảm tài nguyên phần cứng. Nhờ đó, phép chia được thay bằng các phép cộng, trừ và dịch bit, giúp rút ngắn đường dữ liệu và hỗ trợ tính toán liên tục hơn.



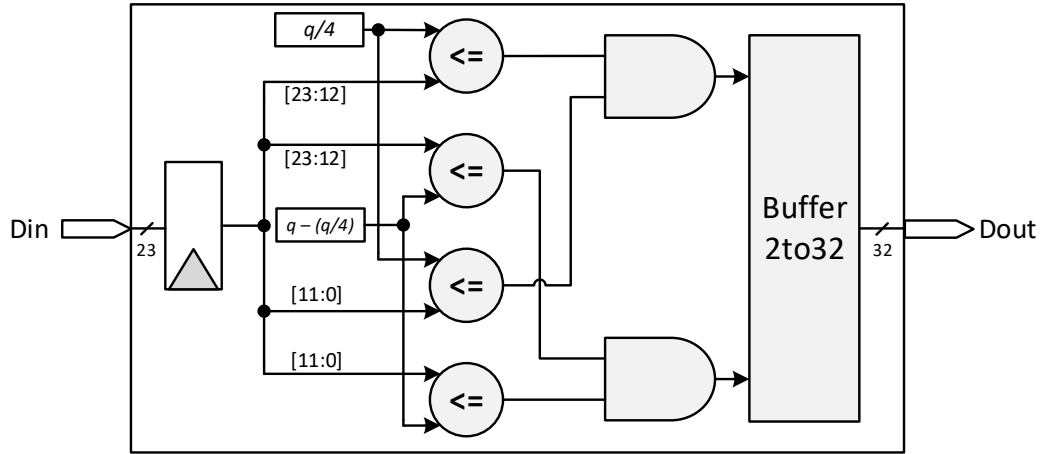
Hình 3.20. Kiến trúc phần cứng $Compress_{du}$

Đối với quá trình giải nén, Hình 3.21 mô tả kiến trúc phần cứng của khối $Decompress_{d_u}$, được dùng để khôi phục xấp xỉ vector đa thức u từ dữ liệu đã được decode, $u = Decompress_q(Decode_{d_u}(c), d_u)$. Về mặt thuật toán, quá trình giải nén có chứa phép nhân với q . Tuy nhiên, để tối ưu tài nguyên phần cứng, phép nhân này được biến đổi thành tổ hợp các phép dịch bit và phép cộng, như minh họa trong Hình 3.21. Cách làm này giúp tránh sử dụng bộ nhân lớn, đồng thời vẫn tạo ra kết quả phù hợp với yêu cầu của phép giải nén trong Kyber.

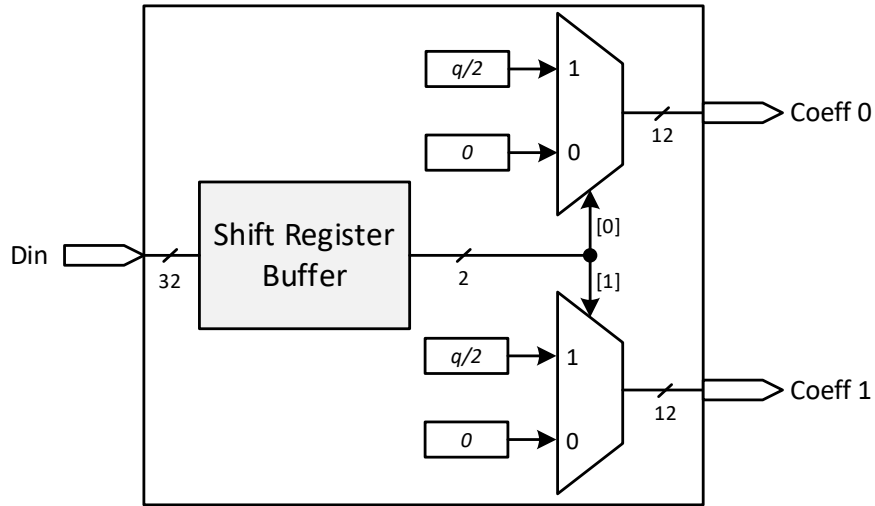


Hình 3.21. Kiến trúc phần cứng a) $Decompress_{d_u}$ b) Multiplier by Q

Tương tự, quá trình giải nén đa thức v được thực hiện bởi khối $Decompress_{d_v}$: $v = Decompress_q(Decode_{d_v}(c), d_v)$. Khối này có nguyên lý hoạt động giống với $Decompress_{d_u}$, nhưng sử dụng tham số $d_v = 4$ thay vì $d_u = 10$. Vì vậy, số bit dịch và độ rộng dữ liệu sau giải nén sẽ được điều chỉnh tương ứng với từng loại dữ liệu bản mã.



Hình 3.22. Kiến trúc phần cứng $Encode_1$

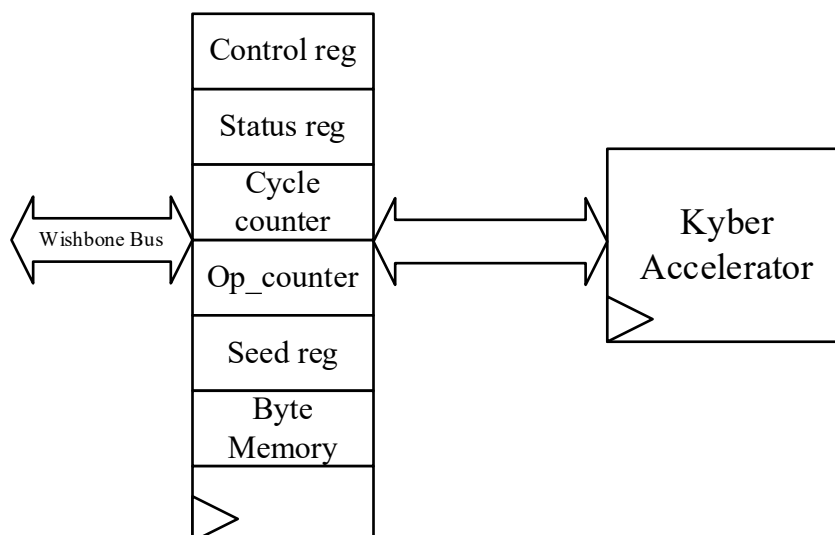


Hình 3.23. Kiến trúc phần cứng $Decode_1$

Hình 3.22 và Hình 3.23 lần lượt trình bày kiến trúc phần cứng của các khối $Encode_1$ và $Decode_1$. Hai khối này được sử dụng để mã hóa và giải mã thông điệp m trong quá trình encapsulation và decapsulation của Kyber. Trong đó, $Encode_1$ chuyển các bit thông điệp thành dạng đa thức phục vụ phép tính trong quá trình mã hóa, còn $Decode_1$ khôi phục lại thông điệp từ kết quả giải mã.

3.2.9. Giao diện Wishbone Bus

Thiết kế phần cứng Kyber được tích hợp vào SoC dưới dạng một ngoại vi ánh xạ bộ nhớ, cho phép lõi xử lý RISC-V điều khiển quá trình tạo khóa, mã hóa và giải mã thông qua giao diện Wishbone Bus. Thông qua cơ chế này, CPU có thể ghi lệnh điều khiển, nạp seed đầu vào, theo dõi trạng thái thực thi và đọc/ghi các vùng dữ liệu như khóa công khai, khóa bí mật, bản mã và shared secret.



Hình 3.24. Sơ đồ Wishbone Bus và Kyber Accelerator

Hình 3.24 trình bày sơ đồ khối giao tiếp giữa Wishbone Bus và Kyber accelerator. Bên trái là cụm thanh ghi điều khiển và trạng thái, bao gồm thanh ghi CONTROL, STATUS, bộ đếm chu kỳ, bộ đếm số phép toán và các thanh ghi seed.

Bảng 3.10: Địa chỉ các thanh ghi của Kyber

Tên thanh ghi	Offset	Truy cập	Chức năng
Control_reg	0x00	R/W	Thanh ghi điều khiển: Bit start dùng để kích hoạt phép toán. Trường opcode chọn chế độ hoạt động: 0 là KeyGen, 1 là Encaps, 2 là Decaps.
Status_reg	0x08	R	Thanh ghi trạng thái: Chứa bit báo done, busy, trạng thái truy cập bộ nhớ ngoài.
Cycle_counter	0x10	R	Bộ đếm số chu kỳ thực thi của phép toán Kyber hiện tại.
Op_counter	0x18	R	Bộ đếm số phép toán hoặc số lần thực thi, dùng để hỗ trợ đo đạc thời gian.
Seed_0 => 7	0x40 => 0x78	W	Tám thanh ghi seed, mỗi thanh ghi 64 bit, tổng cộng 512 bit seed đầu vào cho khối Kyber.

Bảng 3.10 mô tả chức năng của các thanh ghi cho Kyber slave. Vùng byte memory của Kyber bắt đầu tại offset 0x0001_0000 so với không gian địa chỉ nội bộ của Kyber IP. Do đó, khi CPU muốn truy cập dữ liệu như public key, secret key, ciphertext hoặc shared secret, địa chỉ truy cập sẽ được tính bằng địa chỉ nền của Kyber cộng với offset vùng byte memory và offset dữ liệu tương ứng.

Về thao tác điều khiển, trước khi bắt đầu một phép toán, CPU ghi seed cần thiết vào các thanh ghi SEED_0 đến SEED_7 hoặc ghi dữ liệu đầu vào vào vùng byte memory. Sau đó, CPU ghi trường opcode trong thanh ghi CONTROL để chọn chế độ thực thi. Khi bit start được đặt lên 1, Kyber accelerator bắt đầu hoạt động. Trong quá trình xử lý, bit busy trong thanh ghi STATUS được đặt lên 1. Khi phép toán hoàn tất, bit done được đặt lên 1 và CPU có thể đọc kết quả từ vùng dữ liệu tương ứng.

Đối với chế độ KeyGen, CPU nạp seed tạo khóa vào các thanh ghi seed, sau đó ghi opcode = 0 và kích hoạt start. Sau khi hoàn tất, accelerator ghi public key vào vùng PK và secret key vào vùng SK. Đối với chế độ Encaps, CPU cung cấp public key và seed mã hóa, sau đó ghi opcode = 1. Kết quả của quá trình encapsulation là ciphertext trong vùng CT và shared secret trong vùng SS. Đối với chế độ Decaps, CPU cung cấp secret key và ciphertext, sau đó ghi opcode = 2. Khi hoàn tất, shared secret được ghi vào vùng SS.

Tổng thể, việc ánh xạ Kyber thành ngoại vi MMIO giúp IP có thể tích hợp trực tiếp vào vi điều khiển, đồng thời thuận tiện cho firmware điều khiển, kiểm thử và đo số chu kỳ thực thi trên cả mô phỏng lẫn FPGA.

CHƯƠNG 4

KIỂM TRA VÀ ĐÁNH GIÁ KẾT QUẢ

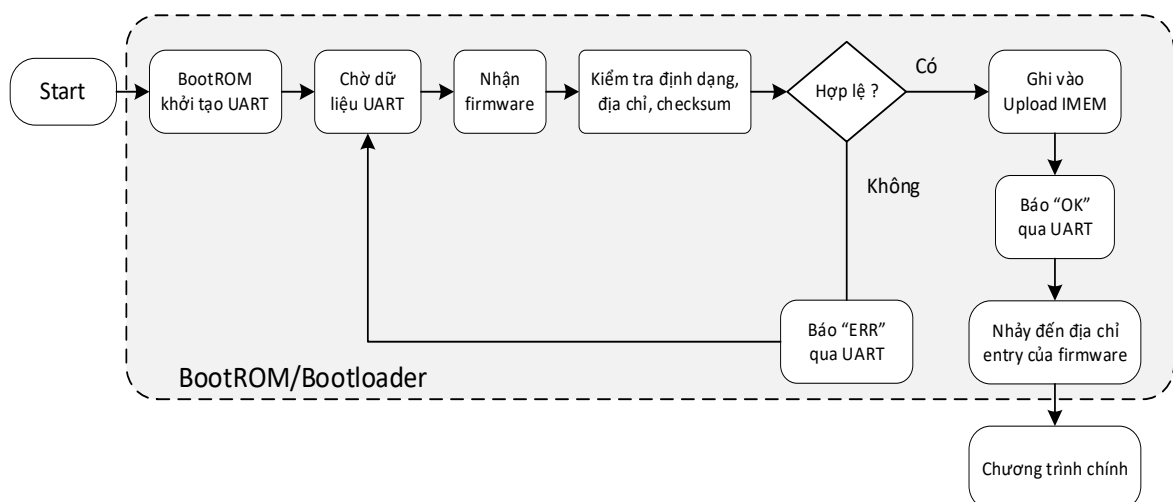
4.1. BOOTLOADER, THƯ VIỆN C VÀ CHƯƠNG TRÌNH NẠP

4.1.1. Thiết kế Bootloader

Bootloader được thiết kế để nạp firmware từ máy tính vào SoC thông qua giao tiếp UART. Sau khi reset hoặc cấu hình FPGA, CPU bắt đầu thực thi chương trình trong BootROM. BootROM khởi tạo UART, chờ dữ liệu từ chương trình nạp trên PC, sau đó nhận firmware và ghi vào vùng Upload IMEM. Firmware được truyền theo từng gói dữ liệu; bootloader kiểm tra thông tin gói, ghi payload vào bộ nhớ tương ứng và phản hồi trạng thái cho chương trình nạp.

Sau khi quá trình nạp hoàn tất, bootloader chuyển quyền điều khiển đến địa chỉ bắt đầu của firmware trong Upload IMEM. Từ thời điểm này, CPU thực thi chương trình chính và có thể điều khiển các ngoại vi như UART, GPIO, Timer, I2C, PIC và Kyber Accelerator thông qua bus Wishbone.

Hình 4.1 mô tả luồng hoạt động của bootloader: khởi tạo UART, chờ và nhận firmware, kiểm tra dữ liệu, ghi vào Upload IMEM, rồi nhảy đến chương trình chính.



Hình 4.1. Sơ đồ hoạt động khi reset hoặc bật vi điều khiển

4.1.2. Thư viện lập trình

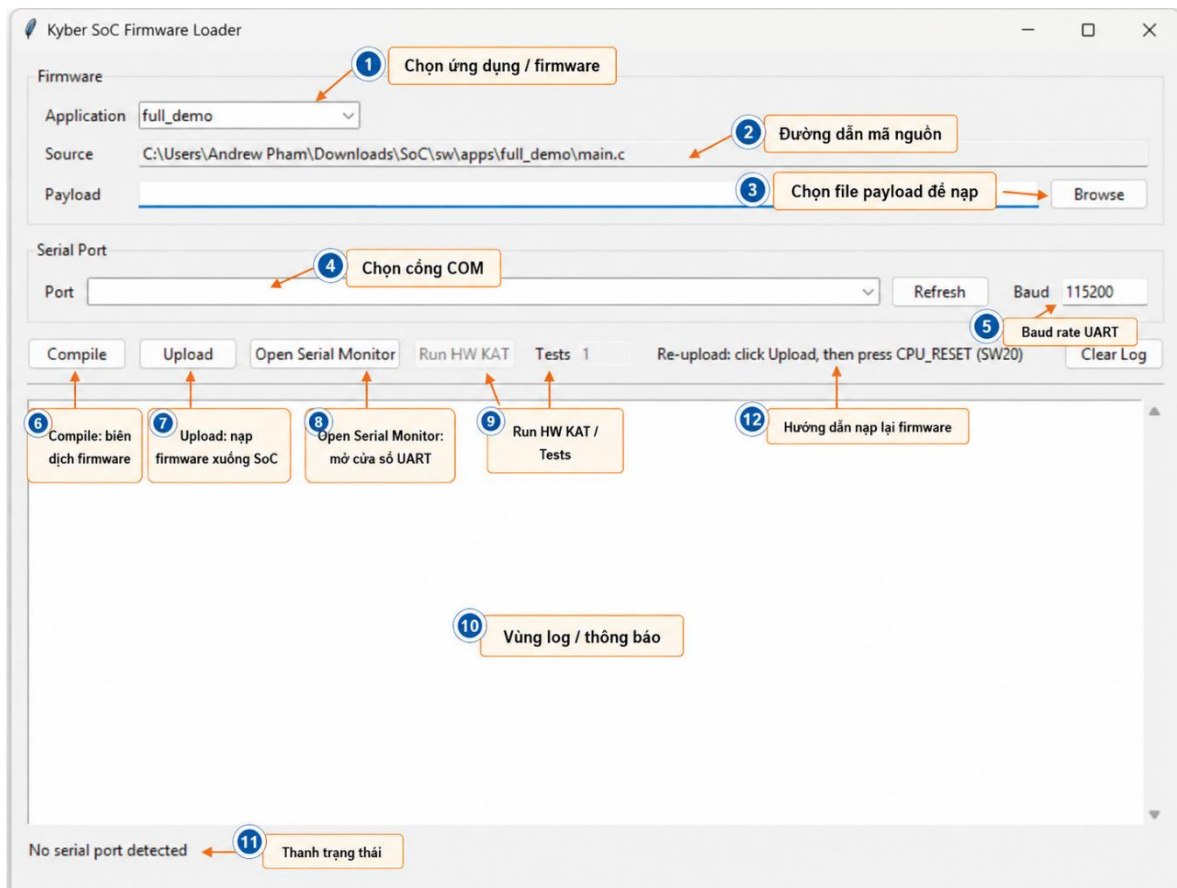
Thư viện lập trình được phát triển bằng ngôn ngữ C, sau đó được biên dịch thành tệp mã máy có định dạng .hex thông qua bộ công cụ RISC-V GNU toolchain. Tệp .hex này sẽ được nạp vào bộ nhớ chương trình để vi điều khiển có thể thực thi. Mục đích của thư viện là cung cấp sẵn các hàm hỗ trợ điều khiển ngoại vi, giúp người lập trình không cần thao tác trực tiếp quá nhiều với từng thanh ghi ở mức thấp.

Thư viện được xây dựng dựa trên memory map của Wishbone Bus, trong đó mỗi ngoại vi trong hệ thống được gán một địa chỉ cơ sở riêng như trình bày ở Bảng 3.2. Từ các địa chỉ cơ sở này, thư viện định nghĩa thêm các offset thanh ghi, các bit điều khiển và các cờ trạng thái cần thiết cho từng ngoại vi. Ví dụ, đối với UART, thư viện khai báo các thanh ghi liên quan đến truyền dữ liệu, nhận dữ liệu, kiểm tra trạng thái FIFO và cấu hình điều khiển ngắt. Nhờ vậy, người lập trình có thể gọi các hàm có sẵn trong thư viện để thực hiện truyền nhận dữ liệu hoặc cấu hình ngoại vi một cách đơn giản hơn, thay vì phải tự ghi đọc từng địa chỉ thanh ghi bằng tay.

4.1.3. Chương trình nạp firmware

Để hỗ trợ việc nạp firmware từ máy tính cá nhân xuống vi điều khiển thông qua UART, nhóm đã xây dựng một chương trình nạp riêng bằng ngôn ngữ Python. Chương trình có giao diện đồ họa đơn giản, trực quan, được phát triển dựa trên thư viện Tkinter. Công cụ này được thiết kế để làm việc tương thích với bootloader đã trình bày ở phần trên, đồng thời hỗ trợ các thao tác cần thiết trong quá trình biên dịch và truyền chương trình xuống vi điều khiển.

Cụ thể, chương trình cho phép biên dịch mã nguồn C thành tệp ELF bằng bộ công cụ RISC-V GNU toolchain. Sau đó, kết quả biên dịch tạo ra tệp thực thi firmware.elf, rồi được chuyển đổi thành các tệp payload như firmware.bin, firmware.hex để phục vụ quá trình nạp. Trong chế độ nạp, dữ liệu từ các section này được gửi xuống vi điều khiển thông qua cổng COM đã chọn. Trong quá trình truyền, chương trình cũng tính checksum và kiểm tra phản hồi OK/ERR từ bootloader để xác nhận dữ liệu đã được nhận đúng. Ngoài ra, công cụ còn tích hợp Serial Monitor, giúp hiển thị dữ liệu nhận được và gửi dữ liệu thủ công qua UART khi cần kiểm tra hoặc debug hệ thống.



Hình 4.2. Giao diện chương trình nạp firmware

Giao diện ở Hình 4.2 hỗ trợ người dùng chọn chương trình ứng dụng cần nạp, quan sát đường dẫn mã nguồn, chọn file payload, chọn cổng UART, thiết lập baudrate và thực hiện các thao tác biên dịch, nạp firmware và theo dõi Serial Monitor. Khi người dùng nhấn **Compile**, chương trình gọi script biên dịch mã nguồn C bằng RISC-V GNU toolchain. Kết quả biên dịch được tạo thành file firmware nhị phân firmware.bin. Khi nhấn **Upload**, GUI gọi script để gửi firmware xuống SoC thông qua cổng UART đã chọn. Bootloader trong BootROM nhận dữ liệu từ UART, kiểm tra định dạng, địa chỉ và checksum của payload và ghi nội dung chương trình vào Upload IMEM. Sau khi nạp thành công, bootloader chuyển quyền điều khiển đến địa chỉ bắt đầu của firmware để CPU thực thi chương trình chính.

Ngoài ra, GUI còn tích hợp **Serial Monitor** cho phép người dùng theo dõi dữ liệu UART hai chiều và quan sát trạng thái thực thi thông qua vùng log của giao diện. Bên cạnh đó, chức năng Run HW KAT (Known Answer Tests) và tùy chọn Tests (chọn số lượng tests) cho phép kích hoạt các bài kiểm tra Accelerator đã tích hợp,

phục vụ đối chiếu kết quả thực thi của bộ tăng tốc Kyber. Nhờ vậy mà việc biên dịch, nạp chương trình, theo dõi UART và kiểm chứng hoạt động của hệ thống có thể được thực hiện tập trung trên một giao diện duy nhất.

4.2. KIỂM TRA TOÀN BỘ HỆ THỐNG

Để đánh giá thiết kế trước khi triển khai trên FPGA, hệ thống được kiểm thử theo hai mức. Ở mức module, từng khối RISC-V, GPIO, Timer, UART và Kyber được mô phỏng. Ở mức tích hợp, firmware được thực thi trực tiếp trên lõi RISC-V và truy cập các ngoại vi qua bus Wishbone.

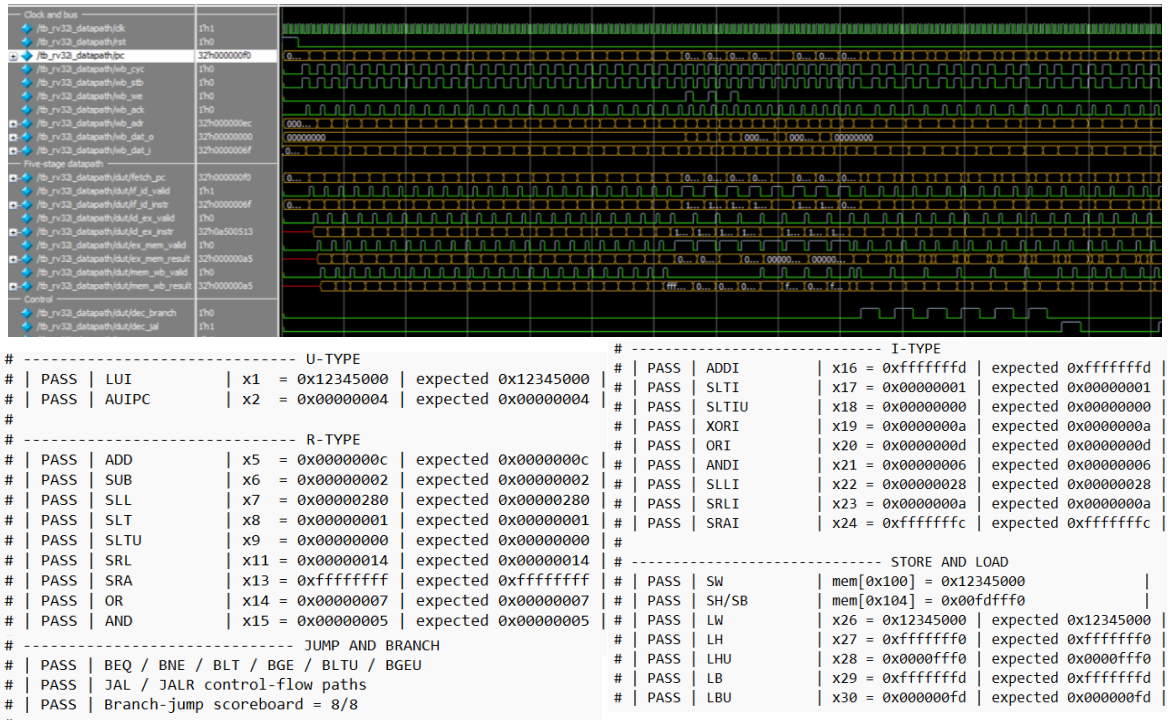
4.2.1. Kiểm tra lõi RISC-V

Để kiểm tra lõi xử lý RISC-V, nhóm đã xây dựng chương trình mẫu thực hiện các lệnh thuộc tập lệnh RV32I, sau đó đối chiếu kết quả thu được với giá trị mong đợi. Việc kiểm tra này giúp đánh giá xem lõi xử lý có thực thi đúng các nhóm lệnh cơ bản, truy cập bộ nhớ và điều khiển luồng chương trình hay không.

Testbench của lõi xử lý nạp một chương trình hợp ngữ RV32I vào bộ nhớ mô phỏng, sau đó theo dõi quá trình tìm nạp lệnh, giải mã, thực thi, giao dịch bộ nhớ và nội dung tập thanh ghi. Chương trình kiểm tra các lệnh U-type gồm LUI, AUIPC; các phép toán R-type gồm ADD, SUB, SLL, SLT, SLTU, XOR, SRL, SRA, OR, AND; và các lệnh I-type tương ứng gồm ADDI, SLTI, SLTIU, XORI, ORI, ANDI, SLLI, SRLI, SRAI.

Nhóm lệnh điều khiển được kiểm tra với BEQ, BNE, BLT, BGE, BLTU, BGEU, JAL và JALR. Mỗi nhánh đúng sẽ làm tăng một biến scoreboard, kết quả cuối cùng đạt 8/8. Nhóm truy cập bộ nhớ kiểm tra SW, SH, SB, LW, LH, LHU, LB, LBU, bao gồm cả trường hợp mở rộng dấu và mở rộng không dấu. Các giá trị ghi trả về thanh ghi và dữ liệu lưu trong bộ nhớ đều được đối chiếu với kết quả tham chiếu sau từng nhóm lệnh. Sau 146 chu kỳ, chương trình ghi giá trị kết thúc 0xA5, không phát sinh trap, và toàn bộ giá trị thanh ghi cũng như bộ nhớ đều trùng với kết quả mong đợi.

Hình 4.3 thể hiện một phần dạng sóng mô phỏng và log kết quả của chương trình kiểm tra lõi xử lý RISC-V kèm thống kê testcases ở Bảng 4.1.



Hình 4.3. Mô phỏng các lệnh LUI, R-TYPE, I-TYPE, JUMP, BRANCH, LOAD và STORE

Bảng 4.1: Tóm tắt testcase kiểm tra lỗi RISC-V RV32I

STT	Nhóm testcase	Lệnh kiểm tra	Nội dung và kết quả mong đợi	Kết quả mô phỏng
1	U-type	LUI, AUIPC	Kiểm tra nạp immediate 20 bit và tính địa chỉ tương đối theo PC.	LUI tạo giá trị 0x12345000; AUIPC cho kết quả 0x00000004.
2	R-type	ADD, SUB, SLL, SLT, SLTU, XOR, SRL, SRA, OR, AND	Kiểm tra các phép toán số học, logic, so sánh có dấu/không dấu và dịch bit giữa hai thanh ghi nguồn.	Các thanh ghi đích từ x5 đến x15 đều khớp giá trị tham chiếu
3	I-type	ADDI, SLTI, SLTIU, XORI, ORI, ANDI, SLLI, SRLI, SRAI	Kiểm tra phép toán với immediate, mở rộng dấu, so sánh và dịch bit.	Các thanh ghi đích từ x16 đến x24 đều khớp giá trị mong đợi.

4	Branch	BEQ, BNE, BLT, BGE, BLTU, BGEU	Kiểm tra điều kiện rẽ nhánh có dấu, không dấu và địa chỉ PC sau nhánh.	Branch-jump scoreboard đạt 8/8.
5	Jump	JAL, JALR	Kiểm tra chuyển hướng PC và ghi giá trị liên kết (PC+4) về thanh ghi đích.	Các lệnh nhảy thực hiện đúng luồng chương trình.
6	Store	SW, SH, SB	Kiểm tra ghi word, halfword và byte vào bộ nhớ; kiểm tra byte- enable khi ghi dữ liệu.	Dữ liệu lưu trong bộ nhớ khớp giá trị mong đợi.
7	Load	LW, LH, LHU, LB, LBU	Kiểm tra đọc word, halfword, byte; kiểm tra mở rộng dấu và mở rộng không dấu.	Giá trị tại các thanh ghi x26 đến x30 khớp giá trị tham chiếu.
8	Tổng hợp	Toàn bộ chương trình kiểm tra	Kiểm tra sự phối hợp giữa các khối fetch, decode, ALU, điều khiển nhảy, truy cập bộ nhớ và write-back.	Chương trình hoàn tất sau 146 chu kỳ, không phát sinh trap; trạng thái thanh ghi, bộ nhớ đều cho kết quả đúng.

Kết quả mô phỏng trong Hình 4.3 và Bảng 4.1 thống kê testcase và kết quả cho thấy lỗi xử lý thực thi đúng các nhóm lệnh cơ bản của RV32I, bao gồm phép toán số học, logic, so sánh, điều khiển luồng chương trình và truy cập bộ nhớ. Toàn bộ testcase đều đạt kết quả mong đợi, không phát sinh trap trong quá trình kiểm tra. Bên cạnh đó, các giá trị ghi trả về RegFile và dữ liệu đọc, ghi trên bộ nhớ đều được đối chiếu với kết quả tham chiếu. Điều này xác nhận các khối tìm nạp lệnh, giải mã, thực thi, điều khiển nhảy và giao tiếp bộ nhớ của lõi hoạt động thống nhất. Kết quả này là cơ sở để tiếp tục tích hợp lỗi xử lý với các ngoại vi trong SoC.

4.2.2. Kiểm tra GPIO

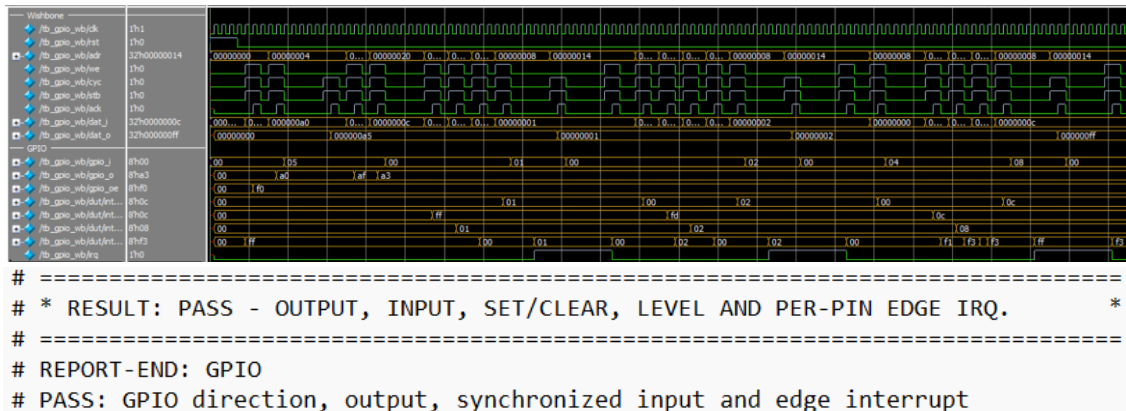
Để kiểm chứng chức năng GPIO, nhóm thực hiện mô phỏng với testbench tự động tạo các giao dịch đọc/ghi Wishbone đến những thanh ghi điều khiển của ngoại vi. Trong quá trình, tín hiệu ACK, dữ liệu đọc, trạng thái ngắt và các ngõ ra GPIO được theo dõi rồi đối chiếu với kết quả mong đợi.

Testbench lần lượt kiểm tra cấu hình hướng dữ liệu qua IO_DIR, rồi xuất và đọc dữ liệu qua IO_VAL. Cũng như chế độ hỗn hợp với bốn bit cao là ngõ ra và bốn bit thấp là ngõ vào. Ngoài ra, các thanh ghi GPIO_SET, GPIO_CLEAR và cơ chế ngắt theo mức hoặc theo cạnh được kiểm chứng thông qua INT_TYPE, INT_METHOD, INT_ENABLE và INT_STATUS. Hình 4.4 trình bày dạng sóng, log mô phỏng và Bảng 4.2 tóm tắt các testcase tương ứng.

Bảng 4.2: Tóm tắt testcase kiểm tra module GPIO

STT	Testcase	Nội dung kiểm tra	Kết quả mong đợi
1	Basic output	Ghi IO_DIR = 0xF0; IO_VAL = 0xA0.	Bốn bit cao được cấu hình output; gpio_oe = 0xF0 và gpio_o = 0xA0.
2	Mixed input /output	Giữ bốn bit cao ở chế độ output, đặt gpio_i = 0x05 rồi đọc IO_VAL.	Bốn bit cao trả về dữ liệu output 0xA, bốn bit thấp trả về dữ liệu input đã đồng bộ 0x5; giá trị đọc là 0xA5.
3	Atomic set/clear	Ghi GPIO_SET = 0x0F, GPIO_CLEAR = 0x0C.	Sau thao tác set, gpio_o = 0xAF; sau thao tác clear, gpio_o = 0xA3.
4	Rising-edge interrupt	Bit 0 phát hiện theo cạnh, chọn cạnh lên, cho phép ngắt và tạo chuyển mức 0 → 1 tại gpio_i[0].	Bit 0 của INT_STATUS được thiết lập và irq_o được kích hoạt; sau khi ghi 1 vào bit 0 của INT_STATUS, IRQ phải được xóa.
5	High-level interrupt	Bit 1 phát hiện theo mức, chọn mức cao, cho phép ngắt và đưa gpio_i[1] lên mức 1.	Bit 1 của INT_STATUS được thiết lập và irq_o được kích hoạt; trạng thái được xóa bằng thao tác W1C.

6	Mixed per-pin edge polarity	Cấu hình bit 2 phát hiện cạnh xuống, bit 3 phát hiện cạnh lên; chỉ cho phép hai nguồn ngắt này. Đầu vào được thay đổi từ 0x04 sang 0x08.	Hai cờ ngắt tại bit 2 và bit 3 phải được thiết lập, tức $(INT_STATUS \& 0x0C) = 0x0C$, đồng thời irq_o được kích hoạt.
---	-----------------------------	--	--



Hình 4.4. Kết quả mô phỏng GPIO

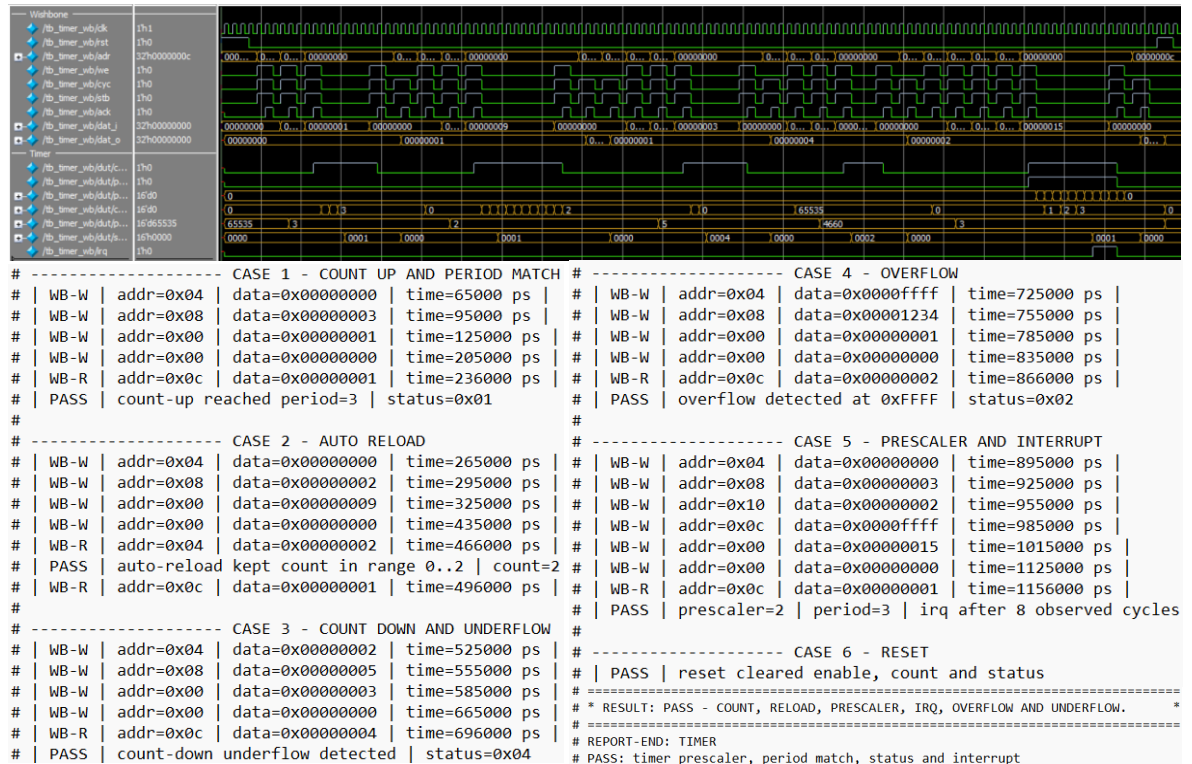
Kết quả mô phỏng trong Hình 4.4 cho thấy toàn bộ chức năng cơ bản của GPIO như xuất ra, đọc vào, mixed I/O, set/clear bit và ngắt (level/edge) đều hoạt động đúng theo thiết kế. Chứng minh này tạo cơ sở vững chắc để nhóm tích hợp vào hệ thống vi điều khiển.

4.2.3. Kiểm tra TIMER

Module Timer được kiểm tra bằng testbench tự động tạo các giao dịch đọc, ghi Wishbone Bus đến các thanh ghi điều khiển. Trong quá trình mô phỏng này, thì các giá trị bộ đếm, cờ trạng thái, tín hiệu irq_o và phản hồi ACK được theo dõi để đối chiếu với các hành vi mong đợi.

Các testcase lần lượt kiểm tra chế độ đếm lên đến giá trị period, tự nạp lại bộ đếm, đếm xuống và phát hiện underflow, phát hiện overflow tại giá trị 0xFFFF, hoạt động của prescaler và cơ chế ngắt Timer. Testbench cũng đồng thời kiểm tra trạng thái của module sau khi reset. Thanh ghi STATUS chứa các cờ period match, overflow và underflow; và tín hiệu irq_o được kích hoạt khi Timer được cho phép ngắt và có ít nhất một cờ trạng thái được thiết lập.

Hình 4.5 là dạng sóng và log kết quả mô phỏng của module Timer. Đồng thời, Bảng 4.3 tóm tắt các testcase và kết quả tương ứng.



Hình 4.5. Kết quả mô phỏng TIMER

Bảng 4.3: Tóm tắt testcase kiểm tra module Timer

STT	Testcase	Nội dung kiểm tra	Kết quả mong đợi	Kết quả mô phỏng
1	Count up and period match	Ghi COUNT = 0, PERIOD = 3, sau đó cho phép Timer đếm lên.	Khi giá trị đếm đạt period, cờ period match tại STATUS[0] phải được thiết lập.	STATUS = 0x01, Timer phát hiện đúng period bằng 3.
2	Auto reload	Ghi COUNT = 0, PERIOD = 2, bật chế độ auto reload và cho Timer chạy trong nhiều chu kỳ.	Khi đạt period, Timer tự nạp lại giá trị đếm về 0; giá trị đếm luôn nằm trong miền từ 0 đến 2.	Sau kiểm tra, COUNT = 2, giá trị bộ đếm không vượt quá period.

3	Count down and underflow	Ghi COUNT = 2, cấu hình đếm xuống và cho Timer tiếp tục chạy đến giá trị 0.	Khi tiếp tục đếm tại giá trị 0, cờ underflow tại STATUS[2] phải được thiết lập.	STATUS = 0x04, Timer phát hiện đúng underflow.
4	Overflow	Ghi COUNT = 0xFFFF, cấu hình Timer đếm lên và đặt PERIOD = 0x1234.	Khi bộ đếm ở giá trị cực đại và tiếp tục đếm lên, cờ overflow tại STATUS[1] phải được thiết lập.	STATUS = 0x02, overflow tại 0xFFFF được phát hiện.
5	Prescaler and interrupt	Ghi COUNT = 0, PERIOD = 3, PRESCALER = 2; bật Timer, prescaler và cho phép ngắt.	Bộ đếm chỉ cập nhật sau mỗi hai chu kỳ clock; khi đạt period, irq_o được kích hoạt. Sau khi đọc STATUS, IRQ phải được xóa.	IRQ được kích hoạt sau 8 chu kỳ quan sát; STATUS = 0x01 và irq_o được xóa sau thao tác đọc.
6	Reset	Kích hoạt reset trong hai chu kỳ clock, kiểm tra các thanh ghi nội bộ chính.	COUNT = 0, COUNT_ENABLE = 0 và STATUS = 0.	Reset xóa đúng enable, giá trị đếm và cờ trạng thái.

Kết quả Hình 4.5 và trong Bảng 4.3 cho thấy bộ đếm hoạt động đúng ở các chế độ đếm lên, đếm xuống và tự nạp lại giá trị đếm. Các cờ period match, overflow và underflow được thiết lập đúng điều kiện, đồng thời prescaler làm giảm tốc độ cập nhật của bộ đếm theo giá trị cấu hình. Cơ chế ngắt Timer cũng hoạt động đúng: khi cờ trạng thái xuất hiện và cho phép ngắt, irq_o được kích hoạt; sau khi đọc thanh ghi STATUS, các cờ trạng thái và tín hiệu IRQ được xóa. Kết quả này xác nhận module Timer hoạt động đúng ở mức kiểm chứng với Wishbone Slave và có thể sử dụng làm nguồn ngắt định thời trong SoC.

4.2.4. Kiểm tra UART

Để kiểm chứng chức năng của module UART_WB, nhóm xây dựng testbench tự động tạo các giao dịch Wishbone để cấu hình thanh ghi điều khiển, đồng thời kiểm tra quá trình truyền và nhận dữ liệu nối tiếp. Mô phỏng được thực hiện với clock hệ thống 16 MHz, baud rate 1 Mbit/s, hệ số oversampling 16 lần và RX FIFO có độ sâu 32 phần tử. Các thông số này được thiết lập riêng cho testbench nhằm rút ngắn thời gian mô phỏng.

Trước hết, testbench kiểm tra hai định dạng khung truyền cơ bản. Với cấu hình 8 bit dữ liệu, không parity và một stop bit (8N1), byte 0xA5 được truyền và nhận đúng, đồng thời ngắt RX được quan sát. Với cấu hình 7 bit dữ liệu, even parity và một stop bit (7E1), dữ liệu 0x55 cũng được nhận chính xác và không xuất hiện cờ lỗi.

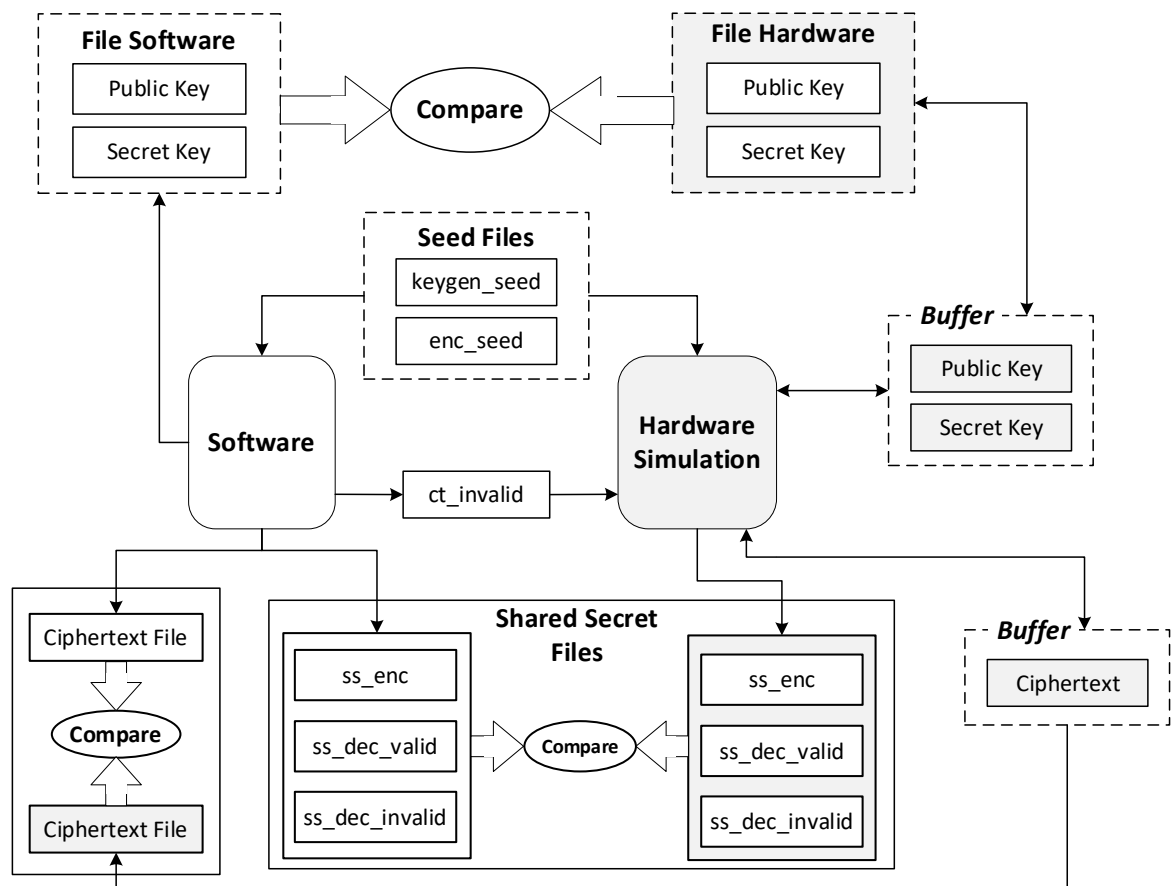
Bên cạnh trường hợp truyền nhận bình thường, testbench còn chủ động tạo frame có parity sai và frame có stop bit bị giữ ở mức thấp để kiểm tra khả năng phát hiện lỗi của UART. Cơ chế RX FIFO được kiểm tra bằng cách gửi liên tiếp các byte 0x12, 0x34 và 0x56, sau đó xác nhận dữ liệu được đọc ra theo đúng thứ tự đã truyền. Cuối cùng, 33 byte được gửi liên tục đến FIFO có độ sâu 32 phần tử nhằm kiểm tra cờ overrun. Bảng 4.4 trình bày tóm tắt các testcase và kết quả mong đợi, Hình 4.6 thể hiện dạng sóng và log kết quả mô phỏng của module UART.

Bảng 4.4: Tóm tắt testcase kiểm tra module UART_WB

STT	Testcase	Nội dung kiểm tra	Kết quả mong đợi
1	Frame 8N1 và RX interrupt	Cấu hình 8 bit dữ liệu, không parity, một stop bit; truyền byte 0xA5.	Dữ liệu nhận bằng 0xA5, không có lỗi frame và ngắt RX được kích hoạt khi có dữ liệu nhận.
2	Frame 7E1	Cấu hình 7 bit dữ liệu, even parity, một stop bit; truyền byte 0x55.	Dữ liệu được nhận đúng và không xuất hiện cờ parity hoặc framing error.
3	Parity error detection	Chủ động truyền frame có bit parity không đúng.	Dữ liệu được nhận, đồng thời cờ parity_error được thiết lập.

Kết quả mô phỏng trong Hình 4.6 cho thấy module UART_WB hoạt động đúng với các định dạng khung 8N1 và 7E1. Cơ chế truyền nhận dữ liệu, ngắt nhận, RX FIFO và các cờ phát hiện lỗi parity, framing và overrun đều được kiểm tra thành công. Việc dữ liệu trong RX FIFO được đọc ra đúng thứ tự cũng xác nhận khả năng tiếp nhận chuỗi dữ liệu liên tiếp của module. Kết quả này cho thấy UART_WB đáp ứng yêu cầu làm giao diện truyền thông nối tiếp và hỗ trợ quá trình nạp firmware, theo dõi log trong hệ thống SoC.

4.2.5. Kiểm tra Kyber Accelerator



Hình 4.7. Mô hình kiểm tra Kyber Accelerator với Golden Model

Hình 4.7 trình bày mô hình kiểm tra phần cứng bộ tăng tốc Kyber. Kết quả của phần cứng được đối chiếu với dữ liệu do phần mềm tham chiếu sinh ra nhằm xác minh tính đúng đắn của các phép sinh khóa, đóng gói khóa và mở gói khóa. Đầu tiên, phần mềm và phần cứng sử dụng các seed xác định để tạo dữ liệu đầu vào giống nhau. Sau mỗi phép toán, testbench đọc dữ liệu đầu ra từ vùng đệm của bộ tăng tốc qua giao diện Wishbone và so sánh từng byte với dữ liệu tham chiếu.

Quy trình chi tiết của việc kiểm tra ở Hình 4.7 như sau:

Khởi tạo khóa:

- Phần mềm tham chiếu và phân cứng cùng sử dụng `keygen_seed` để thực hiện phép sinh khóa để tạo ra cặp khóa.
- Kết quả sinh khóa gồm public key và secret key được tạo bởi chương trình C và lưu làm dữ liệu tham chiếu.
- Sau khi phân cứng hoàn thành KeyGen, testbench đọc public key và secret key từ vùng đệm dữ liệu của Kyber để so sánh với kết quả phần mềm.
- Public key và secret key sau đó được giữ lại trong vùng đệm để phục vụ các phép đóng và mở gói khóa tiếp theo.

Đóng gói khóa:

- Phần mềm tham chiếu sử dụng `enc_seed` cùng public key để thực hiện Encapsulation, từ đó tạo ciphertext và shared secret (`ss_enc`).
- Ciphertext và shared secret do phần mềm tạo được lưu làm dữ liệu tham chiếu.
- Testbench ghi `enc_seed` vào bộ tăng tốc và kích hoạt chế độ tương ứng.
- Phần cứng sử dụng public key đã có trong vùng đệm để tạo ciphertext tương ứng và ghi shared secret vào vùng SS output.
- Ciphertext và shared secret do phần cứng tạo ra được đọc lại để so sánh với ct và `ss_enc` của phần mềm tham chiếu

Mở gói khóa với ciphertext hợp lệ:

- Phần cứng sử dụng secret key và ciphertext đang lưu trong các vùng đệm dữ liệu để thực hiện Decapsulation hợp lệ.
- Shared secret tạo ra sau phép mở gói được đọc từ vùng SS output và đối chiếu với `ss_dec_valid` của chương trình tham chiếu.
- Với ciphertext hợp lệ, shared secret của Decapsulation phải trùng với shared secret đã sinh trong bước Encapsulation.

Mở gói khóa với ciphertext không hợp lệ:

- Phần mềm tạo `ct_invalid` bằng cách thay đổi một bit của ciphertext hợp lệ.
- Phần cứng dùng secret key và ciphertext đã sửa để tạo shared secret dự phòng.
- Kết quả đọc từ SS output so sánh với `ss_dec_invalid` của chương trình C.

Khối Kyber được kiểm tra bằng bộ Known Answer Test tạo từ chương trình tham chiếu C của nhóm đã viết Kyber. Testbench nạp seed qua giao diện Wishbone, khởi động từng phép toán và so sánh từng byte đầu ra của RTL với dữ liệu tham chiếu. Hình 4.8 và 4.9 trình bày kết quả so sánh các file pk, sk, ct, ss của phần mềm và phần cứng tạo ra. Tất cả có tổng cộng 10,000 bài test với các chuỗi seed đầu vào khác nhau. Với mỗi vector, public key 800 byte, secret key 1632 byte, ciphertext 768 byte và shared secret 32 byte đều được đối chiếu đầy đủ.

Các trường hợp kiểm tra gồm sinh khóa, đóng gói khóa dùng chung, mở gói với ciphertext hợp lệ và mở gói với ciphertext bị sửa. Trường hợp ciphertext lỗi được so sánh với shared secret fallback do chương trình tham chiếu tạo ra, nhờ đó đồng thời kiểm tra được đường xử lý từ chối ngầm khi gặp tấn công giả mạo.

Sau khi mô phỏng toàn bộ các test đều cho ra kết quả giữa phần cứng và phần mềm đều giống nhau như Hình minh hoạ 4.8 và 4.9. Qua số lượng test khá lớn như vậy cho thấy phần cứng đảm bảo độ bao phủ về mặt chức năng khá tốt. Như vậy, mô hình kiểm tra này giúp xác minh tính chính xác và đồng nhất giữa các kết quả được tạo ra bởi phần mềm và phần cứng trong quá trình sinh khoá, đóng và giải gói khoá.

```
# =====
# * KYBER512 WISHBONE ACCELERATOR - KNOWN ANSWER TEST *
# =====
# | Vectors=1 | Global range=0..0 |
# | PK=800 B | SK=1632 B | CT=768 B | SS=32 B |
#
# ----- KAT VECTOR 0 / 0 -----
# ////////////////////////////////// KEY GENERATION //////////////////////////////////
# | DONE | keygen | cycles=7805 | state=0x00 |
# | PASS | pk | compared 800 bytes against C reference |
# | PASS | sk | compared 1632 bytes against C reference |
# ////////////////////////////////// ENCAPSULATION //////////////////////////////////
# | DONE | encaps | cycles=10026 | state=0x00 |
# | PASS | ct | compared 768 bytes against C reference |
# | PASS | ss_enc | compared 32 bytes against C reference |
# ////////////////////////////////// VALID DECAPSULATION //////////////////////////////////
# | DONE | decaps_valid | cycles=13753 | state=0x00 |
# | PASS | ss_dec_valid | compared 32 bytes against C reference |
# ////////////////////////////////// INVALID DECAPSULATION //////////////////////////////////
# | DONE | decaps_invalid | cycles=14990 | state=0x00 |
# | PASS | ss_dec_invalid | compared 32 bytes against C reference |
# =====
# * RESULT: PASS - RTL OUTPUT MATCHES KYBER512 C REFERENCE. *
# =====
```

Hình 4.8. Kết quả bài test KAT (Known Answer Test) thứ 1

```

# ----- KAT VECTOR 9999 / 9999 -----
# ////////////////////////////////// KEY GENERATION //////////////////////////////////
# | DONE | keygen      | cycles=7771 | state=0x00 |
# | PASS | pk             | compared 800 bytes against C reference |
# | PASS | sk             | compared 1632 bytes against C reference |
# ////////////////////////////////// ENCAPSULATION //////////////////////////////////
# | DONE | encaps        | cycles=10000 | state=0x00 |
# | PASS | ct            | compared 768 bytes against C reference |
# | PASS | ss_enc        | compared 32 bytes against C reference |
# ////////////////////////////////// VALID DECAPSULATION //////////////////////////////////
# | DONE | decaps_valid   | cycles=13727 | state=0x00 |
# | PASS | ss_dec_valid   | compared 32 bytes against C reference |
# ////////////////////////////////// INVALID DECAPSULATION //////////////////////////////////
# | DONE | decaps_invalid | cycles=14964 | state=0x00 |
# | PASS | ss_dec_invalid | compared 32 bytes against C reference |
# =====
# * RESULT: PASS - RTL OUTPUT MATCHES KYBER512 C REFERENCE. *
# =====

```

Hình 4.9. Kết quả bài test KAT (Known Answer Test) thứ 10,000

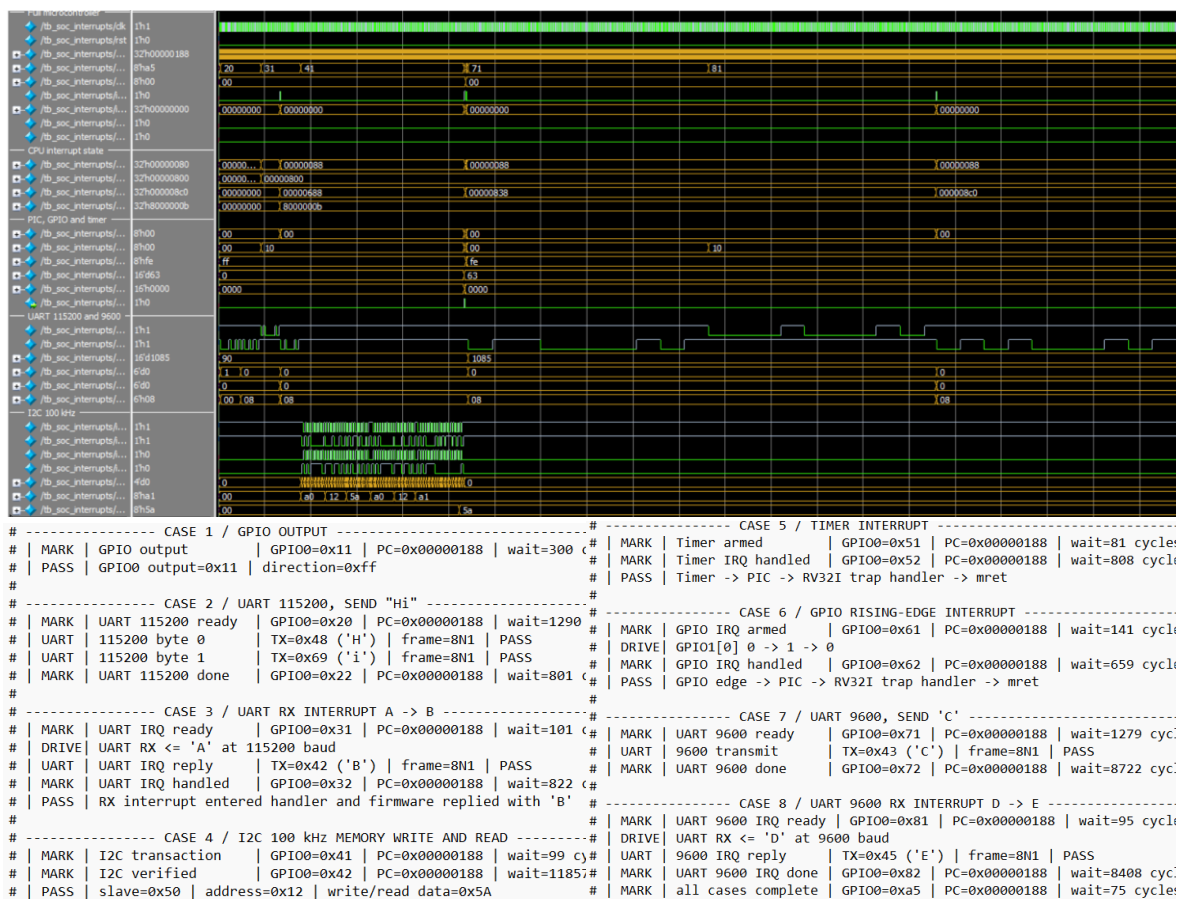
Mô phỏng còn kiểm tra tốc độ thực thi của từng chế độ trong phần cứng Kyber. Xung nhịp mà hệ thống đạt được hiện tại là 167 MHz. Bảng 4.5 dưới đây thống kê chu kỳ thực thi và thời gian trung bình tương ứng từng chế độ qua mô phỏng.

Bảng 4.5: Tốc độ thực thi trung bình của phần cứng Kyber từ mô phỏng

Phép toán	Số chu kỳ ($\times 10^3$)	Thời gian tại 167 MHz
Key Generation	7.78	46.7 us
Encapsulation	10.01	60.1 us
Decapsulation (ct hợp lệ)	13.74	82.4 us
Decapsulation (ct không hợp lệ)	14.97	89.66 us

4.2.6. Kiểm tra toàn bộ vi điều khiển

Sau khi kiểm tra từng module cụ thể trong hệ thống. Nhóm tiến hành mô phỏng và kiểm tra toàn bộ vi điều khiển với các tình huống cụ thể như thể hiện trong Bảng 4.6. Trong mô phỏng này, firmware được thực thi trên lõi RISC-V, điều khiển các ngoại vi thông qua bus Wishbone và xử lý các yêu cầu ngắt thông qua PIC. Mô phỏng sử dụng chu kỳ clock 6 ns, tương đương tần số 166,667 MHz, UART được kiểm tra ở định dạng 8N1 với hai baud rate 115200 bit/s và 9600 bit/s, trong khi giao tiếp I2C được cấu hình ở tốc độ 100 kHz.



Hình 4.10. Kết quả mô phỏng toàn bộ hệ thống

Bảng 4.6: Tóm tắt testcase kiểm tra vi điều khiển

STT	Testcase	Nội dung kiểm tra	Kết quả mô phỏng
1	GPIO output	Firmware cấu hình GPIO0 là ngõ ra và ghi giá trị 0x11.	PASS. gpio0_o = 0x11, gpio0_oe = 0xFF.
2	UART 115200 transmit	Firmware truyền chuỗi “Hi” qua UART với khung 8N1.	Testbench nhận đúng “H” (0x48) và “i” (0x69).
3	UART 115200 RX interrupt	Testbench gửi “A” vào RX; UART tạo ngắt qua PIC và firmware phản hồi gửi “B”.	Nhận “A”, xử lý ngắt và truyền lại “B” (0x42).
4	I2C write/read	Firmware ghi 0x5A vào slave 0x50, địa chỉ 0x12, sau đó đọc lại dữ liệu.	Chuỗi địa chỉ và dữ liệu I2C đúng; dữ liệu đọc lại bằng 0x5A.

5	Timer interrupt	Timer period = 63, tạo yêu cầu ngắt qua PIC; firmware xử lý và dừng Timer.	Luồng Timer - PIC - trình phục vụ ngắt RISC-V => mret hoạt động đúng
6	GPIO rising-edge interrupt	Testbench tạo cạnh lên tại GPIO1[0].	Luồng GPIO1 - PIC - trình phục vụ ngắt RISC-V => mret hoạt động đúng
7	UART 9600 transmit	Firmware cấu hình UART ở 9600 bit/s và truyền “C”.	Testbench nhận đúng C (0x43) với khung 8N1.
8	UART 9600 RX interrupt	Testbench gửi “D” vào RX; firmware xử lý ngắt và phản hồi “E”.	Nhận “D”, xử lý ngắt và truyền lại “E” (0x45).

Kết quả mô phỏng trong Hình 4.10 và Bảng 4.6 cho thấy firmware có thể điều khiển GPIO, UART, I2C và Timer thông qua bus Wishbone. Các trường hợp ngắt từ UART, Timer và GPIO đều được PIC ghi nhận, chuyển đến lõi RISC-V và được xử lý bởi trình phục vụ ngắt trước khi quay lại chương trình chính.

Đối với giao tiếp I2C, dữ liệu ghi đến slave và dữ liệu đọc lại đều khớp giá trị mong đợi. Với UART, quá trình truyền dữ liệu và xử lý ngắt được kiểm tra ở cả hai baud rate 115200 bit/s và 9600 bit/s. Kết quả này xác nhận sự phối hợp đúng giữa lõi RISC-V, Wishbone, PIC và các ngoại vi chính ở mức mô phỏng tích hợp.

Tuy nhiên, testcase này mới kiểm chứng hệ thống trong môi trường mô phỏng với mô hình slave I2C và tín hiệu UART, GPIO do testbench tạo ra. Việc triển khai trên FPGA và thử nghiệm với các thiết bị ngoại vi thực tế sẽ được thực hiện sau đây để đánh giá độ chính xác của hệ thống trong thực tế.

4.3. HIỆN THỰC TOÀN BỘ HỆ THỐNG TRÊN FPGA, KIỂM TRA VÀ ĐÁNH GIÁ KẾT QUẢ

4.3.1. Kết quả tổng hợp

Quá trình tổng hợp được thực hiện trên FPGA ZCU102 để đánh giá hiệu suất và tài nguyên sử dụng của hệ thống. Kết quả tổng hợp cung cấp thông tin chi tiết về

việc phân bổ tài nguyên phần cứng và hiệu quả của thiết kế. Thiết kế được chạy synthesis và implement, xung nhịp tối đa thiết kế đạt được là 190 MHz.

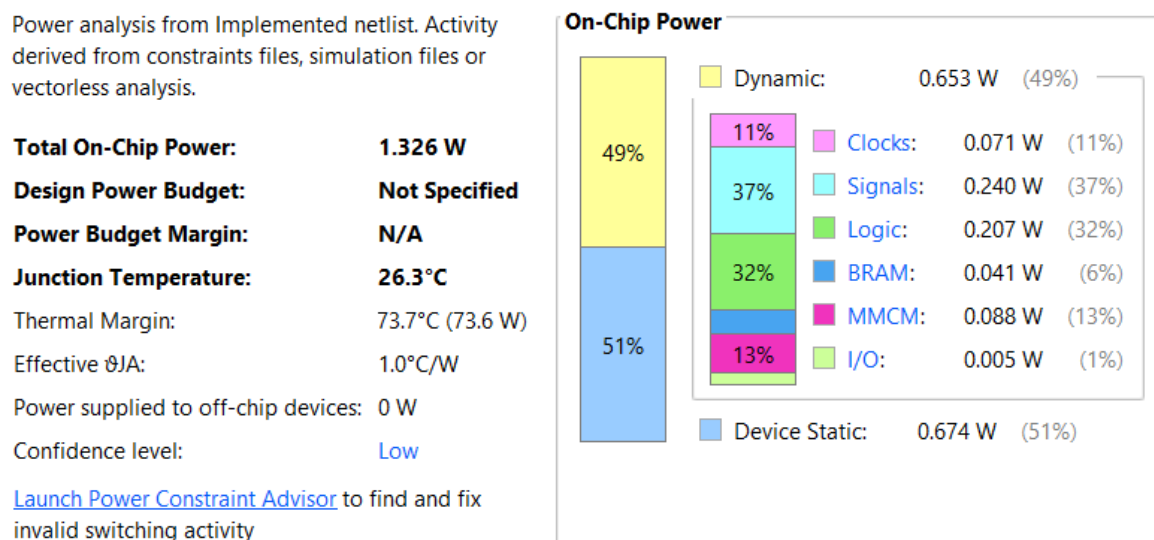
Bảng 4.7 cho thấy kết quả tổng hợp toàn vi điều khiển chiếm 24706 LUTs, 10797 FFs, 25 BRAM và 4793 CLB. Trong đó, khối Kyber Accelerator là thành phần chiếm tài nguyên lớn nhất, với 19443 LUTs và 8091 FFs, tương ứng khoảng 78,0% tổng số LUTs và 73,6% tổng số FFs của toàn hệ thống. Khối RISC-V RV32I là thành phần sử dụng tài nguyên lớn thứ hai, chiếm 3957 LUTs và 1543 FFs. Hai khối này đóng góp lớn nhất về logic tổ hợp. Phần còn lại, các khối khác chiếm chi phí logic tổ hợp và thanh ghi thấp. Cuối cùng về BRAM, tổng thiết kế sử dụng 25 BRAM. Trong đó, SRAM chiếm 8 BRAM, IMEM chiếm 8 BRAM và Kyber Accelerator sử dụng 9 BRAM. Như vậy, phần BRAM chủ yếu được dùng cho bộ nhớ chương trình, bộ nhớ dữ liệu và các vùng lưu trữ nội bộ của Kyber.

Bảng 4.7: Tài nguyên của từng module trong hệ thống trên ZCU102 FPGA

Module	LUTs	FFs	BRAM	CLB
Vi điều khiển	24706	10797	25	4793
GPIO0/1	40	140	0	32
I2C	58	87	0	18
IMEM	0	2	8	2
Kyber	19443	8091	9	3897
SRAM	36	4	8	21
TIMER	78	90	0	25
UART	153	152	0	40
Wishbone bus	0	1	0	1
PIC	3	26	0	7
RISC-V	3957	1543	0	1137

Hệ thống có thể tổng hợp thành công ở các mức tần số từ 125 MHz (tần số có sẵn trên board) đến tối đa 190 MHz, nhóm chọn 167 MHz để cân bằng giữa hiệu năng và tiêu thụ năng lượng (PPA), báo cáo timing cho thấy các đường dữ liệu đều đáp ứng yêu cầu setup time, với giá trị Worst Negative Slack (WNS) đạt 0.192 ns. Đồng thời cũng không vi phạm về hold time, với Worst Hold Slack (WHS) đạt 0.014 ns.

Kết quả Hình 4.11 cho thấy tổng công suất tiêu thụ của thiết kế ở 167 MHz là 1.326 W (đạt 1.177 W và 1.426 W tương ứng ở 125 MHz và 190 MHz). Trong đó, static power chiếm 0.674 W (51%) và dynamic power chiếm 0.653 W (49%) phần lớn đến từ Kyber Accelerator với 0.481 W (36%), I/O power rất nhỏ, chỉ 0.006 W (1%) phản ánh điện năng tiêu thụ qua chân I/O tương đối thấp bởi số giao tiếp ngoại vi không nhiều và tốc độ không cao.

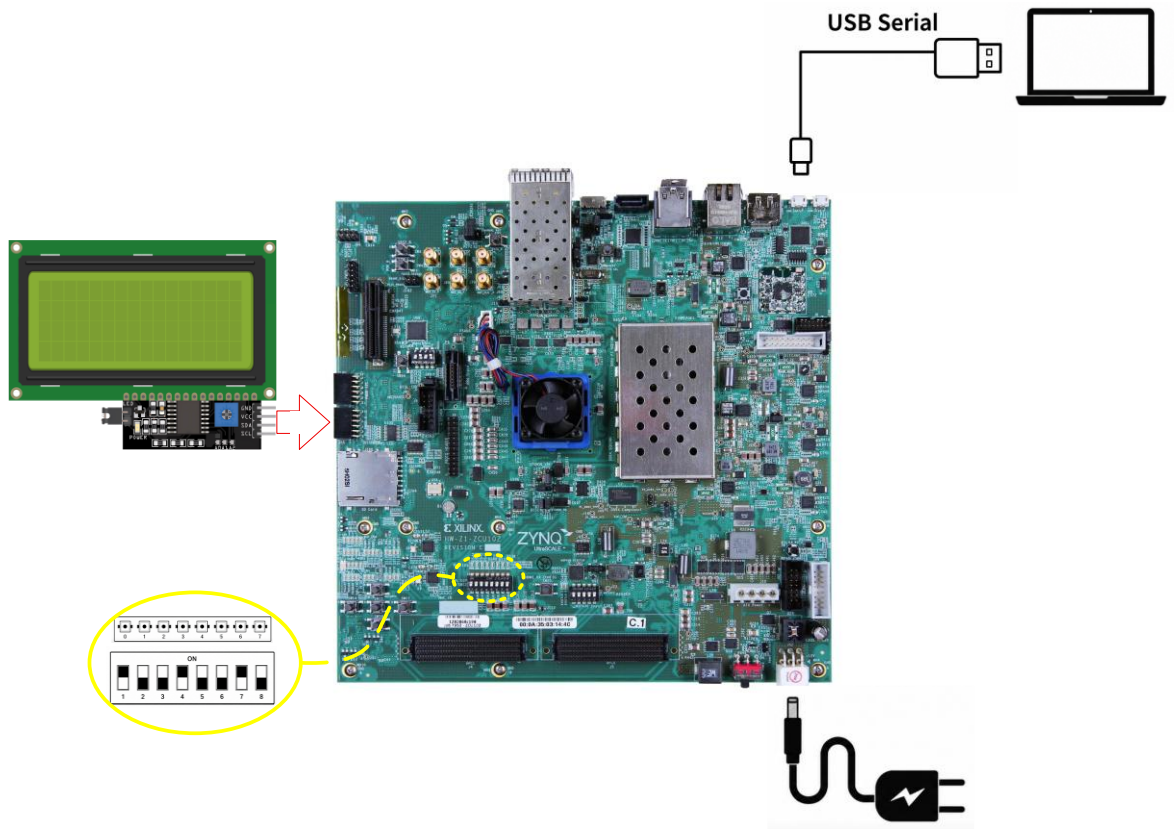


Hình 4.11. Tổng công suất tiêu thụ của thiết kế

Kết luận về kết quả tổng hợp: Kết quả tổng hợp phần nào chứng minh tính khả thi của việc tích hợp thuật toán Kyber vào hệ thống vi điều khiển trên FPGA. Khối Kyber tiêu tốn phần lớn tài nguyên và mức tiêu thụ năng lượng khiến đây là một thách thức lớn cần tiếp tục tối ưu. Nhìn chung, hệ thống vẫn hoạt động ổn định với kết quả này.

4.3.2. Hiện thực trên FPGA

Sau khi hoàn tất kiểm chứng các module và hệ thống qua mô phỏng, hệ thống vi điều khiển được triển khai trên bo mạch ZCU102 để đánh giá khả năng hoạt động trong môi trường phần cứng thực tế. Thiết kế được hiện thực và đánh giá chỉ thông qua phần PL của board (không dùng PS), trong đó lõi RISC-V, các ngoại vi Wishbone và bộ tăng tốc Kyber được tổng hợp vào phần Programmable Logic của FPGA. Máy tính được sử dụng để nạp bitstream, biên dịch firmware, truyền firmware qua UART và quan sát các thông báo trạng thái của hệ thống.



Hình 4.12. Mô hình triển khai hệ thống trên bo mạch FPGA

Hình 4.12 mô tả mô hình kiểm chứng trên FPGA. Các thành phần chính của mô hình bao gồm và chức năng đi kèm như sau:

- **Bo mạch ZCU102:** FPGA sử dụng để hiện thực hệ thống vi điều khiển, kết quả được tổng hợp và triển khai trong vùng Programmable Logic của FPGA.
- **Giao diện JTAG/USB:** được sử dụng để nạp bitstream vào FPGA.
- **Cổng USB Serial/UART:** kết nối giữa bo mạch và máy tính để truyền firmware đến bootloader và xem log.
- **Laptop:** dùng để nạp firmware, theo dõi log và kết quả thử nghiệm qua màn hình GUI với Serial Monitor.
- **LCD I2C:** hiển thị trạng thái khởi động và hoạt động của firmware.
- **8 DIP switch và 8 LED:** DIP switch dùng để chọn các chức năng thử nghiệm; LED hiển thị trạng thái hoạt động của firmware hoặc Timer.

Sau khi đã có mô hình, tiếp theo chúng tôi xây dựng chương trình thử nghiệm nhằm kiểm tra đồng thời các thành phần chính của vi điều khiển trong điều kiện hoạt động thực tế. Khi khởi động, chương trình lần lượt khởi tạo UART, GPIO, PIC,

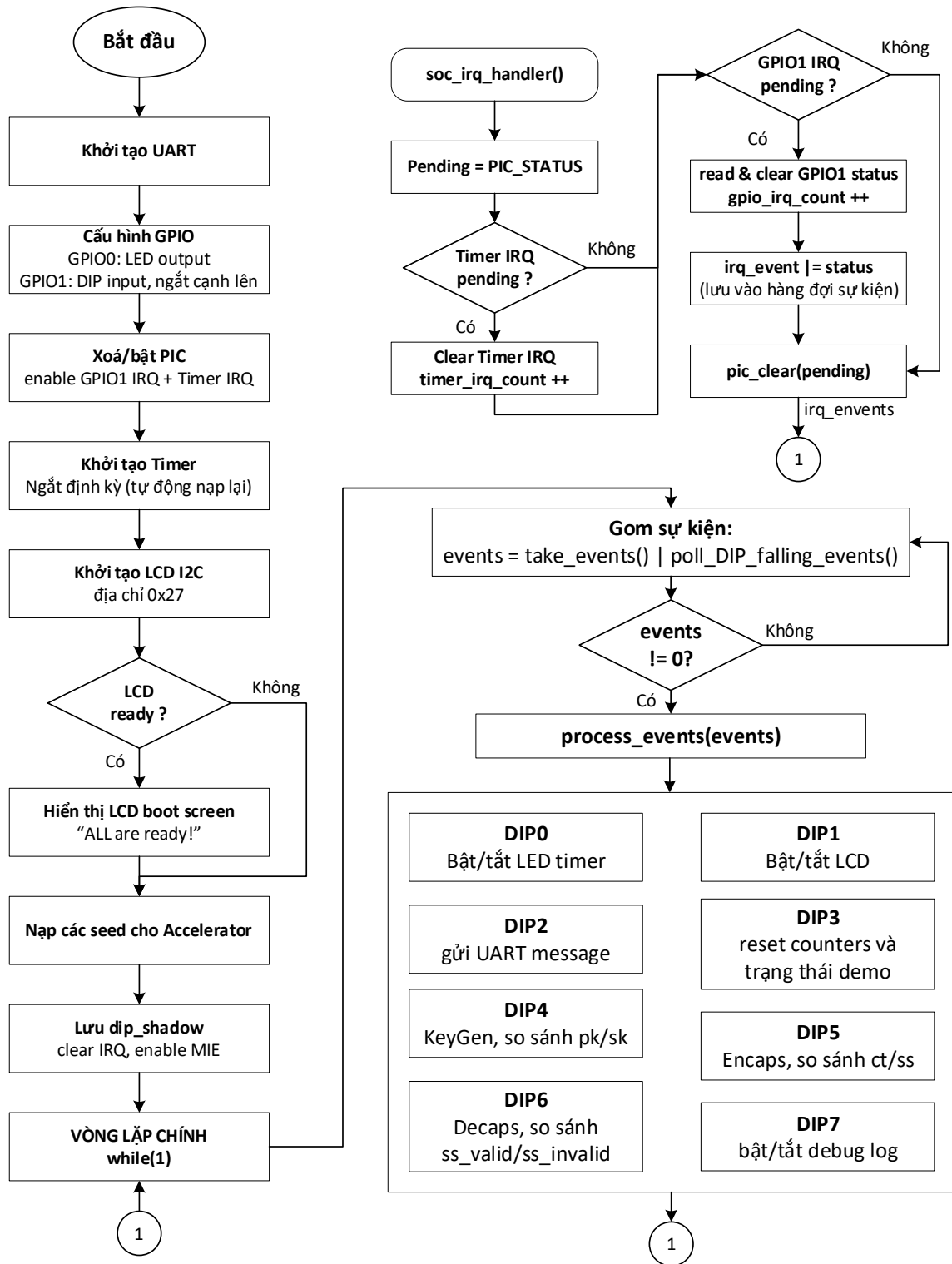
Timer, LCD I2C và bộ tăng tốc. Sau đó, firmware nạp các seed của mô hình tham chiếu cho bộ tăng tốc, reset trạng thái và chờ sự kiện từ DIP switch.

Chương trình sử dụng GPIO0 để điều khiển LED, GPIO1 thì được sử dụng để đọc trạng thái DIP switch. Ngắt GPIO và ngắt Timer được gửi đến PIC, sau đó được lõi RISC-V xử lý trong trình phục vụ ngắt. Các tác vụ có thời gian xử lý dài như Key Generation, Encapsulation, Decapsulation, so sánh với tham chiếu và in dữ liệu qua UART được thực hiện trong vòng lặp chính.

Hình 4.13 trình bày lưu đồ hoạt động của firmware thực nghiệm. Trong đó, ngắt Timer được dùng để cập nhật LED khi chế độ LED timer được bật. Sự kiện từ DIP switch được thu thập bằng ngắt GPIO kết hợp với cơ chế polling cạnh xuống, sau đó được xử lý trong vòng lặp chính theo từng bit DIP. Bảng 4.8 thể hiện chức năng của các DIP switch trong chương trình thực nghiệm.

Bảng 4.8: Chức năng các DIP switch trong chương trình thử nghiệm

DIP	Chức năng	Nội dung thực hiện
DIP0	Timer LED	Bật hoặc tắt chế độ cập nhật LED bằng ngắt Timer.
DIP1	LCD control	Bật hoặc tắt LCD I2C.
DIP2	UART message	Gửi một thông điệp thử nghiệm qua UART.
DIP3	Reset demo state	Xóa bộ đếm ngắt Timer, bộ đếm ngắt GPIO, LED counter và trạng thái KeyGen/Encaps đã lưu.
DIP4	Key Generation	Thực hiện Key Generation, đọc pk và sk từ bộ tăng tốc, sau đó so sánh với mô hình tham chiếu.
DIP5	Encapsulation	Thực hiện Encapsulation, tạo ct và ss_enc. Nếu chưa có keypair, tự động chạy KeyGen trước.
DIP6	Decapsulation	Thực hiện Decapsulation với ct hợp lệ và ct bị thay đổi một bit. So sánh shared secret.
DIP7	Debug mode	Bật hoặc tắt chế độ in thông tin trạng thái DIP, IRQ, LED và các bộ đếm qua UART.

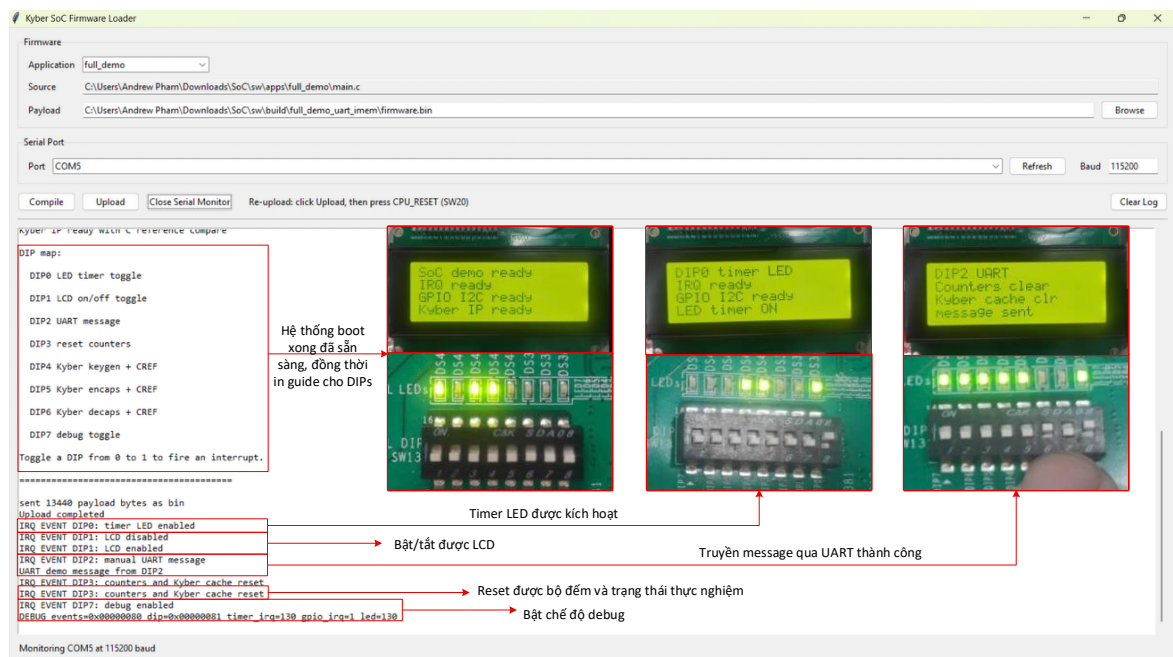


Hình 4.13. Lưu đồ hoạt động của firmware điều khiển FPGA

Lưu đồ trên có thể được diễn đạt ngắn gọn như sau: Sau khi khởi tạo UART, GPIO, PIC, Timer, LCD I2C và bộ tăng tốc Kyber, chương trình nạp các seed tham chiếu, lưu trạng thái ban đầu của DIP switch và đi vào vòng lặp chính. Trong quá trình hoạt động, ngắt Timer được xử lý trực tiếp để cập nhật bộ đếm và trạng thái

LED, trong khi ngắt GPIO1 đọc trạng thái DIP, xóa cờ ngắt và ghi nhận sự kiện vào biến `irq_events`. Tại vòng lặp chính, firmware kết hợp các sự kiện từ ngắt GPIO với cơ chế polling cạnh xuống của DIP switch, sau đó gọi hàm `process_events()` để thực hiện chức năng tương ứng. Các DIP switch lần lượt dùng để điều khiển LED Timer, LCD, UART, reset trạng thái demo, thực hiện Key Generation, Encapsulation, Decapsulation và bật chế độ debug. Các phép toán Kyber và quá trình so sánh với mô hình tham chiếu được thực hiện trong vòng lặp chính thay vì trong trình phục vụ ngắt, giúp ISR ngắn gọn và tránh làm chậm quá trình xử lý ngắt của hệ thống.

Với lưu đồ trên, ta có firmware tương ứng được biên dịch và nạp payload vào Upload IMEM để tiến hành thực nghiệm trên FPGA cho vi điều khiển. Trong quá trình thử nghiệm, người dùng thao tác các DIP switch để kích hoạt các chức năng tương ứng, đồng thời theo dõi trạng thái qua LED, LCD I2C và Serial Monitor trên máy tính. Các thông báo UART được sử dụng để xác nhận từng bước xử lý của firmware và kết quả so sánh với dữ liệu tham chiếu.



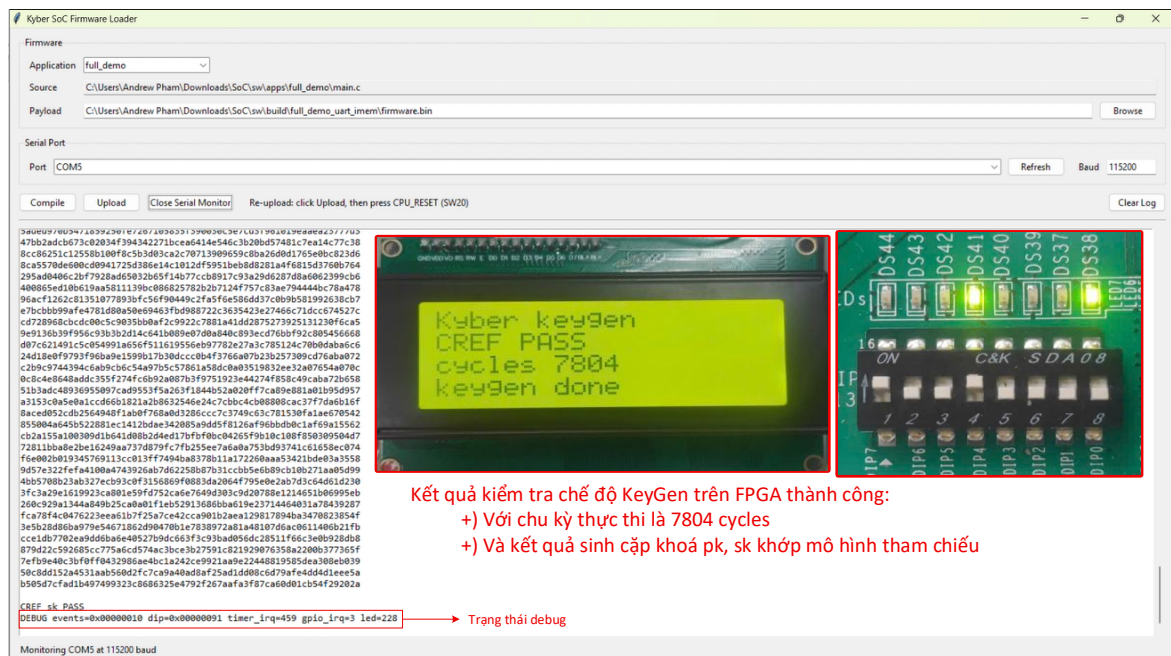
Hình 4.14. Kết quả thực nghiệm các chức năng ngoại vi trên FPGA

Hình 4.14 trình bày kết quả thử nghiệm các ngoại vi cơ bản của hệ thống. Sau khi khởi động, LCD hiển thị trạng thái sẵn sàng của SoC và các ngoại vi. Khi kích hoạt DIP0, Timer interrupt được dùng để cập nhật trạng thái LED. DIP1 cho phép bật hoặc tắt LCD, trong khi DIP2 gửi thông điệp thử nghiệm qua UART. Khi DIP3 được

kích hoạt, firmware xóa các bộ đếm ngắt Timer và GPIO, đưa giá trị LED về 0, đồng thời có thể xóa các cờ trạng thái của phần thực thi mã hoá. Các log trên Serial Monitor xác nhận firmware nhận đúng sự kiện DIP switch, xử lý ngắt và điều khiển các ngoại vi theo luồng đã thiết kế.

DIP7 được sử dụng để bật hoặc tắt chế độ debug. Khi debug mode được kích hoạt, sau mỗi sự kiện DIP được xử lý, firmware gửi thêm thông tin trạng thái qua UART, gồm giá trị sự kiện đang xử lý, trạng thái hiện tại của DIP switch, số lần xảy ra ngắt Timer, số lần xảy ra ngắt GPIO và giá trị đang xuất ra GPIO0. Giá trị GPIO0 này đồng thời được biểu diễn dưới dạng mẫu LED trên bo mạch, do đó người dùng có thể đối chiếu mã hexa trên Serial Monitor với trạng thái các LED đang sáng. Chế độ này hỗ trợ quan sát trực quan quá trình thực thi của firmware.

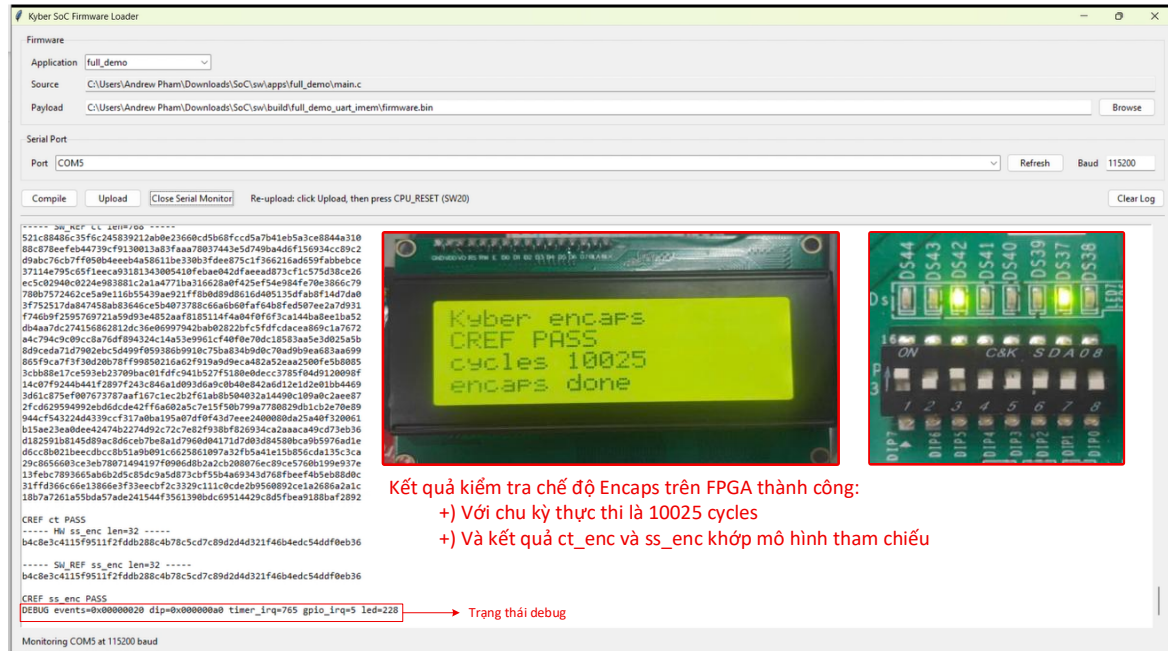
Sau khi trạng thái demo được khởi tạo lại và chế độ debug được bật, firmware tiếp tục thực hiện các phép kiểm tra Kyber thông qua DIP4, DIP5 và DIP6. Các kết quả được hiển thị đồng thời trên LCD, LED và Serial Monitor, trong đó UART được dùng để in log chi tiết và đối chiếu với dữ liệu tham chiếu.



Hình 4.15. Kết quả thực nghiệm KeyGen trên FPGA

Hình 4.15 thể hiện kết quả thực hiện phép Key Generation trên bo mạch FPGA. Khi DIP4 được kích hoạt, firmware điều khiển bộ tăng tốc Kyber tạo public key và secret key từ seed tham chiếu. Sau khi phép toán hoàn thành, firmware đọc dữ liệu từ

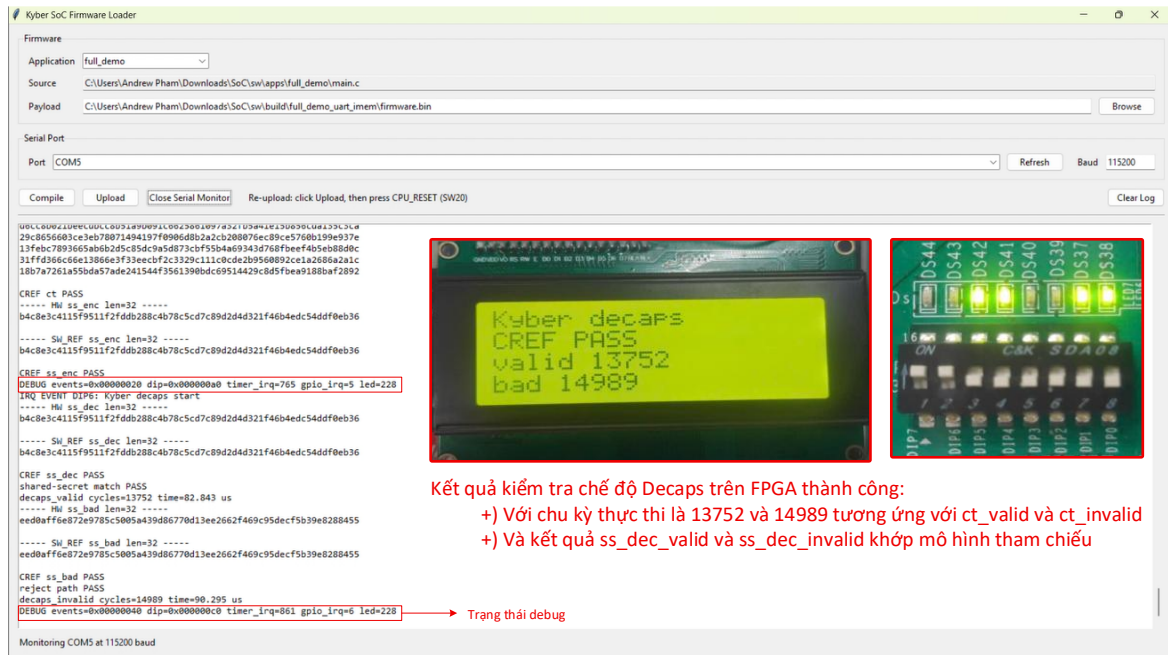
vùng đệm của accelerator, so sánh từng byte với C và hiển thị trạng thái ‘CREF PASS’ qua LCD và UART. Kết quả này xác nhận public key và secret key do phần cứng tạo ra trùng khớp với mô hình tham chiếu. Trong lần thử nghiệm được ghi nhận, bộ tăng tốc hoàn thành phép KeyGen sau khoảng 7.8 nghìn chu kỳ.



Hình 4.16. Kết quả thực nghiệm Encaps trên FPGA

Tiếp theo, Hình 4.16 trình bày quá trình Encapsulation được kích hoạt bằng DIP5. Firmware sử dụng public key đã tạo ở bước trước để thực hiện đóng gói khóa, sau đó đọc ciphertext và shared secret từ bộ tăng tốc để đối chiếu với C. Kết quả ‘CREF PASS’ cho thấy ciphertext cùng shared secret tạo bởi phần cứng khớp với dữ liệu từ chương trình tham chiếu C. Trong lần thử nghiệm được ghi nhận, bộ tăng tốc hoàn thành phép Encapsulation sau khoảng 10 nghìn chu kỳ.

Hình 4.17 minh họa phép Decapsulation được kích hoạt bằng DIP6. Trước hết, firmware sử dụng ciphertext hợp lệ để tạo shared secret và đối chiếu kết quả với shared secret từ bước Encapsulation. Sau đó, firmware thay đổi một bit của ciphertext để tạo trường hợp ciphertext không hợp lệ, rồi kích hoạt lại Decapsulation nhằm kiểm tra nhánh xử lý implicit rejection. LCD và log UART đều hiển thị kết quả PASS cho cả trường hợp ciphertext hợp lệ và ciphertext đã bị thay đổi. Các số chu kỳ của hai trường hợp Decapsulation cũng được ghi nhận riêng, qua đó cho thấy thiết kế xử lý đúng cả nhánh giải đóng gói bình thường và nhánh ciphertext không hợp lệ.



Hình 4.17. Kết quả thực nghiệm Decaps trên FPGA

Kết quả thực nghiệm xác nhận firmware có thể điều khiển các ngoại vi, thu thập sự kiện từ DIP switch, hiển thị trạng thái qua LCD và LED, đồng thời theo dõi chi tiết quá trình thực thi qua UART. Bộ tăng tốc Kyber thực hiện thành công các phép Key Generation, Encapsulation, Decapsulation hợp lệ và Decapsulation với ciphertext không hợp lệ; các kết quả đầu ra đều trùng khớp với C tham chiếu.

4.3.3. Đánh giá phần cứng Kyber và so sánh với các nghiên cứu tiên tiến khác

Để đánh giá hiệu quả tăng tốc phần cứng so với phần mềm, nhóm đã đo và so sánh số chu kỳ thực thi giữa chương trình chạy trên CPU i5-12500H và khối tăng tốc phần cứng. Kết quả được tổng hợp trong Bảng 4.9, gồm số chu kỳ ở từng chế độ hoạt động của Kyber và hệ số tăng tốc speed-up factor. Qua đó, có thể đánh giá hiệu quả của kiến trúc phần cứng trong quá trình thực thi thuật toán.

Bảng 4.9: Số chu kỳ và hệ số tăng tốc của thuật toán Kyber

Chế độ	SW (i5-12500H)	HW	Speed-up factor
KeyGen	61.328	7785	7,86×
Encaps.	75.823	10016	7,56×
Decaps. (valid)	101.486	13743	7,38×
Decaps. (invalid)	102.620	14990	6,85×

Bảng 4.10. So sánh với các nghiên cứu tiên tiến dựa trên kết quả sau khi triển khai thành công CRYSTALS-Kyber Accelerator trên FPGA

Work	Method	Mode	Area						Freq. [MHz]	Key/Enc/Dec Cycles [K]	Key/Enc/Dec Time [us]	ADP [x10 ⁶]
			LUT	FF	BRAM	DSP	SLICE	ENS				
[13]	HW/SW	Key.& Enc.& Dec.	15K	3K	14	11	4000	7844	25	75 / 132 / 142	3000/ 5280/ 5680	23.5/41.4 /44.6
[14]	HW/SW		24K	11K	32	21	-	-	-	150 / 193 / 205	-	-
[4]	HW	Enc.	80K	-	200.5	54	142K	158K	155	- / 49 / 68.8	- / 316 / 444	- / 21/43.3
		Dec.	202K	-	202	354	153K	217K				
[5]	HW (HLS)	Enc.	64K	30K	77	102	16K	41340	98	- / 10.6 / 13	- / 108/132.65	- / 4.46/5.3
		Dec.	56K	27K	62	138	14K	39979				
[6]	HW	Enc.	4.8K	3K	2.5	0	1395	1885	244	- / - / -	278/ 416/ 552	0.54/ 0.78 /1.07
		Key.& Dec.	4.9K	3K	2.5	0	1452	1942				
[7]	HW	Key.& Enc.& Dec.	5K	2K	6.5	2	1317	2791	258	1148 / 1236 / 1172	4593/ 4944/ 4691	12.8/ 13.8 /13.1
[15]	HW		18K	5K	15	6	5000	8540	115	4 / 7 / 10	34.8/ 60.9/ 86.9	0.30/ 0.52 /0.74
This Work	HW		19K	8K	5	0	6145	7321	167	7.78 / 10.01 / 13.74	46.7 / 60.1 / 82.4	0.34/ 0.44 /0.60

Một số công thức cần dùng cho giá trị được tính ở trên như sau: thời gian thực thi $Time = Cycle/Freq.$ $ENS = SLICE + 100 \times DSP + 196 \times BRAM$. Chỉ số $ADP = ENS \times Time/10^6$ (dùng đánh giá độ cân bằng tài nguyên/độ trễ).

Từ Bảng 4.10 có thể thấy thiết kế Accelerator của chúng tôi vượt trội về tốc độ tính toán, đảm bảo thời gian thực thi nhanh với thời gian tương ứng cho từng chế độ KeyGen, Encapsulation và Decapsulation lần lượt là 46.7 μs , 60.1 μs và 82.4 μs . So với các thiết kế HW/SW như [13],[14], thời gian xử lý của thiết kế này thấp hơn rất nhiều, do các phép toán nặng trong Kyber đã được đưa sang phần cứng chuyên dụng. Khi so với các thiết kế phần cứng khác, đề tài vẫn giữ được tốc độ khá tốt trong khi chỉ sử dụng 5 BRAM và không dùng DSP. Thiết kế cũng cho thấy giá trị to lớn của việc chia sẻ tài nguyên dùng chung đa chế độ khi tối ưu tài nguyên hơn rất nhiều so với cách tiếp cận thuần phần cứng [4], [5], [7]. Đặc biệt, so với thiết kế nhẹ [6], mặc dù đạt được tối ưu cực lớn về mặt diện tích nhưng đề xuất này đang tối ưu hơn về mặt ADP. So với [15], thiết kế có ADP KeyGen hơi cao hơn, nhưng ở Encapsulation và Decapsulation lại cho kết quả ADP thấp hơn. Điều này cho thấy đề xuất đạt được mức cân bằng tốt giữa hiệu năng và tài nguyên phần cứng khi triển khai trên FPGA.

CHƯƠNG 5

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. KẾT LUẬN

Khóa luận đã hoàn thành việc thiết kế và hiện thực một vi điều khiển sử dụng lõi RISC-V RV32I, tích hợp khối tăng tốc phần cứng cho thuật toán CRYSTALS-Kyber trên FPGA. Hệ thống bao gồm CPU, IMEM/SRAM, bus Wishbone, UART, GPIO, I2C, Timer, PIC và Kyber Accelerator, cho phép CPU điều khiển các ngoại vi và khối mật mã thông qua cơ chế MMIO.

Khối Kyber Accelerator hỗ trợ các thao tác chính của ML-KEM gồm tạo khóa, mã hóa và giải mã. Các khối chức năng chính được tổ chức theo hướng chia sẻ tài nguyên để giảm phần cứng lặp lại và tăng hiệu quả xử lý.

Thiết kế đã được kiểm thử ở nhiều mức, từ mô phỏng từng module, mô phỏng tích hợp SoC đến kiểm tra CPU điều khiển ngoại vi qua Wishbone. Kết quả phần cứng Kyber được so sánh với mô hình tham chiếu C và cho thấy các dữ liệu như public key, secret key, ciphertext và shared secret đều khớp với giá trị kỳ vọng.

Tổng thể, đề tài đã chứng minh khả năng tích hợp CRYSTALS-Kyber vào một hệ thống vi điều khiển RISC-V hoàn chỉnh. Thiết kế vừa có khả năng xử lý thuật toán mật mã bằng phần cứng, vừa hỗ trợ ngoại vi và cơ chế nạp chương trình cần thiết cho một vi điều khiển.

5.2. Hướng phát triển

Trong tương lai, thiết kế có thể tiếp tục được tối ưu ở các khối chiếm nhiều tài nguyên như Hash Core, NTT/INTT, PWMA và bộ nhớ trung gian nhằm giảm LUT/FF, giảm BRAM và cải thiện tần số hoạt động. Hay có thể giảm độ phức tạp thuật toán để hardware-friendly hơn, từ đó sẽ đạt được những bước ngoặt lớn.

Bên cạnh đó, hệ thống kiểm thử cũng cần được mở rộng với nhiều KAT, random test và regression test tự động, xây dựng môi trường mô phỏng chuẩn chỉnh hơn để tự động phát hiện lỗi và đạt được độ bao phủ cao ở cả code và function:

- Tăng số lượng test vector và chia thành nhiều batch regression tự động.
- Bổ sung assertion cho các giao thức điều khiển, RAM và bus giao tiếp.

Một điều quan trọng khác là bổ sung TRNG để tạo seed ngẫu nhiên trực tiếp trên phần cứng, thay vì phụ thuộc vào seed từ phần mềm hoặc testbench. Ngoài ra, vi điều khiển cũng cần được mở rộng với DMA, SPI, bộ nhớ ngoài hoặc bus AXI để tăng khả năng truyền dữ liệu và tích hợp hệ thống, cụ thể:

- Tích hợp khối TRNG để bộ tăng tốc được hoàn chỉnh, sau đó kiểm tra nghiêm ngặt hơn với các bài kiểm tra Repetition Count Test và Adaptive Proportion Test, ước lượng min-entropy.
- Dùng DMA để giảm tải cho CPU khi truyền dữ liệu.
- Tích hợp SPI hoặc bộ nhớ ngoài để lưu trữ nhiều khóa và ciphertext hơn.
- Mở rộng bus AXI theo hướng dữ liệu rộng hơn hoặc hỗ trợ burst transfer.

Sau khi tối ưu và xác minh đầy đủ trên FPGA, thiết kế cũng có thể được mở rộng để hỗ trợ ML-KEM-768, ML-KEM-1024. Đánh giá lại diện tích, tốc độ, công suất khi tiến tới đánh giá khả năng triển khai sang ASIC.

TÀI LIỆU THAM KHẢO

- [1] Shor, Peter W. "Quantum computing." *Documenta Mathematica* 1, no. 1000 (1998): 467-486.
- [2] Crescenzo, Diamante Simone, Rafael Carrera Rodriguez, Riccardo Alidori, Florent Brugière, Emanuele Valea, Pascal Benoit, and Alberto Bosio. "Hardware accelerator for FIPS 202 hash functions in post-quantum ready SoCs." In *2024 IEEE 30th International Symposium on On-Line Testing and Robust System Design (IOLTS)*, pp. 1-6. IEEE, 2024.
- [3] Nannipieri, Pietro, Stefano Di Matteo, Luca Zulberti, Francesco Albicocchi, Sergio Saponara, and Luca Fanucci. "A RISC-V post quantum cryptography instruction set extension for number theoretic transform to speed-up CRYSTALS algorithms." *IEEE Access* 9 (2021): 150798-150808.
- [4] Wu-Yiming-Huang, Miaoqing-Huang, Zhongkui-Lei and Jiaxuan, "A Pure Hardware Implementation of CRYSTALS-KYBER PQC Algorithm through Resource Reuse," *IEICE Electronics Express*, vol. advpub, p. 17.2020023, 2020.
- [5] Soni-Deepraj and Karri-Ramesh, "Efficient Hardware Implementation of PQC Primitives and PQC algorithms Using High-Level Synthesis," *2021 IEEE Computer Society Annual Symposium on VLSI*, pp. 296-301, 2021.
- [6] He-Ma-Shiyang, Hui-Li, Fenghua-Li and Ruhui, "A lightweight hardware implementation of CRYSTALS-Kyber," *Journal of Information and Intelligence*, vol. 2, no. 2, 949-7159, p. 2, 2024.
- [7] Arpan-Jati, Naina-Gupta, Anupam-Chattopadhyay and Kumar-Sanadhya-Somitra, "A Configurable CRYSTALS-Kyber Hardware Implementation with Side-Channel Protection," *ACM Trans. Embed. Comput. Syst.*, vol. 23, no. 1539-9087, p. 25, 2024.
- [8] Avanzi, R., Bos, J., Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., Schanck, J. M., Schwabe, P., Seiler, G., & Stehlé, D. (2017). CRYSTALS-Kyber Algorithm Specifications and supporting documentation. Available: <https://nist.gov>

- [9] A. Waterman and K. Asanović, “The RISC-V instruction set manual, vol. I, ver. 2.2”, SiFive Inc. and EECS Department, University of California, Berkeley, 2017. [Online]. Available: riscv.org.
- [10] Wade D. Peterson, “WISHBONE System-on-Chip (SoC) Interconnection Architecture for Portable IP Cores”, WISHBONE Classic Bus Cycle — WISHBONE B3, 2019.
- [11] “I2C Master – WISHBONE Compatible”, I2C Master - WISHBONE Compatible | Lattice Reference Design, 15/1/2015.
- [12] Bertoni-Guido, Daemen-Joan, Hoffer-Seth, Peeters-Michaël, Assche-Gilles-Van and V.-R. Keer, "Team Keccak," Team Keccak, [Online]. Available: <https://keccak.team>. [Accessed 31 Tháng 5 2026].
- [13] U. Banerjee, T. S. Ukyab, and A. P. Chandrakasan, “Sapphire: A configurable crypto-processor for post-quantum lattice-based protocols (extended version),” in Proc. IACR, 2019, p. 1140.
- [14] Fritzmann, Tim, Georg Sigl, and Johanna Sepúlveda. "RISQ-V: Tightly coupled RISC-V accelerators for post-quantum cryptography." IACR Transactions on Cryptographic Hardware and Embedded Systems (2020): 239-280.
- [15] Bisheh-Niasar-Mojtaba, Azarderakhsh-Reza and Mozaffari-Kermani-Mehran, "Instruction-Set Accelerated Implementation of CRYSTALS-Kyber," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 68, no. 11, pp. 4648-4659, 2021.